RFC 6455 · STOMP 1.2 · SPRING · STOMPJS

The WebSocket + STOMP Book

From how computers talk to running real-time messaging at company scale: the protocols byte by byte, Spring and client code, scaling, security, reliability, a bug encyclopedia, and interview preparation from entry level to architect.

```
CONNECT
accept-version:1.2
host:chat.example.com
heart-beat:10000,10000

^@
```

13 chapters + **3** appendices **≈55k** words **230+** interview questions

85+ bugs by symptom Checked against source, October 2026

Contents

Start here

How this book is organised · The whole story in one paragraph (the 5-year-old version) · Conventions used in this book · Sources this book is checked against

1 ### Foundations

ELI5: computers sending letters · The layers (just enough networking) · HTTP: the request/response model · The real-time problem · Why do we need STOMP on top of WebSocket? · Publish/subscribe and messaging vocabulary · Stateful vs stateless servers (why WebSocket is harder) · Chapter summary

2 ### The WebSocket protocol

ELI5 · History and standards · The opening handshake · Base framing protocol (the heart of WebSocket) · Control frames in detail · Extensions and sub-protocols · WebSocket over HTTP/2 and HTTP/3 · Connection lifecycle: the full picture · Limits, intermediaries and the real world · Writing a WebSocket by hand (to prove you understand it) · Tools to inspect WebSocket traffic · Chapter summary

3 ### Browser API

ELI5 · The API surface (complete) · A correct minimal client · The five big limitations (and their work-arounds) · Reconnect done right · Many tabs, one connection · Back/forward cache, sleep and mobile · Mixed content and secure contexts · Server-side JavaScript (Node.js) quick reference · Chapter summary

4 ### The STOMP protocol

ELI5 · What STOMP is (and isn't) · Frame anatomy · Connecting · Heart-beating (connection liveness) · Sending: SEND · Subscribing: SUBSCRIBE / UNSUBSCRIBE · Receiving: MESSAGE · Acknowledgement modes: auto, client, client-individual · Receipts: RECEIPT · Errors: ERROR · Transactions: BEGIN / COMMIT / ABORT · Disconnecting gracefully: DISCONNECT · Frames headers quick reference table · A complete annotated session · Write a tiny STOMP client by hand (to truly understand it) · STOMP vs other messaging protocols · Strengths and weaknesses of STOMP · Chapter summary

5 ### Spring server

ELI5 · Dependencies · The minimal working configuration · The architecture: channels and handlers · Annotated controllers · Sending from anywhere: SimpMessagingTemplate · Destinations and patterns · Configuring the broker in detail · Transport and container configuration · User destinations: private messages explained · Session lifecycle events · Interceptors · Customising STOMP ERROR frames · Message ordering · Message converters · Monitoring: WebSocketMessageBrokerStats · Plain WebSocket in Spring (without STOMP), for completeness · Complete example: a real-time to-do list server · Chapter summary

6 ### Clients

ELI5 · Choosing a JavaScript library · @stomp/stompjs v7: complete guide · Legacy API (recognise it in old code) · SockJS fallback · React (web) · React Native (this project's platform!) · Angular with rx-stomp · Vue / Svelte / vanilla · Java client (WebSocketStompClient) · Other platforms · Manual testing with raw tools · Client-side best practices checklist

7 ### Brokers & scaling

ELI5 · Simple broker vs broker relay · How the broker relay works internally · User destinations in a cluster · RabbitMQ in depth · ActiveMQ Classic and ActiveMQ Artemis · The backplane pattern (simple broker + Redis/Kafka) · Capacity planning · Load balancers and reverse proxies · Deployments without pain · Managed real-time services (when not to build it yourself) · Chapter summary

8 **Security**

ELI5 · Threat model (memorise this list) · Transport security: wss:// · Cross-Site WebSocket Hijacking (CSWSH): the #1 WebSocket-specific attack · Authentication options · Authorisation of messages with Spring Security · Token expiry and revocation while connected · Denial-of-service protections · Input validation and output encoding · Logging and privacy · Security checklist (copy into your PR template)

9 **Reliability & performance**

ELI5 · Delivery guarantees: what STOMP over WebSocket really gives you · Ordering · Heart-beats and timeouts: tuning table · Slow consumers and backpressure · Thread pools and blocking · Payload size and format · Observability: what to measure · Load testing · Graceful degradation · Reliability checklist

10 **Testing & bug encyclopedia**

ELI5 · Testing strategy (the pyramid) · The debugging method: layer by layer · The Bug Encyclopedia (symptom → cause → fix) · What do I do when... playbook · Useful log/config snippets

11 **Architecture & practices**

ELI5 · Decision framework: should this feature be real-time at all? · Reference architecture for a company · Destination naming conventions (write them down!) · Message envelope and versioning · Design patterns catalogue · Worked designs (interview-ready) · Organisational best practices · Anti-patterns (and what to do instead) · Migration guides · Chapter summary

12 **Interview Q&A, part 1**

A. Fundamentals comparisons · B. WebSocket protocol (RFC 6455) · C. Browser API clients · D. STOMP protocol · E. Spring WebSocket / STOMP

12B **Interview Q&A, part 2**

F. Security · G. Brokers, clustering scaling · H. Reliability performance · I. Debugging scenario questions · J. System design questions (with answer outlines) · K. Coding / whiteboard tasks · L. Behavioural / experience questions (prepare your stories) · M. Rapid-fire (one-liners) · N. Tricky gotcha questions · O. More questions (added after a research pass on common interview topics) · P. Common wrong answers you will find online (don't repeat them)

13 **Cheat sheet & glossary**

One-page cheat sheet · Glossary · Study plan

APPENDIX A **Other stacks**

ELI5 · Jakarta WebSocket (JSR 356) @ServerEndpoint · Spring's raw WebSocketHandler API (servlet stack) · Spring WebFlux reactive WebSocket · RSocket (application protocol with reactive semantics) · Spring Session integration (keep HTTP sessions alive) · A Kafka-backed real-time gateway (complete pattern) · WebSocket over HTTP/2 and HTTP/3 in practice · Node.js · Socket.IO in depth (and how it differs from STOMP) · ASP.NET Core SignalR · Go · GraphQL subscriptions over WebSocket · CRDT-based sync (collaborative editing): Yjs · MQTT over WebSocket · The big comparison table · Summary

APPENDIX B **Runnable sample**

What it demonstrates (and where the book explains it) · Prerequisites · Run it · Use it from this React Native app · Relay mode (RabbitMQ) · Things deliberately left simple (do these in production) · Two production gotchas this sample uncovered (now in the book)

APPENDIX C

Verification & sources

How it was checked · Facts confirmed against source code / docs sources · Behaviour verified by running the Appendix B sample (Spring Boot 4.1.1 / Spring Framework 7.0.9 / Spring Security 7, stompjs 7.5.0, Node 22) · Facts confirmed by web search (secondary sources; re-check provider docs for your account/region) · Not independently re-verified in this pass · Sources

INTRODUCTION

The WebSocket + STOMP Book

From "what is a network?" to running real-time messaging at company scale, plus interview prep

Who is this for? Everyone. If you have never heard the word "socket", start at Chapter 1. If you already ship Spring/STOMP to production, jump to Chapters 6–10. If your interview is tomorrow, read Chapter 12 (Interview Questions, both parts) and Chapter 13 (Cheat Sheet).

How this book is organised

#	Chapter	What you will be able to do after reading it
1	Foundations: How computers talk	Explain TCP, HTTP, request/response, polling, long polling, SSE, and *why* WebSocket was invented
2	The WebSocket Protocol (RFC 6455), deep dive	Draw the handshake, decode a frame byte by byte, explain masking, fragmentation, ping/pong, close codes, extensions, subprotocols, WebSocket over HTTP/2 and HTTP/3
3	The Browser & JavaScript WebSocket API	Use `new WebSocket()` correctly, know its limits (no headers, no ping), handle reconnects and backpressure
4	The STOMP Protocol (1.0 / 1.1 / 1.2), deep dive	Write STOMP frames by hand, explain every command and header, heart-beats, ack modes, receipts, transactions, escaping
5	Spring WebSocket + STOMP: Server side	Build a production Spring Boot STOMP server; understand channels, annotations, `SimpMessagingTemplate`, user destinations, events, configuration
6	Clients: JavaScript, React, React Native, Angular, Java, Python, SockJS	Connect any client correctly, including the React Native gotchas
7	Message Brokers, Clustering & Scaling	Choose simple broker vs relay, configure RabbitMQ/ActiveMQ, scale horizontally, load balancers, Kubernetes
8	Security	Authenticate (JWT, cookies), authorise destinations, stop CSWSH, CSRF, DoS, data leaks
9	Reliability, Performance & Tuning	Heart-beats, timeouts, backpressure, thread pools, ordering, delivery guarantees, idempotency, monitoring
10	Testing, Debugging & the Bug Encyclopedia	Test everything, debug with tools, and fix 60+ real-world bugs by symptom
11	Architecture, Design Patterns & Organisational Best Practices	Design chat, notifications, dashboards, collaboration; run it in a big company; runbooks

#	Chapter	What you will be able to do after reading it
12	Interview Questions & Answers, Part 1: fundamentals, WebSocket, browser, STOMP, Spring	Answer the core technical questions with confidence
12b	Interview Questions & Answers, Part 2: security, scaling, reliability, debugging scenarios, system design, coding, rapid-fire, gotchas	Handle senior-level, scenario and design rounds
13	Cheat Sheet & Glossary	Look anything up in seconds
A	Appendix A: Beyond Spring STOMP: Jakarta WebSocket, raw Spring handlers, WebFlux, RSocket, Spring Session, Kafka gateway, HTTP/2, Node, Socket.IO, SignalR, Go, GraphQL subscriptions, Yjs, MQTT	Compare and integrate other stacks
B	Appendix B: Runnable sample project: Spring Boot 4 server + TypeScript/React Native client, 11 passing integration tests	Run, read and adapt working code
C	Appendix C: Verification notes & sources	See what was checked, against what

The whole story in one paragraph (the "5-year-old" version)

Imagine you and your friend live in two houses. **HTTP** is like sending a letter: you write a question, post it, and wait for a reply. If you want to know whether your friend has news, you must keep sending letters ("Any news? Any news? Any news?"). That is slow and wasteful.

WebSocket is like installing a **telephone line** between the two houses. You call once (the *handshake*), and after that either of you can talk at any moment, as long as the line stays open.

But a telephone line only carries *sounds*. You still need to agree on *a language*, for example "When I say *SUBSCRIBE kitchen*, tell me every time something happens in the kitchen." **STOMP** is that language. It is a small, simple set of words (`CONNECT`, `SUBSCRIBE`, `SEND`, `MESSAGE` ...) spoken over the WebSocket telephone line.

Finally, if lots of people want to hear about the kitchen, you need a **post office** that copies each kitchen message to everyone who asked. That is the **message broker** (Spring's simple broker, RabbitMQ, ActiveMQ...).

```
Telephone line   = WebSocket  (transport: moves bytes both ways, any time)
Language         = STOMP      (protocol: what the bytes mean - commands, destinations)
Post office      = Broker     (routing: who gets which message)
Your house       = Client     (browser, mobile app, Java service)
Friend's house   = Server     (Spring Boot app, etc.)
```

Keep this picture in your head. Every chapter adds detail to one of these five boxes.

Conventions used in this book

- **ELI5** boxes: a child-friendly explanation before the technical one.
- **Deep dive** boxes: the exact details (byte layouts, spec rules, framework internals).
- **Pitfall** boxes: things that bite people in production.

- **Interview tip** boxes: how this shows up in interviews.
- Code is mostly **Java 17+ / Spring Boot 3.x–4.x / Spring Framework 6.x–7.x** on the server and **TypeScript / `@stomp/stompjs` v7** on the client, because that is the most common enterprise stack. Concepts apply to any language.
- "Spec" means the official STOMP 1.2 specification (stomp.github.io) or RFC 6455 (WebSocket).

Sources this book is checked against

- RFC 6455: *The WebSocket Protocol* (IETF, December 2011)
- RFC 7692: *Compression Extensions for WebSocket* (permessage-deflate)
- RFC 8441: *Bootstrapping WebSockets with HTTP/2*; RFC 9220: *Bootstrapping WebSockets with HTTP/3*
- WHATWG *WebSockets Standard* (the browser API)
- *STOMP Protocol Specification, Version 1.2* (and 1.1, 1.0), stomp.github.io
- Spring Framework Reference: "WebSockets" chapter (WebSocket API, SockJS, STOMP)
- Spring Security Reference: "WebSocket Security"
- `@stomp/stompjs` documentation (stomp-js.github.io)
- RabbitMQ STOMP and Web STOMP plugin documentation
- OWASP WebSocket Security Cheat Sheet

Library APIs change over time. Where something differs between versions, the book says which version. Before you rely on an exact default value in production, confirm it against the version you run.

CHAPTER 1

Chapter 1: Foundations, or How Computers Talk

Goal of this chapter: understand the ground WebSocket stands on. Everything later (handshakes, frames, heart-beats, load balancer timeouts) makes sense only after this.

1.1 ELI5: computers sending letters

Think of the internet as a giant postal system.

- Every computer has an **address** (an *IP address*, like `142.250.183.14`).
- Each computer runs many programs, so letters also need a **room number** (a *port*, like `443`).
- An address plus a port (`142.250.183.14:443`) is like "House 14, Room 443".

When your phone opens a web page, it is sending a letter to "House X, Room 443" saying "please give me the page", and the server sends a letter back.

1.2 The layers (just enough networking)

Networking is built in **layers**. Each layer trusts the one below it and offers something nicer to the one above.

```
+-------------------------------------------------------------+
| Application layer   HTTP, WebSocket, STOMP, MQTT, SMTP ...  |  <- what this book is about
+-------------------------------------------------------------+
| Security layer      TLS (makes https:// and wss://)          |
+-------------------------------------------------------------+
| Transport layer     TCP (reliable, ordered)  /  UDP (fast)   |
+-------------------------------------------------------------+
| Internet layer      IP (addresses, routing)                  |
+-------------------------------------------------------------+
| Link layer          Wi-Fi, Ethernet, 4G/5G                   |
+-------------------------------------------------------------+
```

1.2.1 IP: the address on the envelope

IP moves *packets* from one computer to another. Packets can get lost, duplicated, or arrive out of order. IP does not care.

1.2.2 TCP: the reliable, ordered pipe

TCP (Transmission Control Protocol) makes IP's messy packets look like a clean, ordered **stream of bytes**:

- **Connection-oriented.** A *three-way handshake* (`SYN` → `SYN-ACK` → `ACK`) opens a connection.

- **Reliable.** Lost packets are re-sent.
- **Ordered.** Bytes arrive in the order they were sent.
- **Flow control.** A receiver advertises a *window*: "I can take N more bytes." If the receiver is slow, the sender must wait. This becomes **backpressure** (Chapter 9).
- **Congestion control.** The sender slows down if the network is congested.
- **Full-duplex.** Both sides can send at the same time.

Key point: TCP is a **byte stream**, not a message stream. If you send "HELLO" then "WORLD", the receiver might read "HELLOWOR" and then "LD". Anything built on TCP must define its own **message boundaries** (framing). WebSocket does this with *frames*, and STOMP adds its own framing on top with a `NULL` byte terminator.

Pitfall: TCP "half-open" connections. If a phone loses signal or a laptop goes to sleep, the other side is **not told**. The server still thinks the connection is open until it tries to send and fails, or until a timeout fires. That is why WebSocket has **ping/pong** and STOMP has **heart-beats**. You will see this problem again and again in this book.

1.2.3 UDP

Fire-and-forget packets with no ordering or reliability. Used by games, video calls (WebRTC), and QUIC (which HTTP/3 runs on). Classic WebSocket runs on **TCP**, not UDP.

1.2.4 TLS: the locked envelope

TLS encrypts the TCP stream and proves the server's identity using certificates. `https://` = HTTP over TLS. `wss://` = WebSocket over TLS. In production **always use** `wss://` (Chapter 8 explains why; also, many proxies break unencrypted WebSocket traffic).

1.3 HTTP: the request/response model

HTTP works like a waiter in a restaurant:

1. The client (you) sends a **request**: `GET /menu`.
2. The server (the kitchen) sends a **response**: `200 OK` plus the menu.
3. The waiter does not come back unless you call again.

HTTP

```
GET /api/orders/42 HTTP/1.1
Host: shop.example.com
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 27

{"id":42,"status":"PACKED"}
```

Important properties:

Property	Meaning	Why it matters for real-time
Client-initiated	Only the client can start a conversation	The server **cannot push** "your order shipped!" on its own
Stateless	Each request stands alone	Every request repeats headers (cookies, auth), which costs bytes
Header overhead	Often 500 bytes to several KB per request	Bad for tiny, frequent updates
Half-duplex per request	Request goes, then response comes	No simultaneous two-way talk within one exchange

HTTP versions in one table

Version	Year	Key idea	Relation to WebSocket
HTTP/1.0	1996	New TCP connection per request	–
HTTP/1.1	1997/1999 (RFC 2068/2616; now RFC 9112)	Keep-alive, `Upgrade` header	**The classic WebSocket handshake uses HTTP/1.1** `Upgrade`
HTTP/2	2015 (RFC 7540, now RFC 9113)	Multiplexed streams on one TCP connection, binary framing	WebSocket over HTTP/2 via RFC 8441 (extended CONNECT)
HTTP/3	2022 (RFC 9114)	Runs on QUIC (UDP)	WebSocket over HTTP/3 via RFC 9220

1.4 The real-time problem

"Real-time" on the web means **the server needs to tell the client something the moment it happens**:

- A chat message arrives.
- A stock price changes.
- A delivery driver moves on the map.
- Someone edits the shared document.
- A long job finishes.
- A to-do item gets assigned to you by a teammate.

HTTP alone cannot do this because **the server cannot start a conversation**. People invented several workarounds. You must know all of them for interviews.

1.4.1 Short polling

ELI5: a kid in the back seat asking "Are we there yet?" every 5 seconds.

JS

```
setInterval(async () => {
  const res = await fetch('/api/notifications?since=' + lastId);
```

```
  render(await res.json());
}, 5000);
```

- **Pros:** dead simple; works everywhere; stateless servers; easy to cache and load balance.
- **Cons:** latency up to the interval; most responses are empty ("No, not yet"); wasted CPU, bandwidth and battery; thousands of clients become thousands of requests per second.

1.4.2 Long polling

> **ELI5:** You ask "Are we there yet?" and the parent **doesn't answer until we arrive** (or until a minute passes). Then you immediately ask again.

1. Client sends a request.
2. Server **holds** it open until there is data or a timeout (for example 30 s).
3. Server responds; the client immediately sends a new request.

- **Pros:** near-real-time; works over plain HTTP and through most proxies.
- **Cons:** one held request per client; a full HTTP request/response per message; message ordering and loss in the gap between requests must be handled (use a cursor like `since=lastId`); timeouts on proxies; tricky at scale.
- Historically called **Comet** (along with HTTP streaming). SockJS and CometD use it as a fallback.

1.4.3 HTTP streaming / Server-Sent Events (SSE)

> **ELI5:** The parent keeps talking on a one-way walkie-talkie: "We're at the bridge… now the tunnel… now the shop." You can listen, but to talk back you must use something else.

SSE (`EventSource` in browsers, `Content-Type: text/event-stream`) keeps one HTTP response open and the server writes events into it:

```
event: orderUpdated
id: 1001
data: {"id":42,"status":"SHIPPED"}
```

- **Pros:** simple; plain HTTP; **automatic reconnect** with `Last-Event-ID` built into the browser API; works nicely with HTTP/2 multiplexing; good for one-way feeds (notifications, dashboards, LLM token streaming).
- **Cons: server → client only**; text only (UTF-8); the browser `EventSource` API cannot set custom headers (no `Authorization` header); over HTTP/1.1 browsers limit concurrent connections per domain (commonly 6), so many tabs can exhaust it (HTTP/2 fixes this).

1.4.4 WebSocket

> **ELI5:** A telephone call. One call, then both people talk whenever they want.

- One HTTP request **upgrades** to a persistent, **full-duplex** connection.
- After the handshake, messages carry only **2–14 bytes** of framing overhead (Chapter 2).
- Both sides can send at any time.
- Text and binary data.

1.4.5 Other options you should be able to name

Technology	One-liner
WebRTC DataChannel	Peer-to-peer (browser ↔ browser), runs over SCTP/DTLS/UDP, can be unreliable/unordered; needs signalling (often done *via* WebSocket!)
WebTransport	Newer API over HTTP/3 (QUIC); multiple streams + unreliable datagrams; no head-of-line blocking. Browser support is still growing.
HTTP/2 Server Push	**Not** a real-time messaging tool; it pushed resources (CSS/JS) into the cache. Chrome removed it (2022). Never answer "use HTTP/2 push for chat" in an interview.
gRPC streaming	Bi-directional streaming over HTTP/2; great service-to-service; in browsers needs gRPC-Web, which does not support client streaming / bidi streaming.
MQTT	Lightweight pub/sub protocol for IoT; can run over WebSocket (MQTT over WS).
Socket.IO	A library with its **own protocol** (Engine.IO) that *may use* WebSocket. A plain WebSocket client **cannot** talk to a Socket.IO server.
SignalR	Microsoft's real-time library for .NET; negotiates WebSocket → SSE → long polling.
Push notifications (FCM/APNs)	OS-level push to mobile devices when the app is in background; complements, not replaces, WebSocket.

1.4.6 The comparison table (memorise this)

	Short polling	Long polling	SSE	WebSocket
Direction	Client pull	Client pull (server delays)	Server → client	**Both ways**
Latency	Up to interval	Low	Low	**Lowest**
Overhead per message	Full HTTP req+resp	Full HTTP req+resp	Small (event text)	**Tiny (2–14 bytes frame header)**
Connection	New/reused per poll	Held, re-opened per message	One long response	One persistent connection
Binary data	Yes (HTTP body)	Yes	No (text only)	Yes
Auto reconnect	N/A	Manual	**Built-in**	**Manual** (or library)
Proxy/firewall friendliness	Best	Good	Good	Usually good with `wss://`; old proxies can break it
Server complexity	Lowest	Medium	Low–medium	Higher (stateful connections)
Typical use	Rare updates	Fallback	Feeds, notifications, AI token streams	Chat, games, collaboration, trading, live tracking

Interview tip: "When would you NOT use WebSocket?" When updates are rare (polling is simpler), when data flows only server → client (SSE is simpler and auto-reconnects), when you need offline/background delivery on mobile (push notifications), when responses must be cacheable by CDNs (HTTP), or when the team cannot afford stateful infrastructure.

1.5 Why do we need STOMP on top of WebSocket?

WebSocket gives you a pipe that carries **messages** (text or binary). That's it. It does **not** say:

- What does a message *mean*? Is it a chat message, a subscription request, an error?
- **Where** is it going? (No concept of "topic" or "destination".)
- How do I **subscribe** to only some kinds of messages?
- How do I know the server **received** my message? (No receipts/acks.)
- How do I send **headers/metadata** (content type, correlation ID)?
- How does a **broker** route one message to 10,000 subscribers?

Without a sub-protocol, every team invents its own ad-hoc JSON format:

JSON

```
{"type":"subscribe","channel":"room-1"}
{"type":"chat","channel":"room-1","text":"hi"}
```

...and then re-invents acks, errors, heart-beats and routing, often badly. **STOMP** (Simple Text Oriented Messaging Protocol) is a standard, tiny, text-based messaging protocol that already defines all of this:

```
SUBSCRIBE
id:sub-0
destination:/topic/room-1

^@
```

Because it's a standard:

- Off-the-shelf **clients** exist (stompjs, Java, Python, Go, .NET, Ruby...).
- Off-the-shelf **brokers** speak it (RabbitMQ, ActiveMQ Classic/Artemis...).
- Frameworks like **Spring** can route messages to annotated methods (`@MessageMapping`) just like `@RequestMapping` for HTTP.

```
+----------------------------+
| Your app's messages (JSON) |   e.g. {"text":"hi"}
+----------------------------+
| STOMP frame                |   SEND / destination:/app/chat / body
+----------------------------+
| WebSocket frame            |   opcode=text, FIN=1, masked payload
+----------------------------+
| TLS                        |   encrypted bytes
+----------------------------+
| TCP                        |   ordered reliable byte stream
+----------------------------+
```

1.6 Publish/subscribe and messaging vocabulary

Before STOMP, learn the words messaging people use:

Term	ELI5	Technical
Message	A letter	A unit of data with headers + body
Producer / Publisher	Person who writes letters	Component that sends messages
Consumer / Subscriber	Person who reads letters	Component that receives messages
Destination	The name on the mailbox	A string identifying where a message goes, e.g. `/topic/prices`
Topic (pub/sub)	A radio station: everyone tuned in hears every song	One message → **all** current subscribers
Queue (point-to-point)	A ticket counter: each customer is served by one clerk	One message → **one** consumer (competing consumers)
Broker	The post office	Middleware that stores and routes messages
Subscription	Telling the post office "send me kitchen letters"	Registration of interest in a destination
Ack (acknowledgement)	"Got it!"	Consumer confirms processing so the broker can forget the message
Durable	Mailbox that keeps letters while you're on holiday	Messages/subscriptions survive disconnects/restarts
Fan-out	Photocopying one letter for everyone	Delivering one message to many subscribers
At-most-once / at-least-once / exactly-once	Maybe / definitely, maybe twice / exactly once	Delivery guarantees (Chapter 9)

Important: In STOMP, the spec says destination strings are **opaque** and broker-specific. `/topic/...` and `/queue/...` are only *conventions*. Spring's simple broker treats every destination as pub/sub; RabbitMQ gives `/queue/` and `/topic/` real, different meanings (Chapter 7).

1.7 Stateful vs stateless servers (why WebSocket is "harder")

A REST server can be **stateless**: any request can go to any server instance. A WebSocket server is **stateful**: each server instance holds **open connections** to specific clients.

Consequences (all covered later):

1. **Load balancing** must support long-lived connections and the `Upgrade` mechanism.
2. **Scaling out**: if user A is connected to server 1 and user B to server 2, a message from A to B must cross servers → you need a **broker** or a pub/sub backplane (Chapter 7).
3. **Deployments** drop connections → clients must **reconnect** gracefully (Chapter 9).
4. **Memory**: each connection costs memory (buffers, session objects).

5. **Timeouts**: proxies close "idle" connections → **heart-beats** are required.
6. **Observability**: "requests per second" is no longer the only metric; you track *concurrent connections*, *messages/sec*, *send buffer sizes*, etc.

1.8 Chapter summary

- TCP is a reliable byte stream; it needs framing and can silently half-open.
- HTTP is request/response, client-initiated, header-heavy, so not great for server push.
- Workarounds: short polling, long polling, SSE. Each has trade-offs.
- WebSocket = one upgraded HTTP connection that becomes a full-duplex message channel with tiny overhead.
- STOMP = a standard messaging language on top of WebSocket (destinations, subscribe, ack, receipts, errors, heart-beats).
- Brokers route messages: topics (fan-out) vs queues (one consumer).
- WebSocket servers are stateful, and that drives most of the hard problems in production.

Next: Chapter 2: The WebSocket Protocol

CHAPTER 2

Chapter 2: The WebSocket Protocol (RFC 6455), Deep Dive

Goal: know WebSocket at the wire level, well enough to read a Wireshark capture, explain every byte of a frame, and answer any protocol question in an interview.

2.1 ELI5

1. You **knock** on the server's door using a normal HTTP letter that says "*Can we switch to telephone mode?*" (the **opening handshake**).
2. The server answers "*101: Yes, switching to telephone mode*" and does a small math trick to prove it really understands telephones (the `Sec-WebSocket-Accept` value).
3. From now on, you talk in small **packets called frames.** Each frame has a tiny label ("this is text", "this is the last piece", "this is a goodbye").
4. Every so often one side says "**ping?**" and the other says "**pong!**" to check the line is alive.
5. To hang up politely, one side sends a **close frame** with a reason code, the other answers with a close frame, and the line is closed.

2.2 History and standards

Date	Event
2008	"TCPConnection" idea in HTML5 drafts; renamed WebSocket
2009–2010	Browsers ship draft versions (hixie-75, hixie-76, hybi-xx), some with security problems
Dec 2011	**RFC 6455 published.** Version `13`. This is "WebSocket" today.
Dec 2015	RFC 7692: `permessage-deflate` compression extension
Sep 2018	RFC 8441: WebSocket over HTTP/2 (extended CONNECT)
Jun 2022	RFC 9220: WebSocket over HTTP/3
Ongoing	The browser API is the WHATWG "WebSockets Standard"

The protocol has two parts: the **opening handshake** and the **data transfer** (framing).

URI schemes (RFC 6455 §3):

```
ws://host[:port]/path[?query]     default port 80
wss://host[:port]/path[?query]    default port 443 (TLS)
```

Fragment identifiers (`#...`) are **not allowed** in WebSocket URIs.

2.3 The opening handshake

2.3.1 Client request

```HTTP
GET /ws HTTP/1.1
Host: chat.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
Origin: https://app.example.com
Sec-WebSocket-Protocol: v12.stomp, v11.stomp, v10.stomp
Sec-WebSocket-Extensions: permessage-deflate; client_max_window_bits
Cookie: SESSION=abc123
```

Rules the RFC requires of the client request:

Header	Required?	Meaning
Method `GET`, HTTP/1.1 or higher	Yes	Must be a GET
`Host`	Yes	Normal HTTP host
`Upgrade: websocket`	Yes	Case-insensitive value "websocket"
`Connection: Upgrade`	Yes	Must include the token "Upgrade" (could be `keep-alive, Upgrade`)
`Sec-WebSocket-Key`	Yes	Base64 of **16 random bytes**, new for every connection
`Sec-WebSocket-Version: 13`	Yes	Only `13` is valid for RFC 6455
`Origin`	Sent by browsers	The page's origin; **servers must check it** (CSWSH, Chapter 8)
`Sec-WebSocket-Protocol`	Optional	Sub-protocols the client can speak, in preference order
`Sec-WebSocket-Extensions`	Optional	Extensions the client wants (e.g., compression)
Cookies / Authorization	Optional	Normal HTTP headers; **browsers send cookies but you cannot add custom headers from JS**

2.3.2 Server response

```HTTP
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
Sec-WebSocket-Protocol: v12.stomp
Sec-WebSocket-Extensions: permessage-deflate
```

- Status **must be** `101`. Any other status (200, 302, 401, 403, 404…) means **the handshake failed.** Browsers do *not* follow redirects for WebSocket.

- `Sec-WebSocket-Protocol` in the response must be **one** of the values the client offered (or absent). If the server picks one the client didn't offer, the client must fail the connection.

2.3.3 How `Sec-WebSocket-Accept` is computed

```
accept = base64( SHA-1( Sec-WebSocket-Key + "258EAFA5-E914-47DA-95CA-C5AB0DC85B11" ) )
```

The magic string `258EAFA5-E914-47DA-95CA-C5AB0DC85B11` is a fixed GUID from the RFC.

Worked example (from RFC 6455, verified):

```
key      = dGhlIHNhbXBsZSBub25jZQ==            (base64 of "the sample nonce")
concat   = dGhlIHNhbXBsZSBub25jZQ==258EAFA5-E914-47DA-95CA-C5AB0DC85B11
SHA-1    = b3 7a 4f 2c c0 62 4f 16 90 f6 46 06 cf 38 59 45 b2 be c4 ea
accept   = s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

You can check it yourself:

BASH

```bash
printf '%s' 'dGhlIHNhbXBsZSBub25jZQ==258EAFA5-E914-47DA-95CA-C5AB0DC85B11' \
  | openssl sha1 -binary | base64
# s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

JAVA

```java
String accept = Base64.getEncoder().encodeToString(
    MessageDigest.getInstance("SHA-1").digest(
        (key + "258EAFA5-E914-47DA-95CA-C5AB0DC85B11").getBytes(StandardCharsets.US_ASCII)));
```

Interview trap: "Is `Sec-WebSocket-Key` a security feature?" No. It provides no authentication or encryption. Its purpose is to prove that the server actually understands the WebSocket protocol (so a non-WebSocket server, or a cache, cannot accidentally accept the request) and to stop caching proxies from replaying a previous handshake response. Real security = TLS (`wss://`) + Origin checking + authentication + authorisation.

Why the `Sec-` prefix? Browsers forbid JavaScript from setting headers starting with `Sec-`, so a script using `fetch` / `XMLHttpRequest` cannot forge a WebSocket handshake.

2.3.4 After the 101

The same TCP connection is now a WebSocket connection. No more HTTP on it. If the server ever wants to "redirect" or reject, it must do so **before** the 101 (with a normal HTTP status code).

2.3.5 Handshake failure scenarios

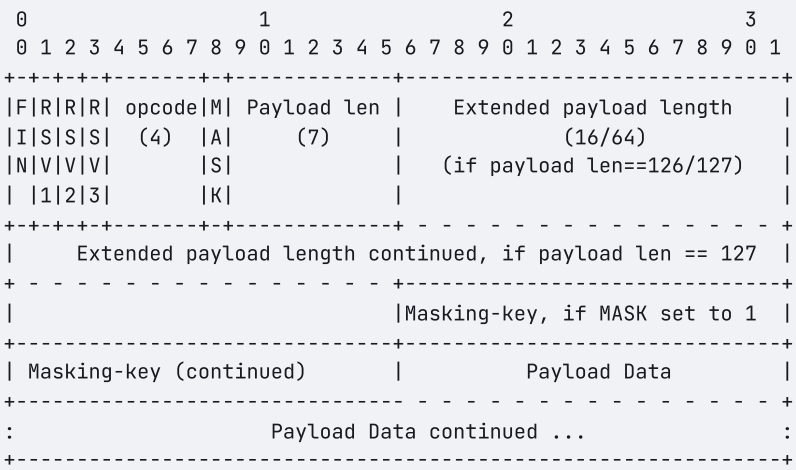
What you see	Likely cause
`Error during WebSocket handshake: Unexpected response code: 200`	You hit a normal HTTP endpoint (SPA fallback serving `index.html`, wrong path, proxy not forwarding `Upgrade`)
`... code: 400`	Missing/invalid `Upgrade` / `Connection` headers (proxy stripped them), or Spring rejected (e.g., unsupported origin can also be 403)

What you see	Likely cause
`... code: 403`	Origin not allowed (`setAllowedOrigins`) or security filter rejected
`... code: 404`	Wrong path (`/ws` vs `/ws/websocket` for SockJS) or context path missing
`... code: 401`	Auth filter on the handshake path
`... code: 502/504`	Reverse proxy/gateway can't reach the backend or timed out
Browser `close` event with code `1006` and no handshake response	TCP/TLS failure, certificate error, CORS-like origin rejection at proxy, server down

---

2.4 Base framing protocol (the heart of WebSocket)

After the handshake, data travels in **frames**:



2.4.1 Field by field

Field	Bits	Meaning
FIN	1	1 = this is the **final** fragment of a message
RSV1, RSV2, RSV3	1 each	Must be 0 unless an extension defines them. `permessage-deflate` uses **RSV1** to mark a compressed message. Unknown non-zero RSV → fail connection.
Opcode	4	Frame type (table below)
MASK	1	1 = payload is masked. **Client→server frames MUST be masked; server→client frames MUST NOT be masked.**
Payload length	7, 7+16, or 7+64	0–125: that's the length. 126: next 2 bytes (unsigned 16-bit) are the length. 127: next 8 bytes (unsigned 64-bit, most significant bit must be 0) are the length. Must use the **minimal** encoding.
Masking key	32	Present only if MASK=1. Random 4 bytes, new per frame.
Payload	rest	Extension data + application data

2.4.2 Opcodes

Opcode	Name	Type
`0x0`	Continuation	Data (fragment of a message)
`0x1`	**Text**	Data, payload must be valid **UTF-8**
`0x2`	**Binary**	Data, arbitrary bytes
`0x3` – `0x7`	Reserved	Future data frames
`0x8`	**Close**	Control
`0x9`	**Ping**	Control
`0xA`	**Pong**	Control
`0xB` – `0xF`	Reserved	Future control frames

Control frames (close/ping/pong):

- Payload ≤ **125 bytes.**
- **Must not be fragmented** (FIN=1 always).
- **May be injected in the middle of a fragmented message** (so a ping isn't stuck behind a huge message).

2.4.3 Header size

Payload length	Server→client header	Client→server header (+4 mask)
0–125 bytes	**2 bytes**	6 bytes
126–65,535 bytes	4 bytes	8 bytes
≥ 65,536 bytes	10 bytes	**14 bytes**

Hence "2 to 14 bytes of overhead per frame". Compare with hundreds of bytes for an HTTP request.

2.4.4 Decoding real frames

Unmasked text frame "Hello" (server → client):

```
81 05 48 65 6c 6c 6f
|  |  └──────────── "Hello" in ASCII
|  └─ 0x05: MASK=0, length=5
└─ 0x81: FIN=1, RSV=000, opcode=0x1 (text)
```

Masked text frame "Hello" (client → server), mask key 37 fa 21 3d:

```
81 85 37 fa 21 3d 7f 9f 4d 51 58
   |  └─ mask key  └─ masked payload
   └─ 0x85: MASK=1, length=5
```

Masking algorithm: `masked[i] = original[i] XOR maskKey[i % 4]`. Unmasking is the same operation.

```
'H'=0x48 ^ 0x37 = 0x7f
'e'=0x65 ^ 0xfa = 0x9f
'l'=0x6c ^ 0x21 = 0x4d
'l'=0x6c ^ 0x3d = 0x51
'o'=0x6f ^ 0x37 = 0x58
```

Ping with no payload: `89 00`. **Pong:** `8a 00`. **256-byte binary message (server):** `82 7e 01 00 <256 bytes>` (`7e` = 126 → 16-bit length `0x0100`). **64 KiB binary message (server):** `82 7f 00 00 00 00 00 01 00 00 <65536 bytes>`.

2.4.5 Why masking exists (classic interview question)

> **ELI5:** Masking scrambles what the browser sends so that a confused "middle-man" (an old proxy or cache) can't be tricked into thinking it's seeing a normal web request.

Researchers (Huang, Chen, Barth, Rescorla, Jackson, *"Talking to Yourself for Fun and Profit"*, 2011) showed that, with the pre-standard WebSocket drafts, an attacker's page could send bytes that **looked like an HTTP request** to transparent proxies that didn't understand WebSocket, leading to **cache poisoning** (the proxy caches the attacker's fake response for a real URL like a popular JavaScript file, affecting other users).

Because the browser picks a **random, unpredictable mask** per frame, the attacker's JavaScript cannot control the exact bytes on the wire. So it cannot craft a fake HTTP request.

- Masking is **not encryption** (the key is sent in the clear right before the data).
- Server→client frames aren't masked because the attacker controls the client side (script in a browser), not the server.
- A server **must close the connection** if it receives an unmasked frame from a client (close code 1002). A client must close if it receives a masked frame from a server.

2.4.6 Fragmentation

A **message** can be split into multiple **frames**:

```
Frame 1: FIN=0, opcode=0x1 (text)   "Hel"
Frame 2: FIN=0, opcode=0x0 (cont)   "lo, Wo"
Frame 3: FIN=1, opcode=0x0 (cont)   "rld"
=> one message: "Hello, World"
```

Rules:

- The first frame carries the real opcode; the rest use `0x0`.
- Fragments of **different** data messages must not interleave (no multiplexing in base protocol).
- Control frames **can** be interleaved between fragments.
- Intermediaries may re-fragment messages (unless extensions forbid it).
- Why fragment? To send a message whose size isn't known up front (streaming), or to keep control frames (ping/close) responsive during huge transfers.

Pitfall: "Message" vs "frame". APIs like the browser's `onmessage` give you **whole messages**. Low-level servers (e.g., Spring's `WebSocketHandler` with `supportsPartialMessages()`) can give you **partial** messages. Also, **STOMP frames are not the same as WebSocket frames**. One STOMP frame is usually one WebSocket message, but a STOMP frame could be split across WebSocket messages, or (less commonly) several STOMP frames could arrive in one WebSocket message. Spring's `BufferingStompDecoder` and stompjs handle this.

2.4.7 Text vs binary messages

- **Text (0x1):** payload must be valid UTF-8 across the whole message. Invalid UTF-8 → close with **1007**.
- **Binary (0x2):** any bytes (images, protobuf, MessagePack, compressed blobs).
- STOMP over WebSocket can use either. Text is easier to debug, binary is required if the body can contain NULL bytes or non-UTF-8 content, and it is also needed for the React Native workaround in Chapter 6.

2.5 Control frames in detail

2.5.1 Ping and Pong

- Either endpoint may send a **Ping** (`0x9`) at any time after the handshake.
- The receiver **must** respond with a **Pong** (`0xA`) "as soon as practical", with the **same application data** as the ping.
- A **Pong may be sent unsolicited** as a unidirectional heart-beat; no response is expected.
- If several pings arrive before a pong can be sent, it's allowed to respond only to the most recent.

Important: the browser JavaScript API cannot send pings or see pongs. Browsers automatically answer server pings with pongs, but your JS code has no `ping()` method. That's why **application-level heart-beats** exist (STOMP heart-beats, or `{"type":"ping"}` messages). Servers (Java, Node `ws`, Go) *can* send protocol pings to detect dead clients.

2.5.2 Close frame and the closing handshake

Payload of a close frame (optional):

```
+----------------------+-----------------------------------+
| 2-byte status code   | UTF-8 reason (≤ 123 bytes)        |
| (network byte order) | (125 - 2 = 123 bytes max)         |
+----------------------+-----------------------------------+
```

Closing handshake:

```
Client                              Server
  |---- Close(1000, "bye") -------->|
  |<--- Close(1000, "bye") ---------|   server echoes a close frame
```

```
  |                                |   server closes TCP first (recommended)
  X  TCP FIN/ACK                   X
```

- After sending a close frame, an endpoint **must not send more data frames**.
- After receiving a close frame, the endpoint must send a close frame back (if it hasn't already).
- The **server should close the underlying TCP connection first** (RFC 6455 §7.1.1), so that the server, not the client, holds the TCP `TIME_WAIT` state and the client can immediately re-open a connection. Application code rarely needs to care; the library does it.
- If the TCP connection dies without a close frame, the API reports **1006 (abnormal closure)**.

2.5.3 Close status codes (memorise the common ones)

Code	Name	When
1000	Normal Closure	Purpose fulfilled, normal logout
1001	Going Away	Server shutting down, or browser tab navigating away
1002	Protocol Error	Malformed frames, unmasked client frame, etc.
1003	Unsupported Data	E.g., binary received by a text-only endpoint
1004	Reserved	Not to be used
1005	No Status Received	**Reserved; never sent on the wire.** API reports it when a close frame had no code.
1006	Abnormal Closure	**Reserved; never sent on the wire.** Connection dropped without a close frame (network loss, crash, proxy timeout, TLS failure). **The most common code you'll debug.**
1007	Invalid Frame Payload Data	E.g., non-UTF-8 in a text message
1008	Policy Violation	Generic "you broke a rule" (e.g., auth failure after connect)
1009	Message Too Big	Message exceeds the receiver's limit (e.g., Spring/Tomcat buffer size)
1010	Mandatory Extension	Client expected the server to negotiate an extension it didn't
1011	Internal Error	Server hit an unexpected condition
1012	Service Restart	(IANA registered) server restarting; client should reconnect
1013	Try Again Later	(IANA registered) temporary overload
1014	Bad Gateway	(IANA registered) gateway received an invalid response
1015	TLS Handshake	**Reserved; never sent.** TLS handshake failure
0–999	Not used	
1000–2999	Protocol	Defined by RFC/extensions
3000–3999	Libraries/frameworks/apps	Registered with IANA (first come first served)
4000–4999	Private use	Your application's own codes (e.g., 4001 "token expired")

From browser JavaScript, `ws.close(code, reason)` accepts only **1000** or **3000–4999**, otherwise it throws `InvalidAccessError`. The reason must be ≤ **123 bytes** of UTF-8, otherwise it throws `SyntaxError`.

> **Pitfall:** Spring defines a few of its own `CloseStatus` constants, e.g. `CloseStatus.SESSION_NOT_RELIABLE` (**4500**), used when a session's send buffer/time limits are exceeded (Chapter 9). If you see 4500 in logs, it's Spring, not the client.

2.6 Extensions and sub-protocols

2.6.1 Extensions (`Sec-WebSocket-Extensions`)

Extensions change the framing itself. The famous one is `permessage-deflate` (RFC 7692):

- Compresses each message with DEFLATE; sets **RSV1** on the first frame of a compressed message.
- Parameters: `server_no_context_takeover`, `client_no_context_takeover`, `server_max_window_bits`, `client_max_window_bits`.
- "Context takeover" = keep the compression dictionary between messages (better ratio, more memory per connection, often ~tens to hundreds of KB per connection depending on window/memory settings).
- Trade-off: **CPU and memory vs bandwidth**. For 100k connections, per-connection compression memory can dominate. Many teams disable context takeover or compression entirely for small JSON messages.
- **Security:** compression of secrets mixed with attacker-controlled data can leak information (CRIME/BREACH-style attacks). Don't compress messages mixing secrets and user-controlled input in the same context.

2.6.2 Sub-protocols (`Sec-WebSocket-Protocol`)

Sub-protocols define the **application-level language** spoken over the socket. For STOMP the registered names are:

```
v10.stomp   STOMP 1.0
v11.stomp   STOMP 1.1
v12.stomp   STOMP 1.2
```

Others you might meet: `mqtt`, `wamp`, `graphql-transport-ws` (GraphQL subscriptions), `graphql-ws` (legacy).

- The client offers a list, and the server chooses **at most one**.
- Spring's `SubProtocolWebSocketHandler` uses this to pick the STOMP handler. If the client sends no sub-protocol header, Spring falls back to its default protocol handler (STOMP, when it's the only one configured).

Hack you'll see in the wild: Because browsers can't set an `Authorization` header, some apps put a token in `Sec-WebSocket-Protocol` (e.g., `new WebSocket(url, ['v12.stomp', token])`). It works but is a hack: the server must echo back exactly **one** offered value, the token ends up in logs, and it abuses a header meant for protocol names. Prefer the STOMP `CONNECT` header approach (Chapter 8).

2.7 WebSocket over HTTP/2 and HTTP/3

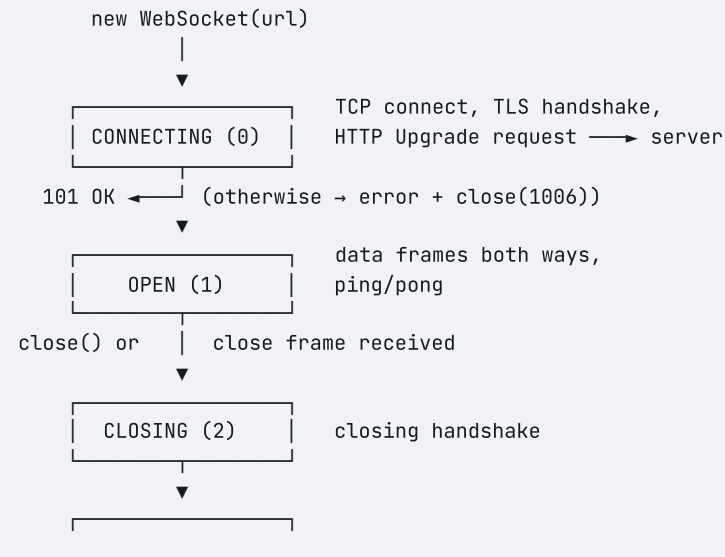
Classic WebSocket requires HTTP/1.1 `Upgrade`, which **does not exist in HTTP/2.** Solutions:

- **RFC 8441 (HTTP/2):** server advertises `SETTINGS_ENABLE_CONNECT_PROTOCOL = 1`. Client sends an **extended CONNECT** request on a new stream:

  ```
  :method = CONNECT
  :protocol = websocket
  :scheme = https
  :path = /ws
  :authority = chat.example.com
  sec-websocket-version = 13
  sec-websocket-protocol = v12.stomp
  origin = https://app.example.com
  ```

  Response `:status = 200` (not 101). There is **no** `Sec-WebSocket-Key/Accept` exchange. WebSocket frames then flow as the stream's DATA. Many WebSockets can share one TCP connection.
- **RFC 9220 (HTTP/3):** same idea over QUIC.
- In practice most production stacks still use **HTTP/1.1 upgrade** from the edge proxy to the app, even if browser ↔ CDN uses HTTP/2. Support depends on each hop (browser, CDN, load balancer, server). Don't assume it.

2.8 Connection lifecycle: the full picture



```
          |   CLOSED (3)    |   onclose(code, reason, wasClean)
          └─────────────────┘
```

2.9 Limits, intermediaries and the real world

2.9.1 Message size limits

The protocol allows messages up to 2^63 bytes. Real servers set limits:

Component	Typical default (verify for your version)
Tomcat (JSR-356) text/binary buffer	8 KB (`maxTextMessageBufferSize` , `maxBinaryMessageBufferSize`)
Spring STOMP inbound message size limit	64 KB (`WebSocketTransportRegistration.setMessageSizeLimit`)
Node `ws` `maxPayload`	100 MiB (default in recent versions)
Nginx	No WS message limit (just proxies bytes)
AWS API Gateway WebSocket	128 KB message, 32 KB frame

Exceeding usually → **1009 Message Too Big** close. Design rule: **don't send huge payloads over WebSocket.** Send a notification with an ID or URL, then fetch the big data over HTTP.

2.9.2 Idle timeouts at intermediaries

Every hop may kill connections that look idle:

Hop	Typical default idle timeout
Nginx `proxy_read_timeout` / `proxy_send_timeout`	60 s
AWS Application Load Balancer	60 s (configurable up to 4000 s)
Azure Application Gateway / Front Door / GCP load balancer	Varies; GCP backend service timeout often applies to WS lifetime (default 30 s, must be raised)
Cloudflare	~100 s idle without traffic
Corporate proxies, NAT routers, mobile carriers	Anything from 30 s to minutes

Solution: heart-beats/pings at an interval **shorter** than the smallest idle timeout in the path (e.g., 10–25 s), plus increased proxy timeouts.

2.9.3 Proxies

- **Forward proxies** (corporate): the browser uses HTTP `CONNECT` tunnelling for `wss://` , which usually works. Plain `ws://` through a proxy that doesn't understand `Upgrade` often fails → another reason for `wss://` .
- **Reverse proxies** (Nginx, HAProxy, Envoy, cloud LBs): must explicitly forward `Upgrade` and `Connection` headers (Chapter 7 has configs).

- **Antivirus / "TLS inspection" products** sometimes buffer or break WebSocket. SockJS fallbacks were invented largely for such environments.

2.10 Writing a WebSocket by hand (to prove you understand it)

A minimal Node.js server **without** libraries (educational only, not production: no fragmentation, no large frames, no close handshake):

JS

```
const http = require('http');
const crypto = require('crypto');
const GUID = '258EAFA5-E914-47DA-95CA-C5AB0DC85B11';

const server = http.createServer((req, res) => res.end('plain http'));

server.on('upgrade', (req, socket) => {
  if (req.headers['upgrade']?.toLowerCase() !== 'websocket') return socket.destroy();
  const accept = crypto.createHash('sha1')
    .update(req.headers['sec-websocket-key'] + GUID).digest('base64');
  socket.write(
    'HTTP/1.1 101 Switching Protocols\r\n' +
    'Upgrade: websocket\r\n' +
    'Connection: Upgrade\r\n' +
    `Sec-WebSocket-Accept: ${accept}\r\n\r\n`);

  socket.on('data', (buf) => {
    const opcode = buf[0] & 0x0f;
    const masked = (buf[1] & 0x80) !== 0;
    let len = buf[1] & 0x7f, offset = 2;
    if (len === 126) { len = buf.readUInt16BE(2); offset = 4; }
    else if (len === 127) { len = Number(buf.readBigUInt64BE(2)); offset = 10; }
    if (!masked) return socket.destroy();          // clients MUST mask
    const mask = buf.subarray(offset, offset + 4); offset += 4;
    const payload = Buffer.alloc(len);
    for (let i = 0; i < len; i++) payload[i] = buf[offset + i] ^ mask[i % 4];

    if (opcode === 0x8) return socket.end(Buffer.from([0x88, 0x00]));  // close
    if (opcode === 0x9) return socket.write(Buffer.concat([Buffer.from([0x8a, len]), payload])); // pong
    if (opcode === 0x1) {
      const reply = Buffer.from('echo: ' + payload.toString('utf8'));
      socket.write(Buffer.concat([Buffer.from([0x81, reply.length]), reply])); // assumes < 126
    }
  });
});
server.listen(8080);
```

In real projects **always use a mature library** (Spring/Tomcat/Jetty, Node `ws`, Go `gorilla/websocket` or `coder/websocket`, Python `websockets`). Hand-written parsers routinely get fragmentation, UTF-8 validation, partial TCP reads (`data` events don't align with frames!), and close handling wrong.

2.11 Tools to inspect WebSocket traffic

- **Chrome/Edge DevTools** → **Network** → filter **"WS"** → click the connection → **Messages** tab (shows each message; STOMP frames are readable text). Firefox has the same.
- **Wireshark**: filter `websocket`; for `wss://` you need TLS keys (`SSLKEYLOGFILE` env var for browsers).
- **`websocat`**: `websocat wss://echo.websocket.org` (a netcat for WebSockets). `wscat` (`npx wscat -c ws://localhost:8080/ws -s v12.stomp`).
- **`curl`** can test the handshake (`curl -i -N -H "Connection: Upgrade" -H "Upgrade: websocket" -H "Sec-WebSocket-Version: 13" -H "Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==" http://localhost:8080/ws`), and recent versions (7.86+) have experimental native `ws://` support.
- **Postman** and **Insomnia** support WebSocket requests (you can type raw STOMP frames, remembering the NULL terminator: Postman lets you send `\u0000`).
- **Burp Suite / OWASP ZAP**: security testing of WebSocket messages.

2.12 Chapter summary

- Handshake: HTTP/1.1 GET + `Upgrade: websocket` + `Connection: Upgrade` + `Sec-WebSocket-Key` + version 13 → **101** + `Sec-WebSocket-Accept = base64(SHA1(key + GUID))`.
- Frames: FIN, RSV1-3, opcode, MASK, length (7/16/64-bit), mask key, payload. 2–14 bytes overhead.
- Client→server frames must be masked (anti-cache-poisoning), server→client must not be.
- Messages can be fragmented. Control frames ≤125 bytes, never fragmented, may interleave.
- Ping/pong for liveness; the browser API can't send pings → use app-level heart-beats.
- Close codes: 1000 normal, 1001 going away, 1006 abnormal (never sent), 1008 policy, 1009 too big, 1011 internal, 4000–4999 yours.
- Sub-protocol for STOMP: `v12.stomp`. Extension: `permessage-deflate`.
- Real world: proxies, idle timeouts, message size limits, always `wss://`.

Next: Chapter 3: The Browser & JavaScript WebSocket API

CHAPTER 3

Chapter 3: The Browser & JavaScript WebSocket API

Goal: master the raw `WebSocket` object. Even if you always use STOMP libraries, they sit on top of this API, and most "mysterious" bugs come from its rules.

3.1 ELI5

The browser gives you a toy telephone called `WebSocket`. You:

1. **Dial** (`new WebSocket(url)`).
2. Wait until it says "**connected**" (`onopen`).
3. **Talk** (`send(...)`) and **listen** (`onmessage`).
4. Notice if the line **breaks** (`onerror`, `onclose`).
5. **Hang up** (`close()`).

The toy telephone can't redial by itself, can't add stickers (custom headers) on the envelope, and can't press the "are you there?" button (ping). You must build those yourself, or use a library.

3.2 The API surface (complete)

```ts
const ws = new WebSocket(url: string | URL, protocols?: string | string[]);
```

Member	Type	Description
`url`	string	Resolved absolute URL
`protocol`	string	Sub-protocol the server selected (`""` if none)
`extensions`	string	Extensions the server selected (e.g., `"permessage-deflate"`)
`readyState`	number	`0` CONNECTING, `1` OPEN, `2` CLOSING, `3` CLOSED (also as constants `WebSocket.OPEN`, etc.)
`bufferedAmount`	number	Bytes queued by `send()` but **not yet transmitted** to the network
`binaryType`	`"blob"` | `"arraybuffer"`	How binary messages are delivered to `onmessage`. Default `"blob"`
`send(data)`	method	`string`, `ArrayBuffer`, `ArrayBufferView` (typed array/DataView), or `Blob`
`close(code?, reason?)`	method	Starts the closing handshake. `code` = 1000 or 3000–4999; `reason` ≤ 123 UTF-8 bytes

Member	Type	Description
`onopen` / `addEventListener('open')`	event	Handshake succeeded
`onmessage`	event (`MessageEvent`)	`event.data` is a `string` (text) or `Blob` / `ArrayBuffer` (binary)
`onerror`	event	Something failed. **Contains no useful details by design** (security)
`onclose`	event (`CloseEvent`)	`code`, `reason`, `wasClean`

Behaviour rules that matter:

- Calling `send()` while **CONNECTING** throws `InvalidStateError`. Wait for `open`.
- Calling `send()` after the socket is **CLOSING/CLOSED** does **not** throw. The data is silently discarded (and `bufferedAmount` increases). Your app must check `readyState`.
- `onerror` is always followed by `onclose`. Put reconnect logic in `onclose`, not `onerror`.
- Invalid URL schemes (anything other than `ws`, `wss`, and in modern browsers `http` / `https`, which are converted) throw `SyntaxError` at construction.
- Duplicated or invalid protocol names → `SyntaxError`.
- `onerror` intentionally hides the reason (e.g., you can't tell "connection refused" from "403") to prevent network scanning from web pages. Debug with DevTools.

3.3 A correct minimal client

JS

```
const ws = new WebSocket('wss://api.example.com/ws', ['v12.stomp']);
ws.binaryType = 'arraybuffer';

ws.addEventListener('open', () => {
  console.log('open, protocol =', ws.protocol);
  ws.send('hello');
});

ws.addEventListener('message', (e) => {
  if (typeof e.data === 'string') console.log('text', e.data);
  else console.log('binary bytes', new Uint8Array(e.data).length);
});

ws.addEventListener('error', () => console.warn('ws error (details in DevTools)'));

ws.addEventListener('close', (e) => {
  console.log(`closed code=${e.code} reason="${e.reason}" clean=${e.wasClean}`);
});
```

3.4 The five big limitations (and their work-arounds)

3.4.1 No custom headers

`new WebSocket()` can't set `Authorization` or any custom header. Options:

Approach	How	Pros	Cons
Cookies	Same-site session/JWT cookie sent automatically on handshake	Transparent, HttpOnly protects from XSS theft	Needs **Origin check** + CSRF thinking (CSWSH); cross-domain cookie rules (`SameSite`)
Query parameter	`wss://host/ws?access_token=...`	Simple, works everywhere	Token appears in **server/proxy access logs**, browser history; use short-lived, single-use "ticket" tokens
Ticket pattern	Get a short-lived one-time ticket via authenticated REST, then put the ticket in the URL	Safer variant of query param	Extra round trip, ticket store
First message auth	Connect, then send credentials as the first message (e.g., **STOMP `CONNECT` headers**)	Clean; the token isn't in URL logs	Handshake itself is unauthenticated; server must enforce timeouts for unauthenticated sockets
`Sec-WebSocket-Protocol` hack	Put token as a "protocol"	Works in browsers	Abuse of the header; logged; must echo

STOMP users nearly always use **STOMP CONNECT headers** (Chapter 8).

3.4.2 No ping API

Browsers respond to server pings automatically, but JS can't send pings or observe pongs. So:

- The **server** should send pings or app-level heart-beats to detect dead clients.
- The **client** needs app-level heart-beats (STOMP heart-beats do exactly this) to detect a dead server/network. Otherwise a half-open connection may look "OPEN" for minutes.

3.4.3 No automatic reconnect

Unlike `EventSource`, `WebSocket` never reconnects. You implement it (or the library does, as `@stomp/stompjs` does via `reconnectDelay`).

3.4.4 No backpressure (in the classic API)

`send()` never blocks. If you send faster than the network can take, data piles up in browser memory. Check `bufferedAmount`:

JS

```
function safeSend(ws, data, maxBuffered = 1_000_000) {
  if (ws.readyState !== WebSocket.OPEN) return false;
  if (ws.bufferedAmount > maxBuffered) return false; // caller should retry later / drop / coalesce
  ws.send(data);
  return true;
}
```

On the **receive** side there is no way to say "slow down" either. Messages are delivered as fast as they arrive. The experimental `WebSocketStream` API (Chromium-based browsers) adds promise/stream-based backpressure. Don't rely on it for cross-browser production code yet.

3.4.5 Same-Origin Policy does NOT protect you

CORS does not apply to WebSocket. Any web page on any origin can open a WebSocket to your server, and the browser will attach your cookies (subject to `SameSite` rules). The only protection is the server **checking the `Origin` header** (and not relying on cookies alone). This is **Cross-Site WebSocket Hijacking (CSWSH)** (Chapter 8).

3.5 Reconnect done right

Bad reconnect logic can **take down your own servers** (all clients reconnecting at the same instant after a deploy, the "thundering herd"). Use **exponential backoff with jitter** and a cap.

TS

```ts
class ReconnectingSocket {
  private ws?: WebSocket;
  private attempt = 0;
  private closedByUser = false;

  constructor(private url: string, private onMessage: (d: any) => void) {}

  connect() {
    this.closedByUser = false;
    this.ws = new WebSocket(this.url);
    this.ws.onopen = () => { this.attempt = 0; };
    this.ws.onmessage = (e) => this.onMessage(e.data);
    this.ws.onclose = (e) => {
      if (this.closedByUser) return;
      if (e.code === 4001 /* our "auth failed" code */) return this.refreshTokenThenConnect();
      const base = Math.min(30_000, 500 * 2 ** this.attempt++);   // 0.5s,1s,2s,4s... max 30s
      const delay = Math.random() * base;                          // "full jitter"
      setTimeout(() => this.connect(), delay);
    };
  }

  close() { this.closedByUser = true; this.ws?.close(1000, 'bye'); }
  private async refreshTokenThenConnect() { /* get new token, then connect() */ }
}
```

Checklist for reconnect:

1. **Backoff + jitter + max delay.**
2. **Don't reconnect** on intentional close or permanent errors (bad credentials).
3. **Re-subscribe** to everything after reconnect (subscriptions die with the connection; STOMP libraries re-run your `onConnect`).
4. **Resync state**: messages sent while you were disconnected are **lost** unless your design stores them (fetch missed data via REST using "last seen id/timestamp").
5. **Online/offline events**: listen to `window.addEventListener('online', ...)` to reconnect immediately when the network returns.
6. **Page visibility**: optionally disconnect hidden tabs after some time and reconnect on `visibilitychange` to save server resources.
7. **Token refresh**: if the access token expired, refresh before reconnecting.

3.6 Many tabs, one connection

Each browser tab opens its own WebSocket by default. 10 tabs = 10 connections per user. Options:

- Accept it (simplest, usually fine).
- **SharedWorker** holds one WebSocket, and tabs talk to it via `MessagePort`. (SharedWorker is well supported on desktop browsers, and mobile support has historically lagged, so check your target browsers before relying on it.)
- **Leader election** with `BroadcastChannel` / Web Locks API: one tab owns the socket and rebroadcasts to others.

3.7 Back/forward cache, sleep and mobile

- Pages with an open WebSocket historically could not enter the **bfcache**. Modern browsers may evict or close the connection when the page goes into the cache. Handle `pagehide` / `pageshow` and reconnect on `pageshow` with `event.persisted === true`.
- Laptops sleeping, phones switching Wi-Fi ↔ 4G, and mobile OSs suspending background apps all **silently kill** TCP connections. Heart-beats + reconnect are not optional.
- On iOS/Android **native apps**, when the app goes to background the OS will soon suspend it. Use **push notifications** for background delivery and reconnect WebSocket on foreground.

3.8 Mixed content and secure contexts

- A page served over `https://` **cannot** open `ws://` (mixed content is blocked). Use `wss://`.
- Relative URLs: modern browsers accept `new WebSocket('/ws')` style relative URLs resolved against the page (and `http(s)` → `ws(s)` conversion). Older browsers required absolute `ws(s)://` URLs. Build the URL explicitly for compatibility:

```JS
const proto = location.protocol === 'https:' ? 'wss:' : 'ws:';
const url = `${proto}//${location.host}/ws`;
```

3.9 Server-side JavaScript (Node.js) quick reference

The most used Node library is `ws`:

```JS
import { WebSocketServer } from 'ws';
const wss = new WebSocketServer({ port: 8080, maxPayload: 64 * 1024 });

wss.on('connection', (ws, req) => {
  // Check origin & auth here (req.headers.origin, req.headers.cookie, req.url)
  ws.isAlive = true;
  ws.on('pong', () => (ws.isAlive = true));
  ws.on('message', (data, isBinary) => ws.send(data, { binary: isBinary }));
});

// Detect dead connections with protocol-level pings
```

```
setInterval(() => {
  for (const ws of wss.clients) {
    if (!ws.isAlive) { ws.terminate(); continue; }
    ws.isAlive = false;
    ws.ping();
  }
}, 30_000);
```

Node 22+ also ships a **built-in `WebSocket` client** (global, browser-compatible API).

3.10 Chapter summary

- API: `new WebSocket(url, protocols)`, `readyState`, `send`, `close`, `bufferedAmount`, `binaryType`, events `open` / `message` / `error` / `close`.
- Can't set headers, can't ping, doesn't reconnect, no backpressure, not protected by CORS.
- Reconnect with exponential backoff + jitter; re-subscribe and resync after reconnect.
- Always `wss://` from HTTPS pages; handle sleep/background/network change.

Next: Chapter 4: The STOMP Protocol

CHAPTER 4

Chapter 4: The STOMP Protocol (1.0 / 1.1 / 1.2), Deep Dive

Goal: know STOMP so well you could write frames by hand, implement a tiny client, and answer any "what does header X do?" question. Everything here follows the official **STOMP 1.2 specification** (stomp.github.io), with notes on 1.0/1.1 differences.

4.1 ELI5

STOMP is a **very small language** for talking to a post office (broker):

- **"Hello, I'm Anna, here's my password."** → `CONNECT`
- **"Hi Anna, you're in."** → `CONNECTED`
- **"Please tell me about everything that goes to the 'kitchen' mailbox."** → `SUBSCRIBE`
- **"Put this letter in the 'kitchen' mailbox."** → `SEND`
- **"Here's a letter from the 'kitchen' mailbox."** → `MESSAGE`
- **"Got it, thanks!"** → `ACK` (or "I can't handle this one" → `NACK`)
- **"Tell me when you've got my letter."** → the `receipt` header → `RECEIPT`
- **"Something went wrong!"** → `ERROR`
- **"Bye!"** → `DISCONNECT`

Every sentence (frame) looks the same: **a command word, some "name:value" stickers (headers), an empty line, the letter contents (body), and a special invisible full stop (the NULL byte).**

4.2 What STOMP is (and isn't)

- **STOMP** = **Simple** (or **Streaming**) **Text Oriented Messaging Protocol**. Earlier it was called TTMP.
- A frame-based, text-based, **interoperable** messaging protocol modelled on HTTP's look.
- Designed to be so simple that you can write a client in an afternoon, in any language, even with `telnet`.
- Transport-agnostic: runs over **raw TCP** (brokers typically listen on port **61613**) or over **WebSocket** (sub-protocol `v12.stomp`), or anything else that is a reliable, ordered, bi-directional stream.
- It does **not** define destination semantics (queue vs topic). Those are broker-specific.
- It does **not** define message bodies (JSON/XML/binary are all fine; you declare `content-type`).

Version	Released	Big additions
1.0	2006-ish	Basic frames: CONNECT, SEND, SUBSCRIBE, UNSUBSCRIBE, ACK, BEGIN, COMMIT, ABORT, DISCONNECT, MESSAGE, RECEIPT, ERROR

Version	Released	Big additions
1.1	2011	**Heart-beating**, **NACK**, **version negotiation** (`accept-version`), **virtual hosting** (`host`), **header escaping**, UTF-8 headers, `id` **on SUBSCRIBE required**, `client-individual` ack mode, `STOMP` command as alias for CONNECT
1.2	Oct 2012	**CRLF line endings allowed**, **simplified ACK/NACK** (single `id` header matching MESSAGE's `ack` header), `\r` escape, clarified repeated headers (first wins), connection lingering, clarified scope/uniqueness of subscription & transaction ids

Spring, stompjs, RabbitMQ and ActiveMQ all support 1.2 (and negotiate down to 1.1/1.0 if needed).

4.3 Frame anatomy

```
COMMAND␊
header1:value1␊
header2:value2␊
␊
Body^@
```

- `␊` = end-of-line: **LF** (`\n` , 0x0A), optionally preceded by **CR** (`\r\n`) in STOMP 1.2.
- The **blank line** separates headers from body (even if there are no headers and no body).
- `^@` = the **NULL octet** (`\0` , 0x00) ends the frame.
- After the NULL, zero or more EOLs **may** follow (that's where heart-beats live).

4.3.1 The formal grammar (STOMP 1.2, augmented BNF)

```
NULL                = <US-ASCII null (octet 0)>
LF                  = <US-ASCII line feed (aka newline) (octet 10)>
CR                  = <US-ASCII carriage return (octet 13)>
EOL                 = [CR] LF
OCTET               = <any 8-bit sequence of data>

frame-stream        = 1*frame

frame               = command EOL
                      *( header EOL )
                      EOL
                      *OCTET
                      NULL
                      *( EOL )

command             = client-command | server-command

client-command      = "SEND"
                      | "SUBSCRIBE"
                      | "UNSUBSCRIBE"
                      | "BEGIN"
                      | "COMMIT"
                      | "ABORT"
                      | "ACK"
                      | "NACK"
                      | "DISCONNECT"
                      | "CONNECT"
                      | "STOMP"
```

```
server-command      = "CONNECTED"
                    | "MESSAGE"
                    | "RECEIPT"
                    | "ERROR"

header              = header-name ":" header-value
header-name         = 1*<any OCTET except CR or LF or ":">
header-value        = *<any OCTET except CR or LF or ":">
```

4.3.2 Encoding rules

- Commands and headers are **UTF-8**.
- Header names and values are **case-sensitive** and **not trimmed** (spaces are part of the value: `destination: /topic/a` has destination `" /topic/a"`, a classic bug).
- Only **SEND, MESSAGE and ERROR** frames may have a body. All other frames **must not** have a body.

4.3.3 Header value escaping (STOMP 1.1+)

Because `:` and newlines delimit headers, they must be escaped inside names/values:

Escape	Means
`\r`	carriage return (0x0D), new in 1.2
`\n`	line feed (0x0A)
`\c`	colon `:`
`\\`	backslash `\`

- Any other escape (e.g., `\t`) is a **fatal protocol error**.
- **Exception:** `CONNECT` and `CONNECTED` frames do **not** use escaping (backward compatibility with 1.0). So a password containing `:` in `passcode` is sent raw.
- Example: header `foo` with value `a:b` → `foo:a\cb`.

4.3.4 Repeated headers

If a header name appears more than once, **only the first value is used.** (Brokers may add history entries below it.)

4.3.5 Body length: `content-length`

- `content-length` = number of **octets** (bytes, not characters!) in the body.
- If present, the receiver reads exactly that many bytes, **even if they contain NULL bytes**, then expects the terminating NULL.
- If absent, the body ends at the first NULL byte. So **a body without `content-length` cannot contain NULL.**
- Frames SHOULD include `content-length` when they have a body.

Pitfall: characters vs bytes. `"héllo"` is 5 characters but **6 bytes** in UTF-8. Computing `content-length` with `string.length` in JS produces corrupted frames for non-ASCII text (emoji!). Libraries handle this via `TextEncoder`. That is one reason stompjs needs a `TextEncoder` polyfill on old React Native.

4.3.6 `content-type`

- MIME type of the body, e.g. `application/json` or `text/plain;charset=utf-8`.
- If absent, the receiver should treat the body as a **binary blob**.
- For `text/*` types with no charset, the default charset is **UTF-8**.
- Spring uses `content-type` to pick a `MessageConverter` (e.g., JSON → Jackson).

4.3.7 Size limits

Servers may limit: number of headers, header line length, body size. Exceeding → `ERROR` + connection close. (Spring: `setMessageSizeLimit`, default 64 KB.)

4.4 Connecting

4.4.1 CONNECT (or STOMP)

```
CONNECT
accept-version:1.2,1.1,1.0
host:chat.example.com
login:anna
passcode:secret
heart-beat:10000,10000

^@
```

Header	1.2 status	Meaning
`accept-version`	**REQUIRED**	Comma-separated versions the client supports. If missing, server assumes 1.0
`host`	**REQUIRED**	Virtual host name (like HTTP Host). RabbitMQ maps it to a vhost
`login`	optional	User identifier
`passcode`	optional	Password
`heart-beat`	optional	`cx,cy` (see §4.5). Default `0,0`

Clients may add **any custom headers** (e.g., `Authorization: Bearer ...`). This is how token auth over STOMP is usually done (Spring reads them as "native headers").

`STOMP` **command:** identical to `CONNECT`. Introduced so that servers can distinguish STOMP from HTTP traffic on the same port (an HTTP request never starts with `STOMP`). Clients using `STOMP` instead of `CONNECT` can only connect to 1.1+/1.2 servers, so most clients still send `CONNECT`.

4.4.2 CONNECTED

```
CONNECTED
version:1.2
heart-beat:10000,10000
session:sess-4f2a
server:RabbitMQ/3.13.7

^@
```

Header	Status	Meaning
`version`	**REQUIRED** (1.1+)	The negotiated version (highest common)
`heart-beat`	optional	Server's `sx,sy`
`session`	optional	Session identifier
`server`	optional	`name/version` of server

4.4.3 Version negotiation

- Server picks the **highest version** present in both `accept-version` and its own supported list.
- If there's no common version, server replies with `ERROR` containing a `version` header listing its supported versions, plus an explanation in the body, then **closes** the connection:

```
ERROR
version:1.2,2.1
content-type:text/plain

Supported protocol versions are 1.2 2.1^@
```

4.4.4 Over WebSocket

- The WebSocket sub-protocol header (`v12.stomp`) and STOMP's `accept-version` are **separate** negotiations. Libraries usually set both consistently.
- The STOMP `CONNECT` frame is the **first WebSocket message** after `open` .

4.5 Heart-beating (connection liveness)

ELI5: Two friends on a walkie-talkie agree: "If I don't hear anything from you for 10 seconds, I'll assume your battery died." So when there's nothing to say, each one just clicks the button ("beep") every few seconds.

4.5.1 Negotiation

- Client sends `heart-beat:cx,cy`
  - `cx` = the smallest interval (ms) **the client can guarantee between its outgoing heart-beats** (0 = "I can't send heart-beats").

- `cy` = the interval (ms) **the client would like to receive heart-beats at** (0 = "I don't want any").
- Server replies `heart-beat:sx,sy` with the same meaning from its side.

Resulting intervals:

```
client → server  :  if cx == 0 or sy == 0 → none,  else every max(cx, sy) ms
server → client  :  if sx == 0 or cy == 0 → none,  else every max(sx, cy) ms
```

Worked example:

```
Client: heart-beat:4000,20000     (can send every 4s, wants to receive every 20s)
Server: heart-beat:10000,10000    (can send every 10s, wants to receive every 10s)

client → server: max(4000, 10000)  = 10000 ms  (client sends at least every 10s)
server → client: max(10000, 20000) = 20000 ms  (server sends at least every 20s)
```

Another:

```
Client: heart-beat:0,10000   → client won't send heart-beats.
Server: heart-beat:10000,10000
client → server: cx == 0 → none
server → client: max(10000, 10000) = 10000 ms
```

4.5.2 What a heart-beat is

- A heart-beat is **a single EOL** (`\n` or `\r\n`) sent between frames.
- **Any data** (a real frame) also counts as proof of life. Heart-beats are only needed when idle.
- The receiver checks: "Have I received *anything* within the interval?" The spec lets implementations add a **tolerance** for network delays. For example, `@stomp/stompjs` declares the server dead after **2×** the negotiated incoming interval with no data (configurable since v7 via `heartbeatToleranceMultiplier`, default `2`), and Spring's simple broker and broker relay use a multiplier of **3×** on the read side (`HEARTBEAT_MULTIPLIER = 3` in their source code). The principle is "a few missed beats = dead". Spring's simple broker implements exactly the negotiation formula above: its read interval is `max(clientHeartbeat[0], serverHeartbeat[1]) × 3` and its write interval is `max(clientHeartbeat[1], serverHeartbeat[0])`.
- When the receiver decides the peer is dead, it **closes the connection** (and the client reconnects).

4.5.3 Why STOMP heart-beats matter so much over WebSocket

- Browsers can't send WebSocket pings (Chapter 3), so STOMP heart-beats are your **only** client-side liveness detection.
- They keep **proxies/load balancers** from closing "idle" connections (Chapter 2).
- They detect **half-open TCP** (phone lost signal).

Pitfall: Spring simple broker heart-beats are OFF until you give it a TaskScheduler. With the simple broker, the server answers CONNECT with `heart-beat:0,0` (no heart-beats) unless you configure `.setTaskScheduler(...)` (and optionally `.setHeartbeatValue(...)`). Chapter 5 shows how.

4.6 Sending: SEND

```
SEND
destination:/app/chat.send
content-type:application/json
content-length:28
receipt:send-42

{"room":"r1","text":"hello"}^@
```

Header	Status	Meaning
`destination`	**REQUIRED**	Where to send. Opaque string, broker-defined
`transaction`	optional	Make this send part of a transaction
`content-type`, `content-length`	recommended	Body description
`receipt`	optional	Ask for a RECEIPT
any other	optional	**User-defined headers are passed through** to subscribers' MESSAGE frames by most brokers

If the server can't process the SEND (bad destination, unauthorised), it sends `ERROR` and closes.

4.7 Subscribing: SUBSCRIBE / UNSUBSCRIBE

```
SUBSCRIBE
id:sub-0
destination:/topic/room.r1
ack:client-individual

^@
```

Header	Status	Meaning
`destination`	**REQUIRED**	What to listen to
`id`	**REQUIRED** (1.1+)	Client-chosen id, **unique within the connection**. Lets one connection subscribe to the same destination multiple times, and it's echoed in every MESSAGE's `subscription` header
`ack`	optional	`auto` (default), `client`, or `client-individual`

Brokers often support **extra subscription headers**, e.g., RabbitMQ `prefetch-count`, `x-queue-name`, `durable`, `auto-delete`, `exclusive`; ActiveMQ `selector` (JMS-style SQL filter), `activemq.prefetchSize`.

```
UNSUBSCRIBE
id:sub-0

^@
```

`id` is REQUIRED and must match a previous SUBSCRIBE's id.

4.8 Receiving: MESSAGE

```
MESSAGE
subscription:sub-0
message-id:007
destination:/topic/room.r1
ack:a1b2
content-type:application/json
content-length:41

{"from":"bob","text":"hi","ts":1700000000}^@
```

Header	Status	Meaning
`destination`	**REQUIRED**	Destination the message was sent to
`message-id`	**REQUIRED**	Unique id for this message (assigned by server)
`subscription`	**REQUIRED**	The `id` of the matching SUBSCRIBE
`ack`	present if the subscription needs explicit ack (1.2)	Arbitrary value to put in the ACK/NACK `id` header
`content-type`, `content-length`	should	Body description
user headers	–	Copied from SEND by most brokers

A MESSAGE is always a response to a SUBSCRIBE (servers never send MESSAGE frames without one).

4.9 Acknowledgement modes: auto, client, client-individual

ELI5:

- **auto** — The postman drops the letter in your box and forgets about it. If you lose it, too bad.
- **client** — The postman waits for you to sign. Signing for letter #5 means "I got #1–#5" (**cumulative**).
- **client-individual** — You sign for each letter separately. Signing #5 says nothing about #3.

Mode	Client sends ACK?	Cumulative?	If connection drops before ACK	Typical use
`auto` (default)	No	–	Message considered delivered when sent; may be **lost**	UI updates, best-effort
`client`	Yes	**Yes:** ACK of message M acks M and **all previous** messages on that subscription	Unacked messages may be redelivered (possibly to another client)	Ordered streams where you process sequentially
`client-individual`	Yes	**No:** acks only that message	Unacked messages may be redelivered	Parallel processing; work queues

4.9.1 ACK and NACK (STOMP 1.2)

```
ACK
id:a1b2
transaction:tx1

^@
```

```
NACK
id:a1b2

^@
```

- 1.2: `id` is **REQUIRED** and equals the MESSAGE's `ack` header.
- 1.1: ACK/NACK required `subscription` and `message-id` headers instead (this changed in 1.2. Old code sending `message-id` breaks against 1.2 servers).
- `NACK` (1.1+) = "I did **not** consume this message." The server may redeliver it, send it to a dead-letter queue, or discard it (broker policy). NACK follows the same cumulative rules as ACK for the subscription's mode.
- Both may be part of a **transaction**.

> **Note:** Spring's **simple broker** does **not** implement ACK/NACK (it's effectively always `auto`). If you need real acknowledgements and redelivery, use a full broker (RabbitMQ/ActiveMQ) via the relay (Chapter 7). RabbitMQ also supports `prefetch-count` to limit unacked messages per subscription.

4.10 Receipts: RECEIPT

Any **client frame except CONNECT** can carry `receipt:<id>`. The server replies:

```
RECEIPT
receipt-id:send-42

^@
```

- Means: "I've **processed** the frame with that receipt id."
- Because STOMP is stream-based, a receipt is also a **cumulative** acknowledgement that all **previous** frames were **received** (though not necessarily fully processed).
- Use cases: confirm a SEND reached the broker; confirm a SUBSCRIBE is active **before** triggering an action whose result you must not miss; graceful DISCONNECT.
- A receipt says nothing about subscribers having received the message.

Pitfall: Spring's simple broker has limited receipt support (Spring docs: the simple broker "does not support acks, receipts, and some other features"). Spring does send a RECEIPT for **DISCONNECT** frames that carry a `receipt` header, but don't build business logic on SEND receipts with the simple broker. With the broker relay, receipts come from the real broker.

4.11 Errors: ERROR

```
ERROR
receipt-id:send-42
content-type:text/plain
content-length:56
message:malformed frame received

Destination header missing on SEND frame (receipt send-42)^@
```

- `message` header: a short description. Body: details.
- If the error relates to a frame with a `receipt` header, the ERROR should include `receipt-id`.
- **The server MUST close the connection right after sending ERROR.** (STOMP 1.2 suggests the server **linger** briefly so the client actually receives the ERROR before the socket closes.) Spring closes the WebSocket with close code **1002** (`CloseStatus.PROTOCOL_ERROR`) right after sending an ERROR frame.
- So in STOMP, **an ERROR is fatal to the connection.** Not ideal for "validation failed" style app errors. In Spring, handle application errors with `@MessageExceptionHandler` and send an error **MESSAGE** to a user queue (e.g., `/user/queue/errors`) instead of letting it become a STOMP ERROR (Chapter 5).

4.12 Transactions: BEGIN / COMMIT / ABORT

```
BEGIN
transaction:tx1

^@
SEND
destination:/queue/orders
transaction:tx1

{"id":1}^@
ACK
id:a1b2
transaction:tx1
```

```
^@
COMMIT
transaction:tx1

^@
```

- `transaction` header **REQUIRED** on BEGIN, COMMIT, ABORT. The id must be unique per connection.
- SENDs and ACK/NACKs carrying the transaction id are held until **COMMIT** (then applied atomically) or discarded on **ABORT**.
- Any un-committed transaction is **implicitly aborted** if the client sends DISCONNECT or the connection fails.
- Support depends on the broker (Spring simple broker: no; RabbitMQ/ActiveMQ: yes).
- Rarely used in browser apps; more common for back-end STOMP clients.

4.13 Disconnecting gracefully: DISCONNECT

```
DISCONNECT
receipt:77

^@
```

Server:

```
RECEIPT
receipt-id:77

^@
```

1. Client sends DISCONNECT with a receipt.
2. Waits for RECEIPT. That guarantees all previous frames were received.
3. Closes the socket.

The client must not send more frames after DISCONNECT. Just closing the socket without DISCONNECT is legal but might lose frames in flight.

4.14 Frames & headers quick reference table

Frame	Direction	Required headers	Optional headers	Body
CONNECT / STOMP	C → S	`accept-version`, `host`	`login`, `passcode`, `heart-beat`	No
CONNECTED	S → C	`version`	`session`, `server`, `heart-beat`	No
SEND	C → S	`destination`	`transaction`, + any	**Yes**
SUBSCRIBE	C → S	`destination`, `id`	`ack`	No

Frame	Direction	Required headers	Optional headers	Body
UNSUBSCRIBE	C → S	`id`		No
ACK	C → S	`id`	`transaction`	No
NACK	C → S	`id`	`transaction`	No
BEGIN	C → S	`transaction`		No
COMMIT	C → S	`transaction`		No
ABORT	C → S	`transaction`		No
DISCONNECT	C → S		`receipt`	No
MESSAGE	S → C	`destination`, `message-id`, `subscription`	`ack`, + any	**Yes**
RECEIPT	S → C	`receipt-id`		No
ERROR	S → C		`message`	**Yes**

Plus: `content-length`, `content-type` on frames with bodies; `receipt` on any client frame except CONNECT.

4.15 A complete annotated session

```
>>> = client to server,  <<< = server to client,  ^@ = NULL

>>> CONNECT
>>> accept-version:1.2,1.1,1.0
>>> host:localhost
>>> heart-beat:10000,10000
>>> Authorization:Bearer eyJhbGciOi...
>>>
>>> ^@
<<< CONNECTED
<<< version:1.2
<<< heart-beat:10000,10000
<<< user-name:anna                ← Spring adds this when a user is authenticated
<<<
<<< ^@
>>> SUBSCRIBE
>>> id:sub-0
>>> destination:/topic/room.general
>>>
>>> ^@
>>> SUBSCRIBE
>>> id:sub-1
>>> destination:/user/queue/errors
>>>
>>> ^@
>>> SEND
>>> destination:/app/chat.send
>>> content-type:application/json
>>> content-length:35
>>>
>>> {"room":"general","text":"hello!"}^@
<<< MESSAGE
<<< destination:/topic/room.general
<<< content-type:application/json
```

```
<<< subscription:sub-0
<<< message-id:5ry2dxkc-0
<<< content-length:58
<<<
<<< {"from":"anna","room":"general","text":"hello!","ts":...}^@
<<< (newline)                        ← server heart-beat when idle
>>> (newline)                        ← client heart-beat when idle
>>> DISCONNECT
>>> receipt:close-1
>>>
>>> ^@
<<< RECEIPT
<<< receipt-id:close-1
<<<
<<< ^@
```

4.16 Write a tiny STOMP client by hand (to truly understand it)

JS

```
// Educational STOMP 1.2 client over a browser WebSocket. No reconnect, no acks.
class TinyStomp {
  constructor(url) {
    this.ws = new WebSocket(url, ['v12.stomp']);
    this.handlers = new Map();
    this.nextId = 0;
    this.buffer = '';
    this.ws.onmessage = (e) => this.onData(e.data);
  }

  frame(command, headers = {}, body = '') {
    const lines = [command];
    for (const [k, v] of Object.entries(headers)) lines.push(`${k}:${v}`);
    // content-length in BYTES, not characters
    if (body) lines.push(`content-length:${new TextEncoder().encode(body).length}`);
    return lines.join('\n') + '\n\n' + body + '\0';
  }

  connect(headers, onConnected) {
    this.ws.onopen = () =>
      this.ws.send(this.frame('CONNECT', { 'accept-version': '1.2', host: location.hostname, 'heart-
beat': '0,0', ...headers }));
    this.onConnected = onConnected;
  }

  subscribe(destination, cb) {
    const id = `sub-${this.nextId++}`;
    this.handlers.set(id, cb);
    this.ws.send(this.frame('SUBSCRIBE', { id, destination }));
    return () => { this.ws.send(this.frame('UNSUBSCRIBE', { id })); this.handlers.delete(id); };
  }

  send(destination, obj) {
    this.ws.send(this.frame('SEND', { destination, 'content-type': 'application/json' },
JSON.stringify(obj)));
  }

  onData(data) {
    this.buffer += data;
    let idx;
    while ((idx = this.buffer.indexOf('\0')) >= 0) {          // naive: assumes no NULL inside bodies
      const raw = this.buffer.slice(0, idx).replace(/^(\r?\n)+/, ''); // strip heart-beats
      this.buffer = this.buffer.slice(idx + 1);
```

```
      if (!raw) continue;
      const [head, ...rest] = raw.split(/\r?\n\r?\n/);
      const body = rest.join('\n\n');
      const [command, ...headerLines] = head.split(/\r?\n/);
      const headers = {};
      for (const line of headerLines) {
        const i = line.indexOf(':');
        const k = line.slice(0, i);
        if (!(k in headers)) headers[k] = line.slice(i + 1);    // first header wins (unescape omitted)
      }
      if (command === 'CONNECTED') this.onConnected?.(headers);
      else if (command === 'MESSAGE') this.handlers.get(headers.subscription)?.(JSON.parse(body),
headers);
      else if (command === 'ERROR') console.error('STOMP ERROR', headers.message, body);
    }
  }
}
```

This deliberately skips header unescaping, `content-length`-based reading, heart-beats and reconnects: exactly the things real libraries do for you. Use `@stomp/stompjs` in real code.

4.17 STOMP vs other messaging protocols

	STOMP	MQTT	AMQP 0-9-1	AMQP 1.0	Custom JSON over WS
Format	Text frames (HTTP-like)	Compact binary	Binary	Binary	Whatever you invent
Complexity	Very low	Low	High	High	Varies
Routing model	Opaque destinations (broker-defined)	Topic hierarchy with `+` / `#` wildcards	Exchanges/bindings/queues	Links/addresses	Yours
QoS / acks	auto/client/client-individual, receipts	QoS 0/1/2	Acks, confirms	Rich settlement	Yours
Browser use	Excellent (over WS)	Good (MQTT over WS)	No (not in browsers directly)	Rare	Common
Typical brokers	RabbitMQ, ActiveMQ, Spring simple broker	Mosquitto, EMQX, HiveMQ, RabbitMQ	RabbitMQ	ActiveMQ Artemis, Azure Service Bus, Qpid	–
Best for	Web apps with Spring/RabbitMQ, simple interop	IoT, constrained devices, mobile	Back-end messaging	Enterprise interop	Small apps, full control

---

4.18 Strengths and weaknesses of STOMP

Strengths: human-readable, trivially debuggable in DevTools, simple, many clients, Spring's first-class support, pub/sub + headers + acks + receipts + heart-beats standardised, works with real brokers.

Weaknesses: text-based (bigger than binary protocols for high-frequency data), `ERROR` always kills the connection, no built-in request/response correlation (you add `correlation-id` / `reply-to` headers by convention), destination semantics vary per broker, no native message ordering/replay/persistence guarantees beyond what the broker offers, spec has been static since 2012.

4.19 Chapter summary

- Frame = `COMMAND\n headers \n\n body \0`. Headers are UTF-8, case-sensitive, untrimmed, escaped (`\n \r \c \\`) except in CONNECT/CONNECTED. First repeated header wins.
- `content-length` is in bytes, allows NULLs in body.
- Client commands: CONNECT/STOMP, SEND, SUBSCRIBE, UNSUBSCRIBE, ACK, NACK, BEGIN, COMMIT, ABORT, DISCONNECT. Server: CONNECTED, MESSAGE, RECEIPT, ERROR.
- Heart-beat negotiation: `max(cx, sy)` client → server, `max(sx, cy)` server → client, 0 disables.
- Ack modes: auto, client (cumulative), client-individual. 1.2 ACK uses `id` = MESSAGE `ack` header.
- ERROR closes the connection. Receipts confirm processing of a frame (and receipt of all before it).
- Destinations are broker-specific strings.

Next: Chapter 5: Spring WebSocket + STOMP Server

CHAPTER 5

Chapter 5: Spring WebSocket + STOMP, the Server Side

Goal: build and *understand* a production-grade STOMP server with Spring Boot. You'll learn the internal message flow, every annotation, every configuration hook, and the "why" behind them. Code targets Spring Boot 3.x/4.x (Spring Framework 6.x/7.x, `jakarta.*`). Spring has supported WebSocket/STOMP since **Spring Framework 4.0 (2013)**, and the core model hasn't changed since.

5.1 ELI5

Spring is like a **big office building**:

- The **front door** (`/ws` endpoint) is where visitors (clients) come in through the WebSocket tunnel.
- A **receptionist** (`clientInboundChannel`) takes every note visitors hand in.
- Notes addressed to "**/app/...**" go to **office workers** (your `@MessageMapping` methods). They read the note, do work, and may write a reply.
- Notes addressed to "**/topic/...**" or "**/queue/...**" go to the **mailroom** (the broker), which photocopies them to everyone who asked to receive that kind of note (subscribers).
- A **courier** (`clientOutboundChannel`) carries notes back out to visitors.
- The **intercom** (`SimpMessagingTemplate`) lets anyone in the building (a REST controller, a scheduled job) drop a note into the mailroom.

5.2 Dependencies

Maven:

XML

```xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-websocket</artifactId>
</dependency>

<!-- Only if you use an external broker relay (RabbitMQ/ActiveMQ).
     Spring's reference docs say: add io.projectreactor.netty:reactor-netty and io.netty:netty-all -->
<dependency>
  <groupId>io.projectreactor.netty</groupId>
  <artifactId>reactor-netty</artifactId>
</dependency>
<dependency>
  <groupId>io.netty</groupId>
  <artifactId>netty-all</artifactId>
</dependency>

<!-- Optional: security for messages -->
```

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-messaging</artifactId>
</dependency>

<!-- Optional: @Valid on payloads -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Gradle:

GROOVY

```
implementation 'org.springframework.boot:spring-boot-starter-websocket'
implementation 'io.projectreactor.netty:reactor-netty'                    // relay only
implementation 'io.netty:netty-all'                                      // relay only
implementation 'org.springframework.security:spring-security-messaging'   // security
```

`spring-boot-starter-websocket` brings `spring-websocket` + `spring-messaging` + an embedded servlet container with WebSocket support (Tomcat by default).

> Note: Spring's STOMP support is part of the **Servlet stack** (Spring MVC). **Spring WebFlux** has a reactive `WebSocketHandler` API but **no built-in STOMP broker/annotation support.** On WebFlux you handle raw WebSocket messages yourself, or consider RSocket.

5.3 The minimal working configuration

JAVA

```
package com.example.realtime.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.messaging.simp.config.MessageBrokerRegistry;
import org.springframework.web.socket.config.annotation.EnableWebSocketMessageBroker;
import org.springframework.web.socket.config.annotation.StompEndpointRegistry;
import org.springframework.web.socket.config.annotation.WebSocketMessageBrokerConfigurer;

@Configuration
@EnableWebSocketMessageBroker           // turns on STOMP over WebSocket + message broker
infrastructure
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws")                         // HTTP URL for the WebSocket handshake
                .setAllowedOriginPatterns("https://*.example.com", "http://localhost:[*]");
        // .withSockJS();  // optional: add SockJS fallback (then clients use http(s)://host/ws)
    }

    @Override
    public void configureMessageBroker(MessageBrokerRegistry registry) {
        registry.setApplicationDestinationPrefixes("/app"); // client SEND to /app/** → @MessageMapping
        registry.enableSimpleBroker("/topic", "/queue");    // in-memory broker handles /topic/** and
```

```
        /queue/**
        registry.setUserDestinationPrefix("/user");        // default is "/user"
    }
}
```

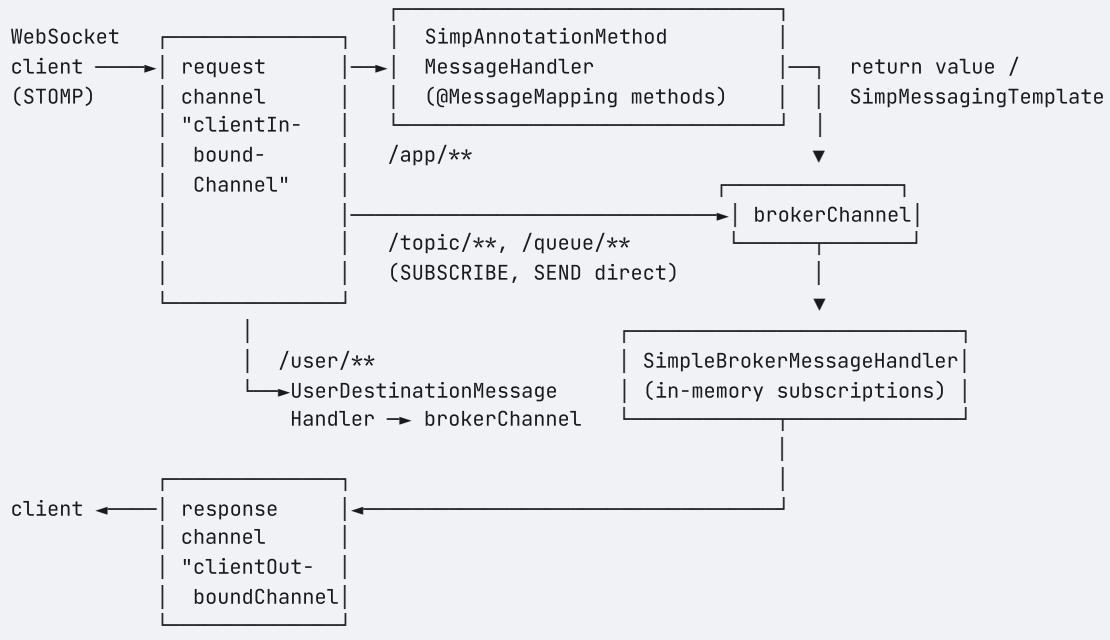
What each piece means:

Code	Meaning
`@EnableWebSocketMessageBroker`	Registers all beans: channels, handlers, broker, `SimpMessagingTemplate`, etc. (via `DelegatingWebSocketMessageBrokerConfiguration`)
`addEndpoint("/ws")`	Clients connect to `ws(s)://host:port/<context-path>/ws`
`setAllowedOrigins` / `setAllowedOriginPatterns`	Which `Origin` headers are accepted. **Default is same-origin only** (since Spring 4.1.5). Wildcard `*` with `setAllowedOrigins` is not allowed together with credentials in newer versions; use patterns
`withSockJS()`	Enables SockJS fallback transports (Chapter 6)
`setApplicationDestinationPrefixes("/app")`	Messages whose destination starts with `/app` go to **annotated controller methods**
`enableSimpleBroker("/topic","/queue")`	Messages (and subscriptions) to these prefixes go to the **in-memory broker**
`setUserDestinationPrefix("/user")`	Destinations starting with `/user` are **translated** to per-session destinations (§5.10)

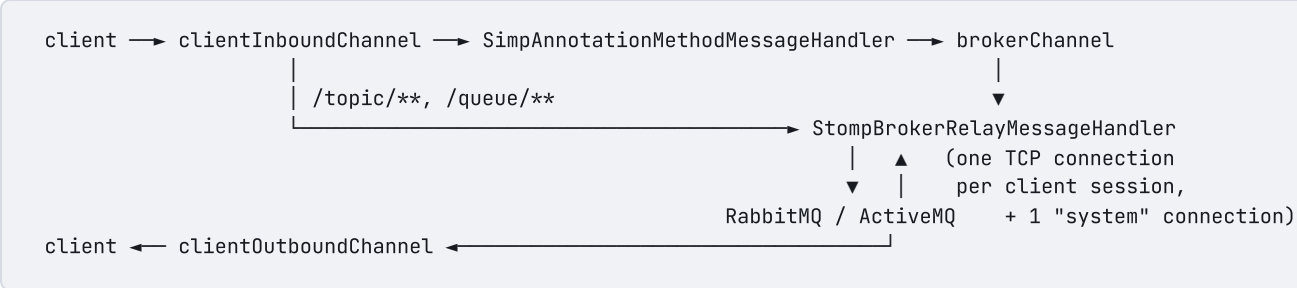
5.4 The architecture: channels and handlers

This diagram is the most important thing in the chapter. Learn to draw it from memory.

5.4.1 With the simple (in-memory) broker



5.4.2 With an external broker (STOMP broker relay)



5.4.3 The three channels

Channel bean	Carries	Default executor
`clientInboundChannel`	Messages **from** WebSocket clients	Thread pool; core size = 2 × available processors
`clientOutboundChannel`	Messages **to** WebSocket clients	Thread pool; core size = 2 × available processors
`brokerChannel`	Messages from server-side application code **to** the broker	Configurable via `configureBrokerChannel`

5.4.4 The main beans (names you'll see in logs and stack traces)

Bean / class	Job
`SubProtocolWebSocketHandler`	The `WebSocketHandler` registered at `/ws`. Delegates to sub-protocol handlers. Manages sessions, decorates them with `ConcurrentWebSocketSessionDecorator` (send limits)
`StompSubProtocolHandler`	Decodes WebSocket messages into STOMP `Message`s (with `StompDecoder` / `BufferingStompDecoder`), sends them to `clientInboundChannel`; encodes outbound messages (`StompEncoder`)
`SimpAnnotationMethodMessageHandler`	Finds and invokes `@MessageMapping` / `@SubscribeMapping` methods for `/app/**`
`SimpleBrokerMessageHandler`	In-memory broker: keeps a subscription registry (`DefaultSubscriptionRegistry`), fans out messages
`StompBrokerRelayMessageHandler`	Forwards messages to an external STOMP broker over TCP (Reactor Netty)
`UserDestinationMessageHandler`	Translates `/user/...` destinations to unique session destinations
`SimpMessagingTemplate` (`brokerMessagingTemplate`)	Send to the broker from anywhere
`DefaultSimpUserRegistry`	Tracks connected users and their sessions/subscriptions
`WebSocketMessageBrokerStats`	Periodically logs stats (every 30 min by default)

5.4.5 Spring's message abstraction (spring-messaging)

JAVA

```java
public interface Message<T> {
    T getPayload();            // byte[] from the wire, or your object after conversion
```

```
    MessageHeaders getHeaders(); // immutable map: simpMessageType, simpSessionId, simpDestination,
                                 // nativeHeaders (the raw STOMP headers), simpUser, ...
}
```

- **`MessageChannel`**: `send(Message)`. **`SubscribableChannel`**: has subscribers (`MessageHandler`s). `ExecutorSubscribableChannel` invokes them on a thread pool.
- **`MessageHeaderAccessor`** family helps read/write headers: `SimpMessageHeaderAccessor` (generic), `StompHeaderAccessor` (STOMP specifics like command, native headers, login).
- **Native headers** = the real STOMP headers (`Authorization`, `destination`, custom ones), stored under `nativeHeaders`. Access with `accessor.getFirstNativeHeader("Authorization")`.
- **`SimpMessageType`**: `CONNECT`, `CONNECT_ACK`, `MESSAGE`, `SUBSCRIBE`, `UNSUBSCRIBE`, `HEARTBEAT`, `DISCONNECT`, `DISCONNECT_ACK`, `OTHER`.

5.4.6 Follow one message end-to-end

1. Browser sends `SEND destination:/app/chat.send` with JSON body.
2. Tomcat receives the WebSocket text message → `SubProtocolWebSocketHandler.handleMessage`.
3. `StompSubProtocolHandler` decodes STOMP → `Message<byte[]>` with headers (`simpSessionId`, `simpUser`, `simpDestination=/app/chat.send`).
4. Sent to **`clientInboundChannel`** (interceptors run: your auth interceptor, Spring Security's authorization).
5. A thread from the inbound pool delivers it to `SimpAnnotationMethodMessageHandler` (destination starts with `/app`), which strips `/app` and matches `@MessageMapping("chat.send")` (or `/chat.send`).
6. Payload converted from JSON → `ChatMessage` via `MessageConverter` (Jackson).
7. Your method returns `ChatMessage`; `SendToMethodReturnValueHandler` sends it to **`brokerChannel`** with destination from `@SendTo`, or the default `/topic/chat.send`.
8. `SimpleBrokerMessageHandler` finds sessions subscribed to that destination and, for each, sends a `MESSAGE` to **`clientOutboundChannel`**.
9. `StompSubProtocolHandler` encodes to a STOMP MESSAGE frame → WebSocket text message → browser.

5.5 Annotated controllers

JAVA

```java
@Controller
public class ChatController {

    @MessageMapping("/chat.send")            // client sends to /app/chat.send
    @SendTo("/topic/chat.public")            // broadcast result to all subscribers
    public ChatMessage send(@Payload @Valid ChatMessage msg, Principal principal) {
        return msg.withSender(principal.getName()).withTimestamp(Instant.now());
    }
}
```

5.5.1 `@MessageMapping`

- Maps messages by **destination** (after the application prefix is removed).

- Can be on the class (prefix) and on methods.
- Supports Ant-style patterns (`/chat.*` , `/rooms/**`) and template variables (`/rooms/{roomId}`) read with `@DestinationVariable` .
- Default path separator is `/` . To use dots (`chat.room.1` , common with RabbitMQ), configure `registry.setPathMatcher(new AntPathMatcher("."))` .
- Message-handling methods are invoked on the **inbound channel thread pool**, concurrently. They must be **thread-safe** (no mutable shared state without synchronisation).

5.5.2 Supported method arguments

Argument	What you get
`Message<?>`	Full message
`MessageHeaders`	All headers
`MessageHeaderAccessor` , `SimpMessageHeaderAccessor` , `StompHeaderAccessor`	Typed header access
`@Payload` (optional annotation)	Converted body; supports `@Valid` / `@Validated`
`@Header("name")`	One header. Spring looks in the top-level message headers first and then in the native STOMP headers, so `@Header("Authorization")` finds a STOMP header sent by the client
`@Headers Map<String,Object>`	All headers
`@DestinationVariable`	Value from destination template `{var}`
`java.security.Principal`	Authenticated user from the WebSocket session

Session id: `@Header("simpSessionId") String sessionId` or `SimpMessageHeaderAccessor#getSessionId()` . Session attributes: `SimpMessageHeaderAccessor#getSessionAttributes()` (a per-WebSocket-session `Map` , filled by handshake interceptors or your code).

5.5.3 Return values

Situation	Where the return value goes
`@MessageMapping` , no `@SendTo`	`brokerChannel` , destination = `/topic` + **input destination** (e.g., `/app/greet` → `/topic/greet`)
`@SendTo("/topic/x")`	`brokerChannel` to `/topic/x` (supports `{var}` placeholders from the destination)
`@SendToUser("/queue/x")`	The current user's `/user/queue/x` (all their sessions, unless `broadcast = false` → only the session that sent the message)
`void` / `null`	Nothing sent
`CompletableFuture` / `ListenableFuture` /reactive types	Sent when completed (async)

`@SendTo` and `@SendToUser` can also be placed on the class (they then apply to all methods in it), and a method-level annotation overrides a class-level one.

5.5.4 `@SubscribeMapping` : request/reply on subscribe

```java
@SubscribeMapping("/rooms")         // client SUBSCRIBEs to /app/rooms
public List<Room> listRooms() {      // return value goes DIRECTLY back to that client
(clientOutboundChannel)
    return roomService.findAll();    // not through the broker, not to other subscribers
}
```

- Handles only `SUBSCRIBE` frames whose destination matches.
- The return value is sent **once, only to the subscribing client**, unless `@SendTo` / `@SendToUser` is present.
- The broker does not store this subscription (it's under `/app`, not a broker prefix).
- Classic use: **load initial data** when the UI subscribes. Unsubscribing is unnecessary but harmless.

5.5.5 Exception handling

```java
@Controller
public class ChatController {

    @MessageMapping("/chat.send")
    public void send(@Payload @Valid ChatMessage msg) { ... }

    @MessageExceptionHandler
    @SendToUser(destinations = "/queue/errors", broadcast = false)
    public ErrorDto handle(MethodArgumentNotValidException ex) {
        return new ErrorDto("VALIDATION_FAILED", ex.getMessage());
    }
}

// Global (since Spring 5.3): @ControllerAdvice also applies to @MessageExceptionHandler
@ControllerAdvice
public class GlobalWsExceptionHandler {
    @MessageExceptionHandler(AccessDeniedException.class)
    @SendToUser(destinations = "/queue/errors", broadcast = false)
    public ErrorDto accessDenied(AccessDeniedException ex) {
        return new ErrorDto("FORBIDDEN", "You can't do that");
    }
}
```

- `MethodArgumentNotValidException` here is `org.springframework.messaging.handler.annotation.support.MethodArgumentNotValidException`, not the MVC one (a common import mistake).
- Exceptions **not** handled by a `@MessageExceptionHandler` are logged and **the client receives nothing**. Silent failures are a classic bug.
- Exceptions thrown in **interceptors** on `clientInboundChannel` (e.g., auth failure on CONNECT) become a STOMP **ERROR frame** and the connection closes. Customise with `StompSubProtocolErrorHandler` (§5.13).

5.5.6 Validation

With `spring-boot-starter-validation` on the classpath, `@Payload @Valid ChatMessage msg` validates using Bean Validation (`@NotBlank`, `@Size(max = 2000)` ...). You can also provide a validator via `WebSocketMessageBrokerConfigurer#getValidator()`.

5.6 Sending from anywhere: `SimpMessagingTemplate`

JAVA

```java
@Service
@RequiredArgsConstructor
public class OrderNotifier {

    private final SimpMessagingTemplate messaging;   // or SimpMessageSendingOperations

    public void orderShipped(Order order) {
        // broadcast to everyone watching this order
        messaging.convertAndSend("/topic/orders." + order.getId(), OrderEvent.shipped(order));

        // private to the owner (all their sessions)
        messaging.convertAndSendToUser(order.getOwnerUsername(), "/queue/notifications",
                new Notification("Your order shipped!"));

        // with custom STOMP headers (they become MESSAGE headers)
        messaging.convertAndSend("/topic/orders." + order.getId(), OrderEvent.shipped(order),
                Map.of("event-type", "ORDER_SHIPPED", "correlation-id", order.getTraceId()));
    }
}
```

- Works from REST controllers, `@Scheduled` jobs, Kafka listeners, `@TransactionalEventListener`s, etc.
- `convertAndSend` sends to `brokerChannel`, so the destination must be a **broker** destination (`/topic`, `/queue`) or a **user** destination. Sending to `/app/...` from the template does **not** invoke your `@MessageMapping` methods.
- Converts the payload using the configured `MessageConverter`s (Jackson JSON by default with Boot).
- `setUserDestinationPrefix` must match the registry's if you configure it manually (Boot/ `@EnableWebSocketMessageBroker` wires it for you).

Pattern: send after commit. If you notify inside a DB transaction and the transaction rolls back, clients received a lie. Publish domain events and send in `@TransactionalEventListener(phase = AFTER_COMMIT)`.

5.7 Destinations and patterns

JAVA

```java
@Controller
@MessageMapping("rooms")
public class RoomController {

    @MessageMapping("{roomId}/send")                    // /app/rooms/42/send
    @SendTo("/topic/rooms.{roomId}")                     // placeholders resolved from destination
vars
    public ChatMessage send(@DestinationVariable String roomId, ChatMessage msg) {
        return msg.inRoom(roomId);
    }
}
```

- **Simple broker subscription patterns:** the simple broker's `DefaultSubscriptionRegistry` matches subscriptions using the `PathMatcher`. Clients **can** subscribe to patterns like `/topic/prices.*` (with a dot-separator matcher), and senders publish to concrete destinations. Be careful: pattern subscriptions are matched for each message and can be abused (subscribe to `/topic/**` to see everything). Restrict with security rules.
- **Simple broker selector header:** the simple broker supports a `selector` header on SUBSCRIBE carrying a Spring Expression Language (SpEL) expression evaluated against message headers (e.g., `headers.foo == 'bar'`). The header name is configurable via `setSelectorHeaderName(...)` on the simple broker registration (pass `null` to turn the feature off). This feature was the source of **CVE-2018-1270** (remote code execution through a crafted selector, fixed in Spring 5.0.5 / 4.3.16 by evaluating with a restricted `SimpleEvaluationContext`). Turn it off if you don't use it, and keep Spring patched.
- **RabbitMQ** has its own destination grammar (Chapter 7). With RabbitMQ use `.` as separator in `/topic` routing keys.

5.8 Configuring the broker in detail

```java
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Bean
    public ThreadPoolTaskScheduler wsHeartbeatScheduler() {
        ThreadPoolTaskScheduler s = new ThreadPoolTaskScheduler();
        s.setPoolSize(1);
        s.setThreadNamePrefix("ws-heartbeat-");
        s.initialize();
        return s;
    }

    @Override
    public void configureMessageBroker(MessageBrokerRegistry registry) {
        registry.setApplicationDestinationPrefixes("/app");
        registry.setUserDestinationPrefix("/user");
        registry.setPathMatcher(new AntPathMatcher("."));       // optional: dot-separated destinations
        registry.setPreservePublishOrder(true);                 // ordered delivery per session (5.1+)
        registry.setCacheLimit(1024);                           // destination cache for the simple
broker

        registry.enableSimpleBroker("/topic", "/queue")
                .setTaskScheduler(wsHeartbeatScheduler())       // REQUIRED for heart-beats to work
                .setHeartbeatValue(new long[] {10_000, 10_000}); // server can send every 10s, wants
client every 10s
    }
}
```

Heart-beat defaults for the simple broker: `0,0` (disabled) unless a `TaskScheduler` is configured; once a scheduler is set, the default becomes `10000,10000`.

Pitfall: circular bean dependency. Injecting a `TaskScheduler` bean that depends on the WebSocket configuration into the configurer can create a cycle. Defining a dedicated scheduler as shown, or using `@Lazy`, avoids it. (Spring's docs show `@Lazy TaskScheduler messageBrokerTaskScheduler` injection.)

5.8.1 External broker relay (production for clusters)

JAVA

```java
@Override
public void configureMessageBroker(MessageBrokerRegistry registry) {
    registry.setApplicationDestinationPrefixes("/app");
    registry.setPathMatcher(new AntPathMatcher("."));
    registry.enableStompBrokerRelay("/topic", "/queue", "/exchange", "/amq/queue")
            .setRelayHost("rabbitmq.internal")
            .setRelayPort(61613)
            .setVirtualHost("/")                          // STOMP "host" header sent in CONNECT
            .setClientLogin("ws-client").setClientPasscode("***")   // used for each client session's
connection
            .setSystemLogin("ws-system").setSystemPasscode("***")   // used for the shared "system"
connection
            .setSystemHeartbeatSendInterval(10_000)
            .setSystemHeartbeatReceiveInterval(10_000)
            .setUserDestinationBroadcast("/topic/unresolved-user-destination")  // multi-instance user
dests
            .setUserRegistryBroadcast("/topic/simp-user-registry");             // multi-instance user
registry
    // .setTcpClient(customReactorNettyTcpClient) for multiple broker addresses / TLS
}
```

Full details in Chapter 7.

5.9 Transport and container configuration

JAVA

```java
@Override
public void configureWebSocketTransport(WebSocketTransportRegistration registration) {
    registration
        .setMessageSizeLimit(128 * 1024)       // max inbound STOMP message size (default 64 KB)
        .setSendBufferSizeLimit(512 * 1024)    // max bytes buffered per session when client is slow
(default 512 KB)
        .setSendTimeLimit(15 * 1000)           // max time for a single send to a session (default 10 s)
        .setTimeToFirstMessage(30 * 1000);     // close if no STOMP frame (CONNECT) arrives in time
(default 60 s)
    // .addDecoratorFactory(handler -> new MyLoggingWebSocketHandlerDecorator(handler));
}
```

Setting	Default	Meaning / when to change
`messageSizeLimit`	64 KB	Larger inbound frames → session closed. STOMP frames split across multiple WebSocket messages are re-assembled up to this size. Raise if clients legitimately send bigger messages
`sendBufferSizeLimit`	512 KB	If a client is slow, outbound messages queue per session. Exceeding the limit **closes the session** (or drops, depending on overflow strategy)
`sendTimeLimit`	10 s	If one send blocks longer than this, the session is considered unreliable and closed

Setting	Default	Meaning / when to change
`timeToFirstMessage`	60 s	Protects against clients that connect and never send CONNECT (resource exhaustion)

Container-level buffer sizes (raw WebSocket messages):

```java
@Bean
public ServletServerContainerFactoryBean createWebSocketContainer() {
    ServletServerContainerFactoryBean container = new ServletServerContainerFactoryBean();
    container.setMaxTextMessageBufferSize(128 * 1024);
    container.setMaxBinaryMessageBufferSize(128 * 1024);
    container.setMaxSessionIdleTimeout(0L);   // ms; 0 = no container idle timeout (rely on heart-beats)
    return container;
}
```

Pitfall: two limits. A message must pass **both** the container's per-message buffer (Tomcat's default text/binary buffer is **8 KB**; a single WebSocket message bigger than that is rejected with close code **1009**) **and** Spring's STOMP `messageSizeLimit` (64 KB, which applies after Spring re-assembles a STOMP frame that a client split across several WebSocket messages). "Works locally with small payloads, fails in prod with big JSON" is often this. Symptoms: close code **1009**, or the session is closed with a "message too big"/ `StompConversionException` log line. Fix: raise both limits deliberately, or (better) keep messages small and fetch large data over HTTP.

5.10 User destinations: private messages explained

ELI5: Everyone subscribes to the **same-looking** mailbox name `/user/queue/notifications`, but Spring secretly gives each person (each session) their **own** box behind the scenes, so nobody can read anyone else's mail.

5.10.1 How it works

Client side:

```js
client.subscribe('/user/queue/notifications', cb);
```

Server side:

```java
messaging.convertAndSendToUser("anna", "/queue/notifications", payload);
```

What happens:

1. On **SUBSCRIBE** to `/user/queue/notifications`, `UserDestinationMessageHandler` (via `DefaultUserDestinationResolver`) rewrites it to a **session-unique** destination: `/queue/notifications-user{sessionId}` (e.g., `/queue/notifications-userwmv3cnnv`). The broker stores *that* subscription.

2. `convertAndSendToUser("anna", "/queue/notifications", x)` sends to `/user/anna/queue/notifications`.
3. The resolver looks up **all sessions of user "anna"** in the `SimpUserRegistry` and sends to each `/queue/notifications-user{sessionId}`.
4. Each of anna's tabs/devices receives it; nobody else does.

Requirements & gotchas:

- The user is identified by `Principal#getName()`. If the session has **no Principal**, sending by username can't work (there's no name → session mapping). Set the user at handshake or CONNECT (Chapter 8).
- The name you pass to `convertAndSendToUser` must equal `principal.getName()` exactly. With JWT this is usually the `sub` claim. With OAuth2 resource server it's the `sub` by default unless you change `principalClaimName`. A mismatch (email vs id) = messages vanish silently.
- With **@SendToUser**, the current user is taken from the incoming message's Principal.
- **Multiple server instances:** sessions on other instances aren't in the local registry. Enable `userDestinationBroadcast` and `userRegistryBroadcast` on the broker relay (Chapter 7).
- With an external broker, each session-unique destination becomes a **broker queue** (e.g., RabbitMQ). Ensure they're auto-deleted when sessions end.

5.10.2 Sending to a specific session (no Principal needed)

JAVA

```java
private MessageHeaders headersForSession(String sessionId) {
    SimpMessageHeaderAccessor accessor = SimpMessageHeaderAccessor.create(SimpMessageType.MESSAGE);
    accessor.setSessionId(sessionId);
    accessor.setLeaveMutable(true);
    return accessor.getMessageHeaders();
}

// The "user" argument is the session id itself:
messaging.convertAndSendToUser(sessionId, "/queue/reply", payload, headersForSession(sessionId));
```

Useful for anonymous users (e.g., a public support widget). The client still subscribes to `/user/queue/reply`.

5.10.3 `@SendToUser` broadcast vs single session

JAVA

```java
@MessageMapping("/report.generate")
@SendToUser(destinations = "/queue/report", broadcast = false)  // only the tab that asked
public Report generate(ReportRequest req) { ... }
```

`broadcast = true` (default) → all sessions of that user (all tabs/devices).

5.11 Session lifecycle events

Spring publishes `ApplicationEvent`s you can listen to with `@EventListener`:

Event	When
`SessionConnectEvent`	A new STOMP CONNECT arrived (before the broker responds). Contains the CONNECT message incl. native headers, session id, user
`SessionConnectedEvent`	Broker replied CONNECTED (shortly after connect). Message is the CONNECTED (`CONNECT_ACK`), the original CONNECT is in header `simpConnectMessage`
`SessionSubscribeEvent`	A SUBSCRIBE arrived
`SessionUnsubscribeEvent`	An UNSUBSCRIBE arrived
`SessionDisconnectEvent`	The STOMP session ended: DISCONNECT frame **or** WebSocket closed. **May be published more than once per session** → handlers must be idempotent. Has `getCloseStatus()`
`BrokerAvailabilityEvent`	Broker became available/unavailable (important for relay: TCP to RabbitMQ lost)

JAVA

```java
@Component
@RequiredArgsConstructor
@Slf4j
public class PresenceTracker {

    private final SimpMessagingTemplate messaging;
    private final Map<String, String> sessionToUser = new ConcurrentHashMap<>();

    @EventListener
    public void onConnected(SessionConnectedEvent event) {
        StompHeaderAccessor acc = StompHeaderAccessor.wrap(event.getMessage());
        Principal user = event.getUser();
        if (user != null) {
            sessionToUser.put(acc.getSessionId(), user.getName());
            messaging.convertAndSend("/topic/presence", new Presence(user.getName(), true));
        }
    }

    @EventListener
    public void onDisconnect(SessionDisconnectEvent event) {
        String username = sessionToUser.remove(event.getSessionId());   // idempotent: second call gets
null
        if (username != null && !sessionToUser.containsValue(username)) { // last session of this user
            messaging.convertAndSend("/topic/presence", new Presence(username, false));
        }
        log.info("Disconnected session={} status={}", event.getSessionId(), event.getCloseStatus());
    }
}
```

Pitfall: Events are published **synchronously** on the thread that handles the message/close. Don't do slow work (DB calls, HTTP calls) directly in listeners. Hand off with `@Async` or a queue. In a **cluster**, local in-memory presence maps are wrong; use Redis or the multi-server user registry.

Connected users: inject `SimpUserRegistry` :

JAVA

```java
int online = simpUserRegistry.getUserCount();
SimpUser anna = simpUserRegistry.getUser("anna");          // sessions & subscriptions
```

```
Set<SimpSubscription> subs = simpUserRegistry.findSubscriptions(s ->
s.getDestination().startsWith("/topic/rooms."));
```

5.12 Interceptors

5.12.1 `HandshakeInterceptor` (HTTP level, before 101)

JAVA

```
public class TenantHandshakeInterceptor implements HandshakeInterceptor {
    @Override
    public boolean beforeHandshake(ServerHttpRequest request, ServerHttpResponse response,
                                   WebSocketHandler wsHandler, Map<String, Object> attributes) {
        String tenant = request.getHeaders().getFirst("X-Tenant-Id");      // from proxy, or query param
        if (tenant == null) { response.setStatusCode(HttpStatus.BAD_REQUEST); return false; } // reject
        attributes.put("tenantId", tenant);   // becomes WebSocket session attribute
(SimpMessageHeaderAccessor#getSessionAttributes)
        return true;
    }
    @Override
    public void afterHandshake(ServerHttpRequest req, ServerHttpResponse res, WebSocketHandler h,
Exception ex) {}
}

registry.addEndpoint("/ws")
        .addInterceptors(new TenantHandshakeInterceptor(), new HttpSessionHandshakeInterceptor())
        .setHandshakeHandler(new CustomPrincipalHandshakeHandler());
```

- `HttpSessionHandshakeInterceptor` copies `HttpSession` attributes into WebSocket session attributes.
- `DefaultHandshakeHandler#determineUser` can be overridden to assign a `Principal` at handshake time (e.g., generate an anonymous UUID principal so user destinations work for guests):

JAVA

```
public class CustomPrincipalHandshakeHandler extends DefaultHandshakeHandler {
    @Override
    protected Principal determineUser(ServerHttpRequest request, WebSocketHandler wsHandler,
                                      Map<String, Object> attributes) {
        Principal user = request.getPrincipal();   // from Spring Security if the handshake was
authenticated
        return user != null ? user : new AnonymousPrincipal(UUID.randomUUID().toString());
    }
}
```

5.12.2 `ChannelInterceptor` (STOMP level, every message)

JAVA

```
public class LoggingAndAuthInterceptor implements ChannelInterceptor {

    @Override
    public Message<?> preSend(Message<?> message, MessageChannel channel) {
        StompHeaderAccessor acc = MessageHeaderAccessor.getAccessor(message, StompHeaderAccessor.class);
        if (acc == null) return message;
        if (StompCommand.CONNECT.equals(acc.getCommand())) {
            // authenticate using native header; acc.setUser(...) (see Chapter 8)
        }
        return message;          // return null to silently drop; throw to reject (→ ERROR frame for
inbound)
    }
```

```
    @Override
    public void afterSendCompletion(Message<?> message, MessageChannel channel, boolean sent, Exception
ex) {
        // metrics, cleanup (e.g., clear ThreadLocal / MDC)
    }
}

@Override
public void configureClientInboundChannel(ChannelRegistration registration) {
    registration.interceptors(new LoggingAndAuthInterceptor());
    registration.taskExecutor().corePoolSize(8).maxPoolSize(32).queueCapacity(10_000);
}

@Override
public void configureClientOutboundChannel(ChannelRegistration registration) {
    registration.taskExecutor().corePoolSize(8).maxPoolSize(32);
}
```

Key details:

- Use `MessageHeaderAccessor.getAccessor(message, StompHeaderAccessor.class)` to get the **mutable** accessor of the message. `StompHeaderAccessor.wrap(message)` creates a **copy**; changes to a copy (like `setUser`) are lost unless you build a new message. "My `setUser` doesn't stick" is this bug.
- `preSend` runs on the **caller's thread** (for inbound: the WebSocket container's I/O thread), before the message is handed to the executor. `ExecutorChannelInterceptor#beforeHandle/afterMessageHandled` run on the executor thread (where your controller runs) and are the right place for `ThreadLocal`s like `SecurityContextHolder`, MDC logging context, tenant context.
- Interceptor **order** matters (your auth interceptor must run before Spring Security's authorization). See Chapter 8.

5.13 Customising STOMP ERROR frames

JAVA

```
public class CustomStompErrorHandler extends StompSubProtocolErrorHandler {
    @Override
    public Message<byte[]> handleClientMessageProcessingError(Message<byte[]> clientMessage, Throwable
ex) {
        Throwable cause = ex.getCause() != null ? ex.getCause() : ex;
        if (cause instanceof AccessDeniedException || cause instanceof BadCredentialsException) {
            StompHeaderAccessor acc = StompHeaderAccessor.create(StompCommand.ERROR);
            acc.setMessage("UNAUTHORIZED");              // ERROR frame "message" header
            acc.setLeaveMutable(true);
            return MessageBuilder.createMessage("Invalid or expired
token".getBytes(StandardCharsets.UTF_8),
                                                acc.getMessageHeaders());
        }
        return super.handleClientMessageProcessingError(clientMessage, ex);
    }
}

@Override
public void registerStompEndpoints(StompEndpointRegistry registry) {
    registry.addEndpoint("/ws").setAllowedOriginPatterns("https://*.example.com");
    registry.setErrorHandler(new CustomStompErrorHandler());
}
```

Never put stack traces or internal details into ERROR frames (information leak).

5.14 Message ordering

By default the inbound and outbound channels use thread pools, so **messages from/to the same client can be processed out of order**. Options:

- `registry.setPreservePublishOrder(true)` (`MessageBrokerRegistry`, 5.1+): messages published **to the same client session** from the broker are sent **in order** (one at a time per session). Small performance cost.
- `registry.setPreserveReceiveOrder(true)` (`StompEndpointRegistry`, 6.1+): messages **from the same client session** are handed to the inbound channel in order.

  > **Caveat (verified in Spring Framework 7.0.9 source and by test):** with this flag on, exceptions thrown by inbound `ChannelInterceptor`s (e.g., your JWT CONNECT check or subscription authorisation) are caught and **only logged** by `OrderedMessageChannelDecorator`. The message is dropped (so security holds), but **no STOMP ERROR frame is sent and the connection stays open**, so clients can't tell why nothing happened. Either avoid the flag when interceptors reject frames, or make clients time out CONNECT and handle it. (Bug Encyclopedia C10.)

- Without these, if order matters, add a sequence number to messages and reorder/ignore stale ones on the client.

5.15 Message converters

Spring Boot auto-configures JSON conversion with Boot's Jackson `ObjectMapper` (so your `spring.jackson.*` settings and modules like `JavaTimeModule` apply).

JAVA

```java
@Override
public boolean configureMessageConverters(List<MessageConverter> messageConverters) {
    // Add custom converters (e.g., protobuf). Return true to also add the default converters; false to
use only yours.
    messageConverters.add(new MyProtobufMessageConverter());
    return true;
}
```

Defaults (when not using Boot's customization): `StringMessageConverter`, `ByteArrayMessageConverter`, and a Jackson JSON converter if Jackson is present. The Jackson 2 converter is `MappingJackson2MessageConverter`, which is **deprecated for removal since Spring Framework 7.0**. Its Jackson 3 replacement is `JacksonJsonMessageConverter` **(since 7.0)**. On Spring 6.x / Boot 3.x keep using `MappingJackson2MessageConverter`; on Spring 7 / Boot 4 use `JacksonJsonMessageConverter`.

Conversion is chosen by the `content-type` header. **Clients should send** `content-type:application/json` for JSON payloads (stompjs doesn't add it automatically). Without it Spring still attempts conversion with the available converters, and the behaviour can surprise you (e.g., the target type is `String` and you get raw JSON).

5.16 Monitoring: `WebSocketMessageBrokerStats`

Spring logs a line like this every 30 minutes (INFO, category `org.springframework.web.socket.config.WebSocketMessageBrokerStats`):

```
WebSocketSession[11 current WS(5)-HttpStream(6)-HttpPoll(0), 372 total, 0 closed abnormally
(0 connect failure, 0 send limit, 0 transport error)], stompSubProtocol[processed CONNECT(302)-
CONNECTED(221)-DISCONNECT(0)],
stompBrokerRelay[null], inboundChannel[pool size = 8, active threads = 0, queued tasks = 0, completed
tasks = 5862],
outboundChannel[pool size = 8, active threads = 0, queued tasks = 0, completed tasks = 3218],
sockJsScheduler[pool size = 4, active threads = 0, queued tasks = 0, completed tasks = 27]
```

How to read it:

- `closed abnormally (... send limit ...)` → slow clients exceeding `sendBufferSizeLimit` / `sendTimeLimit`.
- `connect failure` → sessions closed before CONNECTED (timeouts, auth failures).
- `queued tasks` growing on inbound/outbound → thread pool saturated (slow controllers or slow clients).
- Change the frequency with `stats.setLoggingPeriod(ms)`. Expose numbers to Prometheus via Micrometer gauges you register yourself (Chapter 9).

5.17 Plain WebSocket in Spring (without STOMP), for completeness

JAVA

```java
@Configuration
@EnableWebSocket
public class RawWsConfig implements WebSocketConfigurer {
    @Override
    public void registerWebSocketHandlers(WebSocketHandlerRegistry registry) {
        registry.addHandler(new EchoHandler(),
"/raw").setAllowedOriginPatterns("https://*.example.com");
    }
}

public class EchoHandler extends TextWebSocketHandler {
    @Override public void afterConnectionEstablished(WebSocketSession s) { }
    @Override protected void handleTextMessage(WebSocketSession s, TextMessage m) throws IOException {
        s.sendMessage(new TextMessage("echo: " + m.getPayload()));
    }
    @Override public void afterConnectionClosed(WebSocketSession s, CloseStatus status) { }
}
```

- `WebSocketSession.sendMessage` is **not thread-safe** for concurrent sends. Wrap with `ConcurrentWebSocketSessionDecorator` if multiple threads send to the same session ("TEXT_PARTIAL_WRITING" / `IllegalStateException: The remote endpoint was in state [TEXT_PARTIAL_WRITING]` is the classic Tomcat symptom).
- Jakarta WebSocket (JSR 356) `@ServerEndpoint` also works but doesn't integrate with Spring messaging.

- When to choose raw over STOMP: custom binary protocols, game loops, proxies, or very high-frequency data where STOMP's text overhead matters.

5.18 Complete example: a real-time to-do list server

A realistic feature: users share to-do lists; changes appear live on all members' devices.

JAVA

```
// --- DTOs ---
public record TodoItem(String id, String listId, String title, boolean done, long version) {}
public record TodoEvent(String type, TodoItem item, String actor, Instant at) {}       // type:
CREATED/UPDATED/DELETED
public record TodoCommand(@NotBlank String listId, String itemId, @Size(max = 500) String title, Boolean
done,
                          Long expectedVersion, @NotBlank String clientRequestId) {}
public record ErrorDto(String code, String message, String clientRequestId) {}

// --- Config ---
@Configuration
@EnableWebSocketMessageBroker
public class WsConfig implements WebSocketMessageBrokerConfigurer {
    @Override
    public void registerStompEndpoints(StompEndpointRegistry r) {
        r.addEndpoint("/ws").setAllowedOriginPatterns("https://app.example.com", "http://localhost:
[*]");
    }
    @Override
    public void configureMessageBroker(MessageBrokerRegistry r) {
        r.setApplicationDestinationPrefixes("/app");
        r.enableSimpleBroker("/topic", "/queue")
         .setTaskScheduler(heartbeatScheduler())
         .setHeartbeatValue(new long[]{10_000, 10_000});
        r.setPreservePublishOrder(true);
    }
    @Bean ThreadPoolTaskScheduler heartbeatScheduler() {
        var s = new ThreadPoolTaskScheduler(); s.setPoolSize(1); s.setThreadNamePrefix("wss-hb-");
s.initialize(); return s;
    }
}

// --- Controller ---
@Controller
@RequiredArgsConstructor
public class TodoWsController {
    private final TodoService todos;
    private final ListAccessService access;

    /** Initial snapshot when the client subscribes to /app/lists/{listId} */
    @SubscribeMapping("/lists/{listId}")
    public List<TodoItem> snapshot(@DestinationVariable String listId, Principal user) {
        access.requireMember(user.getName(), listId);
        return todos.findByList(listId);
    }

    /** Client sends to /app/todos.upsert */
    @MessageMapping("/todos.upsert")
    public void upsert(@Payload @Valid TodoCommand cmd, Principal user) {
        access.requireMember(user.getName(), cmd.listId());
        todos.upsert(cmd, user.getName());           // publishes TodoChanged domain event; idempotent
by clientRequestId
    }
```

```
    @MessageExceptionHandler({OptimisticLockingFailureException.class})
    @SendToUser(destinations = "/queue/errors", broadcast = false)
    public ErrorDto conflict(OptimisticLockingFailureException ex) {
        return new ErrorDto("CONFLICT", "Item changed by someone else, please refresh", null);
    }
}

// --- Fan-out after DB commit ---
@Component
@RequiredArgsConstructor
public class TodoBroadcaster {
    private final SimpMessagingTemplate messaging;

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void on(TodoChanged e) {
        messaging.convertAndSend("/topic/lists." + e.item().listId(),
                new TodoEvent(e.type(), e.item(), e.actor(), Instant.now()));
    }
}
```

Client flow (Chapter 6 has the code):

1. Connect with JWT in CONNECT headers.
2. `subscribe('/topic/lists.L1')` for live changes **and** `subscribe('/app/lists/L1')` for the snapshot. Subscribe to the topic **first** so no change is missed between snapshot and live stream, and use `version` to discard stale events.
3. `subscribe('/user/queue/errors')`.
4. `publish('/app/todos.upsert', {...clientRequestId})` and optimistically update the UI; reconcile when the event arrives.

Security note: authorise **subscriptions** to `/topic/lists.{id}` (only members!), see Chapter 8.

5.19 Chapter summary

- `@EnableWebSocketMessageBroker` + `WebSocketMessageBrokerConfigurer`: endpoints, broker, prefixes, channels, transport, converters.
- Three channels (inbound, outbound, broker), and messages flow through handlers: annotation handler (`/app`), broker (`/topic`, `/queue`), user destination handler (`/user`).
- `@MessageMapping`, `@SubscribeMapping`, `@SendTo`, `@SendToUser`, `@MessageExceptionHandler`, `@DestinationVariable`, `Principal`.
- `SimpMessagingTemplate` sends to the broker from anywhere. `convertAndSendToUser` resolves user → sessions.
- Heart-beats with the simple broker need a `TaskScheduler`.
- Transport limits: 64 KB message, 512 KB send buffer, 10 s send time, 60 s time to first message (defaults).
- Ordering requires `setPreservePublishOrder` / `setPreserveReceiveOrder`.
- Events (`SessionConnectedEvent`, `SessionDisconnectEvent` …) for presence; disconnect may fire twice.

Next: Chapter 6: Clients

CHAPTER 6

Chapter 6: Clients (JavaScript, React, React Native, Angular, Java, Python, SockJS)

Goal: connect any client to a STOMP server correctly, robustly and securely, and know the platform-specific traps (especially React Native).

6.1 ELI5

The server is the post office. The **client** is your mailbox at home plus the person who checks it. A good client:

1. Connects and shows its ID card (CONNECT with token).
2. Says which mailboxes it wants (SUBSCRIBE).
3. Re-connects by itself if the line breaks, and **asks for its mailboxes again**.
4. Says "beep" regularly so the post office knows it's alive (heart-beats).
5. Cleans up when the screen that needed messages is closed (UNSUBSCRIBE).

6.2 Choosing a JavaScript library

Library	Status	Notes
`@stomp/stompjs` (v7)	**Actively maintained, the standard**	TypeScript, auto-reconnect, heart-beats, works in browsers, Node, React Native
`@stomp/rx-stomp`	Maintained	RxJS wrapper over stompjs; great for Angular
`@stomp/ng2-stompjs`	**Deprecated**	Use `rx-stomp` with an Angular service instead
`stompjs` (jmesnil, `Stomp.over(ws)`)	**Abandoned** (2014)	Old tutorials use it. Migrate
`webstomp-client`	Unmaintained	Avoid in new code
`sockjs-client`	Low activity	Only if you need SockJS fallbacks

BASH

```
npm install @stomp/stompjs
# optional
npm install @stomp/rx-stomp rxjs
npm install sockjs-client
```

6.3 `@stomp/stompjs` v7: complete guide

> Checked against `@stomp/stompjs` **7.5.0** (the latest release on npm at the time of writing). Option names and defaults below are taken from the package source.

6.3.1 Basic usage

TS

```
import { Client, IMessage, StompSubscription } from '@stomp/stompjs';

const client = new Client({
  brokerURL: 'wss://api.example.com/ws',          // native WebSocket URL (not http://)
  connectHeaders: { Authorization: `Bearer ${getAccessToken()}` },
  heartbeatIncoming: 10000,                        // want server heart-beats every 10s (0 = disable)
  heartbeatOutgoing: 10000,                        // will send heart-beats every 10s
  reconnectDelay: 5000,                            // ms between reconnect attempts (0 = no auto-
reconnect)
  connectionTimeout: 10000,                        // give up a connection attempt if CONNECTED not
received in time
  debug: (msg) => { if (import.meta.env.DEV) console.debug(msg); },
});

client.onConnect = (frame) => {
  // Called on EVERY successful (re)connect. Put all subscriptions here!
  client.subscribe('/topic/chat.public', (message: IMessage) => {
    const payload = JSON.parse(message.body);
    renderMessage(payload);
  });
  client.subscribe('/user/queue/errors', (m) => showError(JSON.parse(m.body)));
};

client.onStompError = (frame) => {
  // STOMP ERROR frame (server will close the connection)
  console.error('Broker error:', frame.headers['message'], frame.body);
};

client.onWebSocketClose = (evt) => console.warn('WS closed', evt.code, evt.reason);
client.onWebSocketError = (evt) => console.error('WS error', evt);

client.activate();      // start connecting (and keep reconnecting)

// Send (only when connected!)
function sendChat(text: string) {
  if (!client.connected) return queueForLater(text);
  client.publish({
    destination: '/app/chat.send',
    body: JSON.stringify({ text }),
    headers: { 'content-type': 'application/json' },
  });
}

// Shut down (e.g., logout)
await client.deactivate();   // sends DISCONNECT, stops reconnecting
```

6.3.2 Configuration reference (most important options)

Option	Default	Meaning
`brokerURL`	–	`ws://` or `wss://` URL. Ignored if `webSocketFactory` is set

Option	Default	Meaning
`webSocketFactory`	–	Function returning a WebSocket-like object (e.g., `new SockJS(url)`)
`connectHeaders`	`{}`	Headers for CONNECT (`login`, `passcode`, `Authorization`, custom)
`disconnectHeaders`	`{}`	Headers for DISCONNECT
`heartbeatIncoming` / `heartbeatOutgoing`	`10000` / `10000`	Heart-beat intervals in ms (see Chapter 4 negotiation)
`reconnectDelay`	`5000`	Delay before reconnect; `0` disables auto-reconnect
`reconnectTimeMode`	`ReconnectionTimeMode.LINEAR`	`EXPONENTIAL` doubles the delay after each failed attempt (no random jitter is added)
`maxReconnectDelay`	15 minutes	Cap for exponential mode (if set below `reconnectDelay`, it is raised to `reconnectDelay`)
`heartbeatToleranceMultiplier`	`2`	Server considered dead after `multiplier × incoming interval` with no data
`heartbeatStrategy`	`TickerStrategy.Interval`	`TickerStrategy.Worker` sends heart-beats from a Web Worker, which helps in throttled background tabs
`onHeartbeatReceived` / `onHeartbeatLost`	–	Callbacks for heart-beat events (handy for "connection quality" UI and metrics)
`onChangeState`	–	Activation state changes (`ACTIVE`, `DEACTIVATING`, `INACTIVE`)
`connectionTimeout`	`0` (off)	Fail and retry if not CONNECTED within this time
`beforeConnect`	–	**async** hook called before each (re)connect, the place to **refresh tokens**
`onConnect`, `onDisconnect`	–	STOMP-level connected/disconnected
`onStompError`	–	ERROR frame received
`onWebSocketClose`, `onWebSocketError`	–	Transport-level events
`onUnhandledMessage`	–	MESSAGE that matched no subscription (e.g., arrived after unsubscribe)
`onUnhandledReceipt`, `onUnhandledFrame`	–	Other unmatched frames
`stompVersions`	1.2, 1.1, 1.0	Versions offered
`splitLargeFrames`, `maxWebSocketChunkSize`	`false`, 8 KB	Non-standard: split large outgoing frames into chunks (Spring supports reassembly)
`forceBinaryWSFrames`	`false`	Send frames as binary WebSocket messages (React Native workaround)
`appendMissingNULLonIncoming`	`false`	Append the NULL terminator that some platforms strip (React Native workaround)
`discardWebsocketOnCommFailure`	`false`	On heart-beat failure, immediately discard the socket instead of waiting for close
`logRawCommunication`	`false`	Log raw frames in `debug`

6.3.3 Subscriptions

```ts
const sub: StompSubscription = client.subscribe(
  '/topic/lists.L1',
  (msg) => { /* ... */ },
  { id: 'list-L1' /* optional custom id */, ack: 'auto' }
);
sub.unsubscribe();   // sends UNSUBSCRIBE
```

- `subscribe()` throws if not connected. That's why you call it **inside** `onConnect`.
- On reconnect, **old subscriptions are gone** (server side). stompjs doesn't automatically re-subscribe; your `onConnect` runs again and re-creates them. `rx-stomp` *does* re-subscribe its `watch()`ed streams automatically.

6.3.4 Acknowledgements (with a full broker)

```ts
client.subscribe('/queue/jobs', (msg) => {
  try { process(JSON.parse(msg.body)); msg.ack(); }
  catch { msg.nack(); }
}, { ack: 'client-individual', 'prefetch-count': '10' /* RabbitMQ */ });
```

6.3.5 Receipts

```ts
const receiptId = crypto.randomUUID();
client.watchForReceipt(receiptId, () => console.log('server processed it'));
client.publish({ destination: '/queue/orders', body: '{}', headers: { receipt: receiptId } });
```

6.3.6 Binary messages

```ts
client.publish({ destination: '/app/upload.chunk', binaryBody: uint8Array,
                 headers: { 'content-type': 'application/octet-stream' } });
client.subscribe('/topic/bin', (m) => { const bytes = m.binaryBody; });
```

6.3.7 Token refresh before every connect

Pitfall (verified in stompjs 7.5.0 source and by test): `beforeConnect` must never throw. stompjs awaits it without a try/catch. If your token refresh fails (auth server down, offline), the rejection is unhandled **and the reconnect loop stops for good.** Always wrap it:

```ts
beforeConnect: async () => {
  try { client.connectHeaders = { Authorization: `Bearer ${await getToken()}` }; }
  catch { /* keep old headers; this attempt fails and normal backoff retries */ }
},
```

Also set `discardWebsocketOnCommFailure: true`, so an attempt stuck in CONNECTING (seen with Node's WebSocket) is dropped and the next attempt starts.

```ts
client.beforeConnect = async () => {
  const token = await authService.getFreshAccessToken();     // refresh if near expiry
  client.connectHeaders = { Authorization: `Bearer ${token}` };
};
```

Pitfall: token expired while connected. STOMP authenticates once at CONNECT. If the token expires an hour later, the connection stays authenticated unless the server enforces expiry (Chapter 8). When the server closes the session because of expiry, `beforeConnect` must fetch a fresh token or you'll loop forever with a stale one.

6.3.8 Publishing safely while disconnected

`client.publish` throws `TypeError: There is no underlying STOMP connection` when not connected. Pattern:

```ts
const outbox: Array<() => void> = [];
function safePublish(p: Parameters<Client['publish']>[0]) {
  if (client.connected) client.publish(p);
  else outbox.push(() => client.publish(p));
}
client.onConnect = () => {
  resubscribeAll();
  while (outbox.length) outbox.shift()!();   // flush; server must dedupe (idempotency key!)
};
```

6.4 Legacy API (recognise it in old code)

```js
// OLD jmesnil stomp.js (2014), don't use in new code
var socket = new SockJS('/ws');
var stompClient = Stomp.over(socket);
stompClient.connect({}, function (frame) {
  stompClient.subscribe('/topic/greetings', function (msg) { ... });
});
stompClient.send('/app/hello', {}, JSON.stringify({ name: 'x' }));
```

`@stomp/stompjs` still offers a compatibility `Stomp.over()` / `CompatClient` (deprecated). The modern mapping: `connect` → `activate` + `onConnect`; `send(dest, headers, body)` → `publish({destination, headers, body})`; `disconnect` → `deactivate`. A key difference: the old `Stomp.over(socket)` can't auto-reconnect when given a socket *instance*. It needs a *factory*.

6.5 SockJS fallback

What: SockJS is a protocol + client library that emulates WebSocket using HTTP streaming/polling when real WebSocket is blocked (old proxies, restrictive corporate networks).

Transport order (tries in order):

1. `websocket`
2. Streaming: `xhr-streaming`, `xdr-streaming` (old IE), `eventsource`, `iframe-eventsource`, `htmlfile`, `iframe-htmlfile`
3. Polling: `xhr-polling`, `xdr-polling`, `iframe-xhr-polling`, `jsonp-polling`

How it works:

1. `GET /ws/info` → `{ "websocket": true, "cookie_needed": true, "origins": ["*:*"], "entropy": 1234 }`
2. Picks a transport and uses URLs like `/ws/{server-id}/{session-id}/{transport}` (e.g., `/ws/123/abcxyz/websocket`, `/ws/123/abcxyz/xhr_streaming`).
3. SockJS frames: `o` = open, `h` = heart-beat, `a["msg1","msg2"]` = array of messages, `c[3000,"Go away!"]` = close.

Server: `registry.addEndpoint("/ws").withSockJS();` Notable server options: `setHeartbeatTime` (default **25 s**), `setDisconnectDelay` (default **5 s**), `setStreamBytesLimit` (default 128 KB), `setHttpMessageCacheSize` (default 100), `setSessionCookieNeeded` (default true, sends JSESSIONID for **sticky sessions**), `setClientLibraryUrl` (for iframe transports).

Client:

```ts
import SockJS from 'sockjs-client';
const client = new Client({
  webSocketFactory: () => new SockJS('https://api.example.com/ws'),   // http(s)://, NOT ws(s)://
});
```

Gotchas:

- With SockJS, the native WebSocket URL is `/ws/websocket` (Spring also accepts raw WebSocket there); plain `/ws` expects the SockJS protocol. Using `new WebSocket('wss://host/ws')` against a `.withSockJS()` endpoint fails (wrong path). Either use SockJS on the client, or register **two endpoints**: one with and one without SockJS.
- HTTP fallbacks need **sticky sessions** in clusters (all requests of a SockJS session must hit the same server).
- SockJS can't use custom headers either. And for HTTP transports, cookies are sent normally.
- In 2025+ most teams **skip SockJS** (WebSocket works nearly everywhere over `wss://` on 443). Keep it only if you have real users behind blocking proxies (check your metrics: Spring stats show `HttpStream` / `HttpPoll` session counts).

6.6 React (web)

6.6.1 A robust hook with a singleton client

```tsx
// stomp.ts: ONE client for the whole app
import { Client, IMessage } from '@stomp/stompjs';

type Listener = (msg: IMessage) => void;
const listeners = new Map<string, Set<Listener>>();   // destination → callbacks
```

```
const subs = new Map<string, { unsubscribe(): void }>();

export const stomp = new Client({
  brokerURL: `${location.protocol === 'https:' ? 'wss' : 'ws'}://${location.host}/ws`,
  reconnectDelay: 3000,
  heartbeatIncoming: 10000,
  heartbeatOutgoing: 10000,
  beforeConnect: async () => {
    stomp.connectHeaders = { Authorization: `Bearer ${await getToken()}` };
  },
  onConnect: () => {
    subs.clear();
    for (const dest of listeners.keys()) subscribeDest(dest);   // re-subscribe after reconnect
  },
});

function subscribeDest(dest: string) {
  subs.set(dest, stomp.subscribe(dest, (m) => listeners.get(dest)?.forEach((l) => l(m))));
}

export function addListener(dest: string, l: Listener): () => void {
  if (!listeners.has(dest)) {
    listeners.set(dest, new Set());
    if (stomp.connected) subscribeDest(dest);
  }
  listeners.get(dest)!.add(l);
  return () => {                                    // cleanup
    const set = listeners.get(dest)!;
    set.delete(l);
    if (set.size === 0) {
      listeners.delete(dest);
      if (stomp.connected) subs.get(dest)?.unsubscribe();
      subs.delete(dest);
    }
  };
}
```

TSX

```
// useStompTopic.ts
import { useEffect, useRef } from 'react';
import { addListener } from './stomp';

export function useStompTopic<T>(destination: string | null, onMessage: (data: T) => void) {
  const cb = useRef(onMessage);
  cb.current = onMessage;                           // always call latest callback, no re-subscribe
  useEffect(() => {
    if (!destination) return;
    return addListener(destination, (m) => cb.current(JSON.parse(m.body)));
  }, [destination]);
}

// Usage
function ChatRoom({ roomId }: { roomId: string }) {
  const [messages, setMessages] = useState<ChatMsg[]>([]);
  useStompTopic<ChatMsg>(`/topic/rooms.${roomId}`, (msg) => setMessages((prev) => [...prev, msg]));
  ...
}
```

Why this design:

- **One connection per app**, not per component (a common bug: each component creates its own `Client` → dozens of sockets).
- **Reference counting** per destination → no duplicate SUBSCRIBEs, no duplicate messages.

- **Cleanup** on unmount → no leaks, no "setState on unmounted component".
- `useRef` **for callback** → avoids re-subscribing on every render.
- **React 18 StrictMode** mounts/unmounts effects twice in dev. The ref-counted design tolerates it. A naive `useEffect(() => { client.activate(); return () => client.deactivate(); }, [])` will connect/disconnect/connect in dev, so don't panic when you see it.
- Use `setMessages(prev => ...)` (functional update) to avoid **stale closure** bugs.

Activate on login, deactivate on logout:

```ts
onLogin(() => stomp.activate());
onLogout(() => stomp.deactivate());
```

6.7 React Native (this project's platform!)

React Native's `WebSocket` is a native implementation (OkHttp on Android, SocketRocket/`NSURLSession`-based on iOS), not a browser one. Known STOMP issues:

6.7.1 The NULL byte problem

React Native has historically **truncated text WebSocket messages at the NULL (`\0`) character**, because strings cross the native bridge as C strings in some code paths. Since every STOMP frame ends with NULL, frames get corrupted: incoming frames lose their terminator, and outgoing frames may be cut.

Fix (documented by stompjs):

```ts
const client = new Client({
  brokerURL: 'wss://api.example.com/ws',
  forceBinaryWSFrames: true,          // send STOMP frames as binary WebSocket messages
  appendMissingNULLonIncoming: true,  // re-add the NULL terminator that RN chopped off incoming frames
  // ...
});
```

6.7.2 `TextEncoder` / `TextDecoder`

stompjs needs `TextEncoder` / `TextDecoder`. Older RN/JS engines lacked them. Recent React Native versions using **Hermes** provide `TextEncoder`. If you get `ReferenceError: Can't find variable: TextEncoder`, add a polyfill at the top of `index.js`:

```js
import 'text-encoding';           // or 'fast-text-encoding'
```

6.7.3 Android cleartext and emulator hosts

- `ws://` (cleartext) is **blocked by default** on Android 9+ (and iOS ATS blocks non-TLS). Use `wss://`, or allow cleartext for dev hosts only (`network_security_config.xml` / `NSAppTransportSecurity` exceptions).

- From the Android emulator, your laptop's `localhost` is `10.0.2.2`. From a physical device, use your machine's LAN IP. `ws://localhost:8080/ws` from the emulator connects to the emulator itself.

6.7.4 App lifecycle

```ts
import { AppState, AppStateStatus } from 'react-native';
import NetInfo from '@react-native-community/netinfo';

AppState.addEventListener('change', (state: AppStateStatus) => {
  if (state === 'active') { if (!client.active) client.activate(); }
  else if (state === 'background') { /* optionally deactivate after a grace period to save battery */ }
});

NetInfo.addEventListener((s) => {
  if (s.isConnected && !client.connected && client.active) {
    // stompjs will retry after reconnectDelay; you can force a faster retry by deactivate()+activate()
  }
});
```

- iOS suspends apps in background within seconds. The socket dies. Use **push notifications (APNs/FCM)** for background alerts and **resync over REST** on foreground.
- Heart-beats + `connectionTimeout` are essential on mobile (network switches Wi-Fi ↔ cellular).

6.7.5 Complete RN example for the to-do app

```ts
// src/realtime/todoSocket.ts
import { Client } from '@stomp/stompjs';
import { getAccessToken } from '../auth';

export function createTodoSocket(onEvent: (e: TodoEvent) => void, listIds: () => string[]) {
  const client = new Client({
    brokerURL: 'wss://api.example.com/ws',
    forceBinaryWSFrames: true,
    appendMissingNULLonIncoming: true,
    heartbeatIncoming: 10000,
    heartbeatOutgoing: 10000,
    reconnectDelay: 3000,
    connectionTimeout: 8000,
    beforeConnect: async () => {
      client.connectHeaders = { Authorization: `Bearer ${await getAccessToken()}` };
    },
    onConnect: () => {
      for (const id of listIds()) {
        client.subscribe(`/topic/lists.${id}`, (m) => onEvent(JSON.parse(m.body)));
      }
      client.subscribe('/user/queue/errors', (m) => console.warn('server error', m.body));
      // After (re)connect: re-fetch lists over REST to fill any gap while offline
    },
    onStompError: (f) => console.warn('STOMP error', f.headers.message),
  });
  return client;
}
```

6.8 Angular with rx-stomp

```ts
// rx-stomp.service.ts
import { Injectable } from '@angular/core';
import { RxStomp } from '@stomp/rx-stomp';

@Injectable({ providedIn: 'root' })
export class RxStompService extends RxStomp {}

// app.config.ts (standalone) or a factory
export function rxStompServiceFactory(auth: AuthService) {
  const rx = new RxStompService();
  rx.configure({
    brokerURL: 'wss://api.example.com/ws',
    heartbeatIncoming: 10000,
    heartbeatOutgoing: 10000,
    reconnectDelay: 3000,
    beforeConnect: async (client) => { client.connectHeaders = { Authorization: `Bearer ${await
auth.token()}` }; },
  });
  rx.activate();
  return rx;
}

// component
this.sub = this.rxStomp.watch('/topic/chat.public')
  .pipe(map((m) => JSON.parse(m.body) as ChatMsg))
  .subscribe((msg) => this.messages.push(msg));
// ngOnDestroy: this.sub.unsubscribe()   → sends UNSUBSCRIBE automatically

this.rxStomp.publish({ destination: '/app/chat.send', body: JSON.stringify({ text }) });
this.rxStomp.connectionState$.subscribe((state) => this.status = RxStompState[state]);
```

`watch()` re-subscribes automatically after reconnects. `publish()` while disconnected queues by default in rx-stomp (check the version's docs for behaviour) whereas stompjs throws.

6.9 Vue / Svelte / vanilla

Same principles: one client in a module/store (Pinia store, Svelte store), ref-counted subscriptions, unsubscribe in `onUnmounted` / `onDestroy`.

6.10 Java client (`WebSocketStompClient`)

Used for **service-to-service**, **integration tests**, **load tests**, and **desktop/Android (JVM) clients**.

```java
WebSocketClient transport = new StandardWebSocketClient();       // JSR-356 client (e.g., Tomcat's)
WebSocketStompClient stompClient = new WebSocketStompClient(transport);
stompClient.setMessageConverter(new MappingJackson2MessageConverter());   // Spring 6.x (Jackson 2)
// Spring 7.x (Jackson 3): stompClient.setMessageConverter(new JacksonJsonMessageConverter());
ThreadPoolTaskScheduler scheduler = new ThreadPoolTaskScheduler();
scheduler.initialize();
stompClient.setTaskScheduler(scheduler);                          // needed for heart-beats (& receipts
timeouts)
stompClient.setDefaultHeartbeat(new long[]{10_000, 10_000});
```

```
StompHeaders connectHeaders = new StompHeaders();
connectHeaders.add("Authorization", "Bearer " + token);

StompSession session = stompClient
        .connectAsync("wss://api.example.com/ws", new WebSocketHttpHeaders(), connectHeaders, new
MySessionHandler())
        .get(10, TimeUnit.SECONDS);                              // connectAsync since Spring 6.0
(connect() deprecated)

session.subscribe("/topic/chat.public", new StompFrameHandler() {
    @Override public Type getPayloadType(StompHeaders headers) { return ChatMessage.class; }
    @Override public void handleFrame(StompHeaders headers, Object payload) {
        ChatMessage msg = (ChatMessage) payload;
        System.out.println(msg);
    }
});

session.send("/app/chat.send", new ChatMessage("hello from Java"));

// Receipts
session.setAutoReceipt(true);
StompSession.Receiptable r = session.send("/app/chat.send", new ChatMessage("with receipt"));
r.addReceiptTask(() -> System.out.println("received by server"));
r.addReceiptLostTask(() -> System.out.println("no receipt in time"));   // receipt time limit defaults
to 15 s (setReceiptTimeLimit); needs a TaskScheduler

session.disconnect();
```

JAVA

```
class MySessionHandler extends StompSessionHandlerAdapter {
    @Override public void afterConnected(StompSession session, StompHeaders connectedHeaders) { /*
subscribe here */ }
    @Override public void handleException(StompSession s, StompCommand cmd, StompHeaders h, byte[]
payload, Throwable ex) { ex.printStackTrace(); }
    @Override public void handleTransportError(StompSession s, Throwable ex) { /* reconnect with backoff
*/ }
}
```

Notes:

- `WebSocketStompClient` **does not auto-reconnect.** Implement it in `handleTransportError` with backoff.
- SockJS from Java: `new SockJsClient(List.of(new WebSocketTransport(new StandardWebSocketClient()), new RestTemplateXhrTransport()))`.
- For high connection counts in load tests, tune the JSR-356 client's buffers and use a dedicated thread pool.
- Spring Framework 7 / Boot 4 move to Jackson 3: `MappingJackson2MessageConverter` is deprecated for removal in 7.0. Use `JacksonJsonMessageConverter`.
- `WebSocketStompClient`'s constructor sets heart-beats to `0,0`; calling `setTaskScheduler(...)` switches them to `10000,10000` (from the Spring source). It only sends a heart-beat when nothing else was sent.
- Spring's docs advise that when you simulate **many** clients from one machine (load tests), you consider turning heart-beats off, because each connection schedules its own heart-beat tasks.

6.11 Other platforms

Platform	Library	Notes
Python	`stomp.py`	Mature STOMP client for **TCP** brokers (RabbitMQ/ActiveMQ on 61613). For STOMP-over-WebSocket servers like Spring, check `stomp.py`'s WebSocket support in your version or write frames over the `websockets` library (frame format is trivial, see Chapter 4)
Go	`go-stomp/stomp`	TCP; combine with a WebSocket `net.Conn` adapter for WS
.NET	`Apache.NMS.STOMP`, community STOMP-over-WS libs	Many .NET shops use SignalR instead
Kotlin / Android	**Krossbow**	Coroutine-based, multiplatform STOMP client over WebSocket (OkHttp/Ktor)
Swift / iOS	`SwiftStomp`, `StompClientLib`	Check maintenance status; or URLSessionWebSocketTask + small frame codec
Ruby	`stomp` gem	TCP
CLI	`websocat`, `wscat`, Postman	Manual testing (remember the `\0` terminator!)

6.12 Manual testing with raw tools

BASH

```
# wscat (Node)
npx wscat -c ws://localhost:8080/ws -s v12.stomp
> CONNECT
accept-version:1.2
host:localhost

^@
```

In terminals you often can't type NULL easily. Use tools that allow escape sequences (Postman: `\u0000`; websocat with `--binary` and printf).

BASH

```
printf 'CONNECT\naccept-version:1.2\nhost:localhost\n\n\0' | websocat -n --binary -1
ws://localhost:8080/ws
```

6.13 Client-side best practices checklist

- ☐ Exactly **one** STOMP connection per app (per tab) unless you have a strong reason.
- ☐ Subscriptions created in `onConnect` (or via rx-stomp `watch`) so they survive reconnects.
- ☐ **Unsubscribe** when screens/components are destroyed.
- ☐ Heart-beats on (10–25 s), lower than every proxy's idle timeout.
- ☐ Reconnect with backoff (+ jitter if your library supports it, or manage `reconnectDelay` dynamically).

- [ ] Refresh token in `beforeConnect`; handle auth ERROR by re-login, not infinite retries.
- [ ] Never `publish` without checking `connected` (or use an outbox).
- [ ] Send `content-type: application/json` for JSON.
- [ ] Idempotency key (`clientRequestId`) on commands that might be retried.
- [ ] After reconnect, **resync** missed data via REST (`since=lastEventId`).
- [ ] Show connection state in UI ("Reconnecting…") and disable actions that need the socket.
- [ ] React Native: `forceBinaryWSFrames` + `appendMissingNULLonIncoming`, `wss://`, emulator host `10.0.2.2`.
- [ ] Turn off verbose `debug` logging in production (it can log tokens in CONNECT frames!).

Next: Chapter 7: Brokers, Clustering & Scaling

CHAPTER 7

Chapter 7: Message Brokers, Clustering & Scaling

Goal: run STOMP for thousands to millions of users. Understand simple broker vs broker relay, configure RabbitMQ/ActiveMQ, scale horizontally, configure load balancers/proxies/Kubernetes, and deploy without dropping everyone.

7.1 ELI5

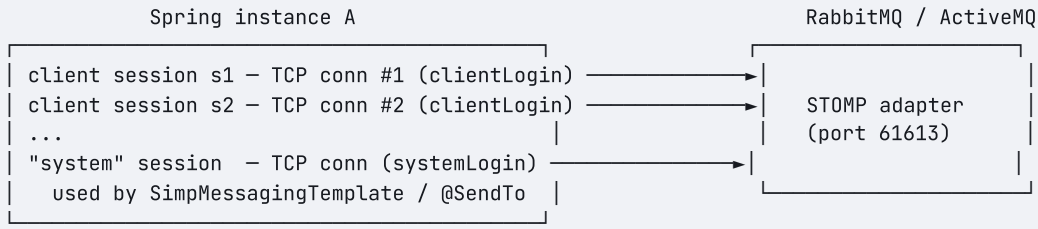
One post office (one server) can only serve so many houses. When the town grows, you open **more post offices**. But now a letter from someone near post office A to someone near post office B must travel between offices. You need a **central sorting hub** that all post offices are connected to. That hub is the **external message broker** (RabbitMQ, ActiveMQ) or a **backplane** (Redis, Kafka).

7.2 Simple broker vs broker relay

	Simple broker (`enableSimpleBroker`)	**STOMP broker relay** (`enableStompBrokerRelay`)
Where	In-memory, inside your Spring app	External broker (RabbitMQ, ActiveMQ, Artemis)
Setup	Zero	Run & operate a broker
Clustering	**No**: each instance only knows its own subscribers	**Yes**: broker connects all instances
STOMP features	Subset: no ACK/NACK, no transactions, limited receipts	Full (whatever the broker supports)
Persistence/ durability	None (lost on restart)	Durable queues, persistent messages, DLQs
Throughput	Very high for one node (no network hop)	Network hop to broker; broker becomes a shared resource
Connections	None extra	**One TCP connection per client session** + one shared "system" connection
Destination semantics	Every destination is pub/sub	Broker-defined (`/queue` competing consumers, `/topic` fan-out, `/exchange` ...)
Best for	Single instance, dev, prototypes, small apps, or with a custom backplane	Clusters, enterprise, reliability needs

Interview tip: "Why do my users on server 2 not receive messages sent on server 1?" → Simple broker isn't clustered. Use a broker relay or a pub/sub backplane.

7.3 How the broker relay works internally



- For each WebSocket client CONNECT, Spring opens a **dedicated TCP connection** to the broker and sends a STOMP CONNECT using `clientLogin` / `clientPasscode` (not the end-user's credentials!). Default `guest` / `guest`.
- Messages from server-side code (`brokerChannel`: `SimpMessagingTemplate`, `@SendTo` return values) go over a single shared **system connection** (`systemLogin` / `systemPasscode`), with heart-beats (default 10 s both ways).
- Frames from clients to broker destinations (`/topic/**`, `/queue/**`) are **forwarded** as-is; MESSAGE frames from the broker are forwarded back to the right WebSocket session.
- Client heart-beats are negotiated **with the broker** (the relay passes the client's heart-beat header through).
- If the system connection drops, Spring publishes `BrokerAvailabilityEvent(false)`, `SimpMessagingTemplate` sends fail with `MessageDeliveryException` ("Message broker not active"), and new client CONNECTs get an ERROR. Spring reconnects automatically (every 5 s by default).
- Requires **Reactor Netty** on the classpath. Spring's reference docs say to add `io.projectreactor.netty:reactor-netty` and `io.netty:netty-all`.
- Defaults from the Spring source: `clientLogin` / `clientPasscode` / `systemLogin` / `systemPasscode` = `guest`; system heart-beats `10000` ms both ways; the system connection reconnects every **5 s** (`FixedIntervalReconnectStrategy(5000)`).

Capacity implication: 50,000 WebSocket users across your app instances = **~50,000 TCP connections to the broker**. Size the broker (file descriptors, memory, Erlang process limits for RabbitMQ) accordingly. If that's too many, use the **backplane pattern** (§7.7) instead.

7.3.1 Full relay configuration

JAVA

```java
@Override
public void configureMessageBroker(MessageBrokerRegistry registry) {
    registry.setApplicationDestinationPrefixes("/app");
    registry.setPathMatcher(new AntPathMatcher("."));          // RabbitMQ topic routing keys use dots
    registry.enableStompBrokerRelay("/topic", "/queue", "/exchange", "/amq/queue", "/temp-queue")
            .setRelayHost("rabbitmq.internal")
            .setRelayPort(61613)
            .setVirtualHost("/")                               // "host" header in STOMP CONNECT
            .setClientLogin("ws-client")
            .setClientPasscode(env.getRequiredProperty("broker.client.password"))
            .setSystemLogin("ws-system")
            .setSystemPasscode(env.getRequiredProperty("broker.system.password"))
```

```
            .setSystemHeartbeatSendInterval(10_000)
            .setSystemHeartbeatReceiveInterval(10_000)
            .setAutoStartup(true)
            .setUserDestinationBroadcast("/topic/unresolved-user-destination")
            .setUserRegistryBroadcast("/topic/simp-user-registry");
}
```

7.3.2 Multiple broker nodes / TLS / failover

The relay connects to one host:port. For failover, put the broker cluster behind a TCP load balancer, or supply a custom `TcpClient` that rotates through addresses:

JAVA

```
@Override
public void configureMessageBroker(MessageBrokerRegistry registry) {
    registry.enableStompBrokerRelay("/topic", "/queue").setTcpClient(createTcpClient());
    registry.setApplicationDestinationPrefixes("/app");
}

private ReactorNettyTcpClient<byte[]> createTcpClient() {
    Iterator<InetSocketAddress> addresses = Iterables.cycle(List.of(
            new InetSocketAddress("rabbit-1", 61614), new InetSocketAddress("rabbit-2",
61614))).iterator(); // Guava cycle, or your own round-robin
    return new ReactorNettyTcpClient<>(
            client -> client.remoteAddress(addresses::next).secure(),   // .secure() enables TLS (STOMP
over TLS is often port 61614)
            new StompReactorNettyCodec());
}
```

(Spring's reference documentation shows this approach; adapt the address-rotation code to your utilities.)

7.4 User destinations in a cluster

Problem: user anna is connected to instance B. Instance A calls `convertAndSendToUser("anna", ...)`. A's local `SimpUserRegistry` doesn't know anna's sessions.

Spring's solution (broker relay only):

- `setUserDestinationBroadcast("/topic/unresolved-user-destination")`: if a user destination can't be resolved locally, it is broadcast to this topic; every instance listens and the one that has anna's sessions delivers it.
- `setUserRegistryBroadcast("/topic/simp-user-registry")`: each instance periodically broadcasts its local user registry, so every instance has a `MultiServerUserRegistry` view of **all** users (useful for "who's online" counts across the cluster).

7.5 RabbitMQ in depth

7.5.1 Enabling STOMP

```bash
rabbitmq-plugins enable rabbitmq_stomp        # STOMP over TCP, port 61613 (TLS commonly 61614)
rabbitmq-plugins enable rabbitmq_web_stomp    # STOMP over WebSocket, port 15674, path /ws
```

`docker run -d -p 5672:5672 -p 15672:15672 -p 61613:61613 rabbitmq:3-management` then enable the plugin inside (or build an image with `RUN rabbitmq-plugins enable --offline rabbitmq_stomp`).

> The `guest` user can only connect from **localhost** by default. Create dedicated users for the relay.

7.5.2 RabbitMQ STOMP destinations

Destination	SUBSCRIBE behaviour	SEND behaviour	Use
`/exchange/<exchange>[/<routing-pattern>]`	Creates an exclusive, auto-delete queue bound to the exchange with the pattern	Publishes to the exchange with routing key	Full control over AMQP routing
`/queue/<name>`	Subscribes to queue `<name>` (created if needed); several subscribers **compete**	Sends to that queue via default exchange	Work queues, point-to-point
`/amq/queue/<name>`	Subscribes to an **existing** queue (not created)	Sends to existing queue	Queues managed elsewhere (AMQP apps)
`/topic/<routing-key>`	Creates a queue bound to `amq.topic` with the routing key/pattern (auto-delete by default; durable with `durable:true` + `auto-delete:false` and a stable `id` / `x-queue-name`)	Publishes to `amq.topic`	Pub/sub with wildcards
`/temp-queue/<name>`	(Used in `reply-to`)	Creates a temporary private reply queue	Request/reply

Topic wildcards (AMQP style, dot-separated):

- `*` matches exactly one word: `/topic/prices.*` matches `prices.AAPL`, not `prices.us.AAPL`.
- `#` matches zero or more words: `/topic/prices.#` matches `prices`, `prices.AAPL`, `prices.us.AAPL`.

Useful headers:

Header	Frame	Effect
`prefetch-count`	SUBSCRIBE	Max unacked messages delivered (with `ack:client` / `client-individual`)
`durable` / `auto-delete` / `exclusive`	SUBSCRIBE	Queue properties
`x-queue-name`	SUBSCRIBE	Explicit name for the generated queue

Header	Frame	Effect
`x-message-ttl`, `x-expires`, `x-max-length`, `x-dead-letter-exchange`, `x-queue-type`	SUBSCRIBE	Queue arguments (TTL, expiry, length limit, DLX, quorum/stream queues)
`persistent:true`	SEND	Message is persisted (if the queue is durable)
`reply-to`	SEND	Reply destination, e.g. `/temp-queue/r`
`x-stream-offset`	SUBSCRIBE	Where to start reading a stream queue

7.5.3 User destinations with RabbitMQ (avoid queue leaks)

Spring's `/user/queue/x` becomes `/queue/x-user{sessionId}` → RabbitMQ creates a named queue per session which, per RabbitMQ's docs, is **durable, non-exclusive and not auto-deleted by default** for `/queue/` destinations. So it can outlive the session and leak. Spring's docs recommend using destinations that RabbitMQ makes **auto-delete**, e.g. have clients subscribe to `/user/exchange/amq.direct/position-updates` and send with `convertAndSendToUser(user, "/exchange/amq.direct/position-updates", payload)`. Then the per-session queue is auto-deleted when the session ends. Alternatively set `x-expires` /policies on such queues. **Monitor queue counts** in production: a steadily growing count = leak.

7.5.4 RabbitMQ operational tips

- Use **quorum queues** for durable work queues; classic queues for transient per-session queues.
- Set **policies** (`rabbitmqctl set_policy`) for TTL/expiry/max-length instead of per-subscribe headers where possible.
- Watch: connections count, channels, memory alarms (publishers get **blocked**), file descriptors, unacked messages.
- For very high fan-out to browsers, consider **RabbitMQ Web STOMP** directly (browsers connect to RabbitMQ), but then you lose Spring's `@MessageMapping`, auth integration and user destinations; and you expose the broker to the internet. Most enterprises keep Spring in front.

7.6 ActiveMQ Classic and ActiveMQ Artemis

ActiveMQ Classic (`activemq.xml`):

```xml
<transportConnectors>
  <transportConnector name="stomp" uri="stomp://0.0.0.0:61613?transport.maxFrameSize=1048576"/>
  <transportConnector name="ws" uri="ws://0.0.0.0:61614"/>
</transportConnectors>
```

`/queue/x` → JMS queue `x`; `/topic/x` → JMS topic `x`. Supports JMS-style `selector` header on SUBSCRIBE, durable topic subscriptions (`activemq.subscriptionName` + client-id), `persistent:true` on SEND.

ActiveMQ Artemis: acceptors in `broker.xml`, e.g.

```xml
<acceptor name="stomp">tcp://0.0.0.0:61613?
protocols=STOMP;anycastPrefix=/queue/;multicastPrefix=/topic/</acceptor>
```

`anycastPrefix` / `multicastPrefix` map `/queue/` to anycast (point-to-point) and `/topic/` to multicast (pub/sub). STOMP over WebSocket is supported on acceptors (the default all-protocols acceptor on 61616 accepts it too).

7.7 The backplane pattern (simple broker + Redis/Kafka)

When you want horizontal scale **without** one broker TCP connection per user:



1. Any instance that wants to broadcast publishes to Redis (or Kafka), **not** directly to its local broker.
2. Every instance subscribes to Redis and, on receive, calls `SimpMessagingTemplate.convertAndSend(...)` locally.
3. Each local simple broker delivers to its own connected clients.

```java
public record ClusterEnvelope(String destination, String user, String payloadJson, Map<String, String>
headers) {}

@Service
@RequiredArgsConstructor
public class ClusterMessaging {
    private final StringRedisTemplate redis;
    private final ObjectMapper json;

    public void broadcast(String destination, Object payload) throws JsonProcessingException {
        redis.convertAndSend("ws-broadcast", json.writeValueAsString(
            new ClusterEnvelope(destination, null, json.writeValueAsString(payload), Map.of())));
    }
    public void toUser(String user, String destination, Object payload) throws JsonProcessingException {
        redis.convertAndSend("ws-broadcast", json.writeValueAsString(
            new ClusterEnvelope(destination, user, json.writeValueAsString(payload), Map.of())));
    }
}

@Configuration
public class RedisBackplaneConfig {
    @Bean
    RedisMessageListenerContainer container(RedisConnectionFactory cf, SimpMessagingTemplate messaging,
ObjectMapper json) {
        var c = new RedisMessageListenerContainer();
        c.setConnectionFactory(cf);
        c.addMessageListener((message, pattern) -> {
            try {
                ClusterEnvelope env = json.readValue(message.getBody(), ClusterEnvelope.class);
                Object payload = json.readTree(env.payloadJson());
                if (env.user() == null) messaging.convertAndSend(env.destination(), payload);
                else messaging.convertAndSendToUser(env.user(), env.destination(), payload);  // no-op
if user not on this node
            } catch (IOException e) { /* log + metric */ }
        }, new ChannelTopic("ws-broadcast"));
```

```
            return c;
        }
    }
```

Trade-offs:

- **Redis pub/sub** is fire-and-forget (at-most-once): if an instance is disconnected from Redis briefly, it misses messages. **Redis Streams** or **Kafka** give replay at the cost of complexity.
- Every instance receives every message (fine up to a point). For huge scale, **shard** channels (e.g., per room/tenant) and only subscribe to shards with local subscribers.
- Simple and cheap; very common in practice.

7.8 Capacity planning

7.8.1 How many connections can one server hold?

It depends on memory per connection and message rate, not a magic number. Rough components per connection in a Spring/Tomcat stack: socket buffers (kernel), Tomcat WebSocket session and buffers (the 8 KB default text/binary buffers are allocated per session), Spring session objects, `ConcurrentWebSocketSessionDecorator` buffers when clients are slow, the subscription registry entries, plus your own per-session state.

Measure it: load test with N idle connections, then N connections × your real message rate, and watch heap, GC, CPU, and p99 latency. Many teams see **tens of thousands of mostly idle STOMP connections per instance** on a few GB of heap; heavy traffic lowers that.

7.8.2 OS and network limits

- **File descriptors:** each socket = 1 fd. Raise `ulimit -n` (e.g., `LimitNOFILE=1000000` in systemd, container `ulimits`).
- **Ephemeral ports** between a proxy and backend: one (src IP, dst IP, dst port) tuple supports ~28k–64k concurrent connections (`net.ipv4.ip_local_port_range`). For huge numbers, add more backend IPs/ports or more proxy IPs.
- **Kernel tuning:** `net.core.somaxconn`, `tcp_max_syn_backlog` for reconnect storms; TCP keepalive settings.
- **Memory for TLS:** TLS sessions add memory. Terminate TLS at the load balancer to offload, or account for it.

7.8.3 Fan-out math (the hidden cost)

Messages/second **delivered** = messages/sec published × subscribers per message. Example: 100 rooms × 1 msg/s × 1,000 users per room = **100,000 deliveries/s.** The outbound channel and network, not your controllers, become the bottleneck. Mitigations: batching/coalescing (send the latest state every 100–250 ms instead of every tick), deltas instead of full objects, binary formats, smaller audiences (shard topics), and dropping intermediate updates for slow clients.

7.9 Load balancers and reverse proxies

7.9.1 Requirements

1. Forward `Upgrade` and `Connection` headers (HTTP/1.1 to backend).
2. Long **idle/read timeouts** (or rely on heart-beats shorter than the timeout).
3. **Sticky sessions: not needed for pure WebSocket** (one TCP connection stays on one backend), but **required for SockJS HTTP fallbacks** and helpful if you keep server-side state across reconnects.
4. Balance by **least connections** rather than round-robin (connections are long-lived).
5. **Connection draining** on deploys.

7.9.2 Nginx

NGINX

```
map $http_upgrade $connection_upgrade {
    default upgrade;
    ''      close;
}

upstream realtime_backend {
    least_conn;
    server app1:8080 max_fails=3 fail_timeout=10s;
    server app2:8080 max_fails=3 fail_timeout=10s;
    # ip_hash;   # or sticky (nginx plus / cookie) if you use SockJS fallbacks
}

server {
    listen 443 ssl;
    server_name api.example.com;
    # ssl_certificate ...;

    location /ws {
        proxy_pass http://realtime_backend;
        proxy_http_version 1.1;                          # REQUIRED (Upgrade is HTTP/1.1)
        proxy_set_header Upgrade $http_upgrade;          # REQUIRED (hop-by-hop headers aren't forwarded
by default)
        proxy_set_header Connection $connection_upgrade; # REQUIRED
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 3600s;                        # default 60s; keep > heart-beat interval at
least
        proxy_send_timeout 3600s;
        proxy_buffering off;
    }
}
```

Spring behind a proxy: set `server.forward-headers-strategy=framework` (or `native`) so Spring sees the original scheme/host. This matters for **origin checks** and SockJS URLs.

7.9.3 HAProxy

```
defaults
    timeout connect 5s
    timeout client  60s
    timeout server  60s
```

```
    timeout tunnel  1h          # applies once the connection is upgraded to WebSocket
backend realtime
    balance leastconn
    server app1 10.0.0.1:8080 check
    server app2 10.0.0.2:8080 check
```

7.9.4 Cloud load balancers (verify current limits in provider docs)

- **AWS ALB:** native WebSocket support; idle timeout default **60 s** (raise it, and heart-beat below it). Stickiness is optional (needed for SockJS).
- **AWS NLB:** TCP passthrough; TCP idle timeout default 350 s.
- **AWS API Gateway WebSocket APIs:** managed connections with Lambda integrations. It doesn't speak STOMP natively; it has limits such as 2-hour max connection duration, 10-minute idle timeout, 128 KB messages.
- **GCP HTTP(S) LB:** WebSockets supported; the backend service **timeout** acts as the max connection lifetime for WebSockets (default 30 s, so raise it!).
- **Azure Application Gateway / Front Door:** WebSocket supported; check idle timeouts.
- **Cloudflare:** proxies WebSockets; idle connections without traffic may be closed after ~100 s; also connections can be closed during Cloudflare deploys → reconnect logic.

7.9.5 Kubernetes

- **Service** type ClusterIP/LoadBalancer works (TCP).
- **Ingress (NGINX-based):** WebSocket works; raise timeouts:

  YAML
  ```yaml
  metadata:
    annotations:
      nginx.ingress.kubernetes.io/proxy-read-timeout: "3600"
      nginx.ingress.kubernetes.io/proxy-send-timeout: "3600"
      # for SockJS fallbacks:
      nginx.ingress.kubernetes.io/affinity: "cookie"
  ```

  (**The community `ingress-nginx` controller was retired in March 2026**: the Kubernetes project announced it in November 2025, and after March 2026 there are no further releases or security patches. Some vendors offer extended support. New setups should use a **Gateway API** implementation (e.g., Envoy Gateway, Istio, NGINX Gateway Fabric, Traefik, or your cloud's gateway). The same rules apply on any of them: WebSocket upgrade enabled, long idle/route timeouts, and affinity only for SockJS.)
- **Istio/Envoy:** WebSocket upgrade supported; set route `timeout: 0s` (disabled) or long; idle timeouts via `DestinationRule` / `EnvoyFilter` (`idle_timeout`).
- **HPA:** CPU-based autoscaling is a poor fit (idle connections use memory, not CPU). Scale on **connection count** or **memory** with custom metrics.
- **Imbalance after scale-out:** new pods get no existing connections. Options: accept it; enforce a **max connection age** (server closes connections older than X hours with jitter, clients reconnect and spread); use least-connections balancing.

7.10 Deployments without pain

When a pod/instance stops, all its WebSocket clients disconnect. To make that harmless:

1. **Clients reconnect** automatically with **jitter** (spread the herd).
2. **Readiness first:** on shutdown, mark the instance **not ready** (remove from LB) before closing connections. In Kubernetes, use a `preStop` hook (e.g., `sleep 10`) so endpoints update.
3. **Close gracefully:** Spring closes sessions when the context stops (`SubProtocolWebSocketHandler` closes sessions with status **1001 Going Away**). Clients should treat 1001/1012 as "reconnect soon".
4. `terminationGracePeriodSeconds` long enough for in-flight work.
5. **Roll slowly** (maxUnavailable small) so the remaining pods can absorb reconnects; scale up before deploying if needed.
6. **Resync after reconnect** (REST fetch "since last event id") so nobody misses updates during the switch.
7. **Version compatibility:** during rolling deploys old and new servers run side by side. Keep message formats **backward compatible** (add fields, don't rename/remove; version your payloads).

Spring Boot: `server.shutdown=graceful` waits for active **HTTP requests**; WebSocket sessions are closed when the WebSocket infrastructure stops. Test your shutdown sequence explicitly.

7.11 Managed real-time services (when not to build it yourself)

Service	STOMP?	Notes
Amazon MQ (ActiveMQ/RabbitMQ)	Yes (broker)	Managed broker for the Spring relay
AWS API Gateway WebSocket	No	Custom JSON; Lambda-based; connection IDs
Azure Web PubSub / SignalR Service	No STOMP	Own protocols (and MQTT for Web PubSub)
Pusher, Ably, PubNub	No	SaaS pub/sub with SDKs, presence, history
CloudAMQP	Yes (RabbitMQ)	Managed RabbitMQ with STOMP plugin

Choose managed services when real-time isn't your core competency and the cost/lock-in is acceptable.

7.12 Chapter summary

- Simple broker = in-memory, single-node, partial STOMP. Relay = external broker, cluster-ready, full STOMP, one TCP connection per client to the broker.
- For multi-instance user destinations: `userDestinationBroadcast` + `userRegistryBroadcast`.
- RabbitMQ: `/exchange`, `/queue`, `/amq/queue`, `/topic` (amq.topic, `*` / `#`, dot-separated), `/temp-queue`; watch for per-session queue leaks.
- Backplane alternative: simple broker on each node + Redis/Kafka fan-out.
- Proxies must forward Upgrade/Connection and have long timeouts. Sticky sessions only for SockJS fallbacks.
- Plan capacity on connections, memory, fan-out; deploy with draining + jittered reconnect + resync.

Next: Chapter 8: Security

CHAPTER 8

Chapter 8: Security

Goal: make a STOMP/WebSocket system that can't be hijacked, eavesdropped, abused, or used to read other users' data, and explain every attack and defence in interviews.

8.1 ELI5

Your post office needs:

1. **Locked trucks** so nobody can read letters on the road → **TLS (`wss://`).**
2. **A guard at the door** who checks ID cards → **authentication.**
3. **Rules about mailboxes:** "You can read *your* mailbox, not your neighbour's." → **authorisation** of SUBSCRIBE and SEND.
4. **A check that the visitor really came from your own website, not a trick website → Origin check** (CSWSH protection).
5. **Limits** so one naughty kid can't stuff a million letters in the slot → **rate limits, size limits, connection limits.**
6. **Not trusting what's written in letters → input validation, output encoding.**

8.2 Threat model (memorise this list)

#	Threat	Example	Defence
1	Eavesdropping / tampering	Wi-Fi attacker reads chat	`wss://` only, HSTS, modern TLS
2	**Cross-Site WebSocket Hijacking (CSWSH)**	`evil.com` opens a socket to your server; the browser attaches the victim's cookie	**Check `Origin`**; don't rely on cookies alone (token in CONNECT); SameSite cookies; Spring Security CSRF on CONNECT
3	Missing authentication	Anyone connects and subscribes	Authenticate at handshake or CONNECT; reject unauthenticated frames
4	**Broken authorisation on SUBSCRIBE**	User subscribes to `/topic/orders.123` that belongs to someone else (IDOR)	Authorise every SUBSCRIBE by destination + ownership
5	Broken authorisation on SEND	User sends directly to `/topic/admin.broadcast`, bypassing your controller	Deny client SEND to broker prefixes; only allow `/app/**`
6	Wildcard subscriptions	Subscribe `/topic/**` or `/topic/orders.#` (RabbitMQ) to see everything	Deny pattern subscriptions; exact-match rules

#	Threat	Example	Defence
7	Token leakage	Token in URL query → access logs, proxy logs	Tokens in CONNECT headers; short-lived one-time tickets if in URL; redact logs
8	Stale auth	Token expires/revoked but socket stays open for days	Enforce expiry on each message; server-side session termination; max connection age
9	Denial of service	Connection floods, huge frames, subscription floods, message floods, slow readers	Limits at LB and app: connections/IP, `messageSizeLimit`, rate limits, `sendBufferSizeLimit`, `timeToFirstMessage`
10	Injection	XSS via chat message rendered as HTML; SpEL injection (CVE-2018-1270)	Encode output; validate input; patch Spring; disable unused selector header
11	Information leakage	Stack traces in ERROR frames; user enumeration via presence topics	Generic error messages; authorise presence data
12	Message spoofing	Client sets `from: "admin"` in payload	**Never trust identity fields from the client**; take identity from `Principal`
13	Broker exposure	RabbitMQ Web STOMP open to internet with `guest`	Keep broker private; dedicated creds; TLS to broker
14	Compression side channels	Secrets + attacker data compressed together (CRIME/BREACH style)	Avoid compressing secrets with attacker-controlled data

8.3 Transport security: `wss://`

- Always use `wss://` in production (TLS). Pages on HTTPS **must** use `wss://` (mixed content).
- TLS also improves **connectivity** (proxies can't inspect/break the upgrade inside a TLS tunnel).
- Terminate TLS at the LB or at the app; if terminated at the LB, the LB→app hop should be inside a trusted network or re-encrypted (zero-trust environments).
- Use TLS for the **relay → broker** connection too (`.secure()` on the Reactor Netty client; RabbitMQ STOMP TLS port).

8.4 Cross-Site WebSocket Hijacking (CSWSH): the #1 WebSocket-specific attack

8.4.1 How it works

1. Victim is logged in to `app.example.com` (session cookie).
2. Victim visits `evil.com`.
3. `evil.com` runs `new WebSocket('wss://app.example.com/ws')`.
4. **CORS does not apply to WebSockets.** The browser happily performs the handshake and attaches the victim's cookies (depending on `SameSite`).
5. If the server authenticates by cookie alone and doesn't check `Origin`, the attacker's page now has an **authenticated socket**: it can read the victim's private messages and send messages as the victim.

8.4.2 Defences (use several)

1. **Validate** `Origin` at the handshake against an allow-list. In Spring: `setAllowedOrigins(...)` / `setAllowedOriginPatterns(...)` (default = same origin only). **Never** use `setAllowedOriginPatterns("*")` in production with cookie auth.
2. **Don't authenticate by cookie alone**: require a token in the STOMP CONNECT header (the attacker's page doesn't know it).
3. **SameSite cookies** (`Lax` / `Strict`) reduce cookie sending on cross-site requests. A WebSocket handshake from `evil.com` to your site is a cross-site request, so `SameSite=Lax/Strict` cookies are generally not sent. But be careful: "same-site" ≠ "same-origin" (a compromised `sub.example.com` is same-site).
4. **Spring Security CSRF on CONNECT**: `@EnableWebSocketSecurity` requires a CSRF token in the CONNECT frame by default.

> `Origin` can be forged by **non-browser** clients (curl, scripts). That's fine: CSWSH is about abusing a victim's *browser*, which always sends the real Origin. Non-browser clients must still authenticate with credentials they actually own.

8.5 Authentication options

Option	Where	When to use
A. Authenticate the HTTP handshake	Spring Security filter chain on `/ws` (session cookie, or token in query/cookie)	Classic web apps with server sessions; the `Principal` from the HTTP request becomes the WebSocket user automatically
B. Authenticate the STOMP CONNECT frame	`ChannelInterceptor` reads `Authorization` native header	SPAs/mobile with JWT/OAuth2 access tokens; **most common today**
C. Ticket in query string	Handshake	When a proxy/gateway must authenticate before upgrade; use short-lived, one-time tickets
D. mTLS	TLS layer	Service-to-service

Spring's docs say it plainly: for token-based auth, **authenticate on the STOMP CONNECT frame** because browsers can't send headers on the handshake.

8.5.1 Option A: handshake auth with Spring Security (cookie session)

JAVA

```java
@Bean
SecurityFilterChain http(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(a -> a
            .requestMatchers("/ws/**").authenticated()      // handshake requires a logged-in session
            .anyRequest().authenticated())
        .formLogin(Customizer.withDefaults());
    return http.build();
}
```

The authenticated `Principal` of the HTTP request is attached to the WebSocket session and to every STOMP message (`simpUser`). Combine with Origin checks + CSRF on CONNECT.

8.5.2 Option B: JWT in the STOMP CONNECT frame (recommended for SPAs/mobile)

JAVA

```
@Configuration
@Order(Ordered.HIGHEST_PRECEDENCE + 99)                // run BEFORE Spring Security's message
interceptors
@RequiredArgsConstructor
public class WebSocketAuthConfig implements WebSocketMessageBrokerConfigurer {

    private final JwtDecoder jwtDecoder;                // from spring-security-oauth2-resource-server
    private final JwtAuthenticationConverter jwtAuthConverter = new JwtAuthenticationConverter();

    @Override
    public void configureClientInboundChannel(ChannelRegistration registration) {
        registration.interceptors(new ChannelInterceptor() {
            @Override
            public Message<?> preSend(Message<?> message, MessageChannel channel) {
                StompHeaderAccessor acc = MessageHeaderAccessor.getAccessor(message,
StompHeaderAccessor.class);
                if (acc != null && StompCommand.CONNECT.equals(acc.getCommand())) {
                    String header = acc.getFirstNativeHeader("Authorization");
                    if (header == null || !header.startsWith("Bearer ")) {
                        throw new BadCredentialsException("Missing bearer token");
                    }
                    try {
                        Jwt jwt = jwtDecoder.decode(header.substring(7));          // verifies
signature, exp, nbf, iss (if configured)
                        AbstractAuthenticationToken auth = jwtAuthConverter.convert(jwt);
                        acc.setUser(auth);                                         // becomes Principal
for this session
                        acc.getSessionAttributes().put("tokenExp", jwt.getExpiresAt());
                    } catch (JwtException e) {
                        throw new BadCredentialsException("Invalid token", e);     // → STOMP ERROR,
connection closed
                    }
                }
                return message;
            }
        });
    }
}
```

Important details:

- `@Order(Ordered.HIGHEST_PRECEDENCE + 99)`: Spring's docs recommend this so your interceptor is registered **before** Spring Security's authorization interceptor; otherwise the CONNECT is rejected as unauthenticated before you set the user.
- Use `MessageHeaderAccessor.getAccessor(...)` (mutable accessor), not `StompHeaderAccessor.wrap(...)` (copy).
- **You only need to set the user on CONNECT.** Spring remembers it for the session (`StompSubProtocolHandler` stores it) and attaches it to later messages.
- The thrown exception becomes an ERROR frame (customise via `StompSubProtocolErrorHandler`, Chapter 5) and the socket closes. The client shouldn't blindly retry with the same token.
- With the **broker relay**, the broker still sees `clientLogin` credentials, not the user's. Your Spring layer is the security boundary.

Client:

```ts
new Client({ brokerURL: 'wss://api.example.com/ws',
             beforeConnect: async (c) => { c.connectHeaders = { Authorization: `Bearer ${await token()}`
}; } });
```

8.5.3 Option C: one-time ticket

1. Client calls `POST /api/ws-ticket` with its normal `Authorization` header → server stores `ticket → userId` in Redis for 30 s.
2. Client connects `wss://api.example.com/ws?ticket=abc`.
3. A `HandshakeInterceptor` / `DefaultHandshakeHandler#determineUser` consumes (deletes) the ticket and assigns the Principal. Logs only ever contain a dead, single-use ticket.

8.6 Authorisation of messages with Spring Security

8.6.1 Modern configuration (`@EnableWebSocketSecurity` , Spring Security 5.8+/6+)

```java
@Configuration
@EnableWebSocketSecurity
public class WebSocketSecurityConfig {

    @Bean
    AuthorizationManager<Message<?>> messageAuthorizationManager(
            MessageMatcherDelegatingAuthorizationManager.Builder messages) {
        messages
            .nullDestMatcher().authenticated()                                  // CONNECT, DISCONNECT,
HEARTBEAT, UNSUBSCRIBE...
            .simpSubscribeDestMatchers("/user/queue/errors").permitAll()
            .simpDestMatchers("/app/**").hasRole("USER")                         // client SENDs only
to controllers
            .simpSubscribeDestMatchers("/user/**", "/topic/lists.*").hasRole("USER")
            .simpTypeMatchers(SimpMessageType.MESSAGE, SimpMessageType.SUBSCRIBE).denyAll() //
everything else: no
            .anyMessage().denyAll();
        return messages.build();
    }
}
```

What this does:

- Messages without a destination (CONNECT, etc.) need an authenticated user.
- Clients may only **SEND** to `/app/**` , so they **cannot SEND directly to** `/topic/**` to impersonate the server.
- Subscriptions only to allowed patterns.
- **Deny by default** (`anyMessage().denyAll()`): new destinations are blocked until someone thinks about them.

Legacy configuration (`AbstractSecurityWebSocketMessageBrokerConfigurer` , `configureInbound(MessageSecurityMetadataSourceRegistry)`) was deprecated in 5.8/6.x and is **removed in Spring Security 7**. Migrate to `@EnableWebSocketSecurity` + `AuthorizationManager` . If you still have SpEL-

based rules, Spring Security offers `MessageExpressionAuthorizationManager` to ease migration.

> Note from the Spring Security reference: Spring Security secures **only the** `clientInboundChannel`, not the outbound channel (for performance: one inbound message can fan out to many outbound ones). Secure **subscriptions**, not outgoing messages.

8.6.2 CSRF and `@EnableWebSocketSecurity`

- By default, `@EnableWebSocketSecurity` requires a **CSRF token** in the CONNECT frame headers (because browser cookie-authenticated sessions are vulnerable to CSWSH). The client fetches the token (e.g., from a `/csrf` endpoint or a meta tag) and adds it to `connectHeaders` using the header name from the `CsrfToken` (commonly `X-CSRF-TOKEN` / `X-XSRF-TOKEN`). In Spring Security 6 the expected value is the masked (XOR-encoded) token form by default. Send the value your server's `CsrfToken` provides.
- If you use **only header tokens (JWT in CONNECT)** and no cookies, CSRF provides little value. The current Spring Security reference states: *"At this point, CSRF is not configurable when using `@EnableWebSocketSecurity`, though this will likely be added in a future release."* Spring Security's docs show the alternative: **don't use the annotation**, and register the components yourself:

JAVA

```
@Configuration
@Order(Ordered.HIGHEST_PRECEDENCE + 100)          // after your JWT CONNECT interceptor (+99)
@RequiredArgsConstructor
public class WebSocketSecurityNoCsrfConfig implements WebSocketMessageBrokerConfigurer {
    private final ApplicationContext context;
    private final AuthorizationManager<Message<?>> authorizationManager;   // the bean built above

    @Override
    public void addArgumentResolvers(List<HandlerMethodArgumentResolver> resolvers) {
        resolvers.add(new AuthenticationPrincipalArgumentResolver());       // enables
@AuthenticationPrincipal
    }

    @Override
    public void configureClientInboundChannel(ChannelRegistration registration) {
        AuthorizationChannelInterceptor authz = new
AuthorizationChannelInterceptor(authorizationManager);
        authz.setAuthorizationEventPublisher(new SpringAuthorizationEventPublisher(context));
        registration.interceptors(new SecurityContextChannelInterceptor(), authz);   // no CSRF
interceptor
    }
}
```

(Verify against the Spring Security version you use; this area has evolved across 5.8 → 6.x → 7.x.)

8.6.3 Fine-grained (ownership) authorisation on SUBSCRIBE

Role checks aren't enough: "is user anna a **member of list L1**?" requires data. Do it in an interceptor:

JAVA

```
@Component
@RequiredArgsConstructor
public class SubscriptionGuard implements ChannelInterceptor {
    private static final Pattern LIST_TOPIC = Pattern.compile("^/topic/lists\\.([A-Za-z0-9-]{1,64})$");
    private final ListAccessService access;
```

```
    @Override
    public Message<?> preSend(Message<?> message, MessageChannel channel) {
        StompHeaderAccessor acc = MessageHeaderAccessor.getAccessor(message, StompHeaderAccessor.class);
        if (acc == null || !StompCommand.SUBSCRIBE.equals(acc.getCommand())) return message;

        String dest = acc.getDestination();
        Principal user = acc.getUser();
        if (user == null || dest == null) throw new AccessDeniedException("Unauthenticated");

        Matcher m = LIST_TOPIC.matcher(dest);
        if (m.matches()) {
            if (!access.isMember(user.getName(), m.group(1))) {
                throw new AccessDeniedException("Not a member of list");   // → ERROR + close
            }
            return message;
        }
        if (dest.startsWith("/user/queue/") || dest.equals("/topic/announcements")) return message;
        throw new AccessDeniedException("Unknown destination");        // deny by default
    }
}
```

Register it in `configureClientInboundChannel` (after the auth interceptor).

Alternative: soft rejection. Throwing produces an ERROR and **kills the whole connection**. Sometimes you want to reject only the subscription: return `null` from `preSend` to **silently drop** the SUBSCRIBE and send an error to `/user/queue/errors` via `SimpMessagingTemplate`. Choose consciously; silent drops can confuse clients.

Note on performance: the check runs once per SUBSCRIBE (not per message), so a DB/cache lookup is fine. But **membership can change later:** when a user is removed from a list, you must also **terminate their existing subscription.** Either close their sessions, or have the server-side logic filter (e.g., send list events only via user destinations to current members). Revocation is a commonly missed requirement.

8.6.4 Authorising SENDs

Put **business authorisation in the controller/service** (`access.requireMember(...)`), just like REST. Interceptor rules handle coarse checks; services handle data-level checks. `@PreAuthorize` works on `@MessageMapping` methods when method security is enabled (`@EnableMethodSecurity`), since the `SecurityContext` is populated for message handling by Spring Security's `SecurityContextChannelInterceptor` .

8.6.5 Never trust client-provided identity

JAVA

```
// WRONG
@MessageMapping("/chat.send") @SendTo("/topic/chat")
public ChatMessage send(ChatMessage m) { return m; }          // client can set m.sender = "ceo"

// RIGHT
public ChatMessage send(ChatMessage m, Principal p) { return m.withSender(p.getName()); }
```

8.7 Token expiry and revocation while connected

A JWT is checked at CONNECT. The socket may live for **days**. Strategies:

1. **Check expiry on every inbound message** (cheap):

```java
@Override
public Message<?> preSend(Message<?> message, MessageChannel channel) {
    SimpMessageHeaderAccessor acc = MessageHeaderAccessor.getAccessor(message,
SimpMessageHeaderAccessor.class);
    if (acc != null && acc.getSessionAttributes() != null) {
        Instant exp = (Instant) acc.getSessionAttributes().get("tokenExp");
        if (exp != null && Instant.now().isAfter(exp)) throw new CredentialsExpiredException("Token
expired");
    }
    return message;
}
```

This stops expired clients from **sending**, but they still **receive** subscribed messages.

2. **Server-initiated close of expired/revoked sessions.** Keep references to `WebSocketSession`s and close them:

```java
@Component
public class SessionRegistry implements WebSocketHandlerDecoratorFactory {
    private final Map<String, WebSocketSession> sessions = new ConcurrentHashMap<>();
    @Override
    public WebSocketHandler decorate(WebSocketHandler handler) {
        return new WebSocketHandlerDecorator(handler) {
            @Override public void afterConnectionEstablished(WebSocketSession s) throws Exception {
                sessions.put(s.getId(), s); super.afterConnectionEstablished(s);
            }
            @Override public void afterConnectionClosed(WebSocketSession s, CloseStatus st) throws
Exception {
                sessions.remove(s.getId()); super.afterConnectionClosed(s, st);
            }
        };
    }
    public void close(String sessionId, CloseStatus status) throws IOException {
        WebSocketSession s = sessions.get(sessionId);
        if (s != null && s.isOpen()) s.close(status);
    }
}
// register: registration.addDecoratorFactory(sessionRegistry) in configureWebSocketTransport
```

A scheduled job closes sessions whose `tokenExp` passed, or listens to a "user revoked/logged out" event and closes all their sessions (`new CloseStatus(4001, "token expired")`). The client refreshes the token in `beforeConnect` and reconnects. (The WebSocket session id equals the STOMP `simpSessionId`.)

3. **Max connection age** (e.g., 1–12 hours) forces periodic re-authentication.
4. **Refresh-in-band** (advanced): client sends a new token via a `/app/auth.refresh` message; server validates and updates the session's expiry.

8.8 Denial-of-service protections

Attack	Control
Too many connections from one IP/user	LB/WAF connection limits; per-user session cap in a CONNECT interceptor (`simpUserRegistry.getUser(name).getSessions().size()`)

Attack	Control
Connect but never send CONNECT	`setTimeToFirstMessage` (default 60 s; lower it, e.g., 10 s)
Huge frames	`setMessageSizeLimit`; container buffer sizes; LB body limits
Message floods	Per-session rate limiting (token bucket) in an inbound interceptor; close abusive sessions
Subscription floods	Cap subscriptions per session; deny wildcard destinations
Slow readers (memory blow-up)	`setSendBufferSizeLimit`, `setSendTimeLimit` (sessions get closed)
Expensive handlers	Bounded inbound thread pool + queue; timeouts in handlers; offload heavy work
Reconnect storms	Client jitter/backoff; server-side `429`-like close code (1013 Try Again Later) with longer retry

Rate limit example (per session, simple token bucket):

JAVA

```java
public class RateLimitInterceptor implements ChannelInterceptor {
    private final Map<String, Bucket> buckets = new ConcurrentHashMap<>();   // Bucket4j, or roll your
own
    @Override
    public Message<?> preSend(Message<?> message, MessageChannel channel) {
        StompHeaderAccessor acc = MessageHeaderAccessor.getAccessor(message, StompHeaderAccessor.class);
        if (acc == null || acc.getCommand() != StompCommand.SEND) return message;
        Bucket b = buckets.computeIfAbsent(acc.getSessionId(),
                id -> Bucket.builder().addLimit(Bandwidth.simple(20, Duration.ofSeconds(1))).build());
        if (!b.tryConsume(1)) throw new MessageDeliveryException("Rate limit exceeded");
        return message;
    }
    // remove bucket on SessionDisconnectEvent to avoid memory leaks
}
```

8.9 Input validation and output encoding

- Validate payloads (`@Valid`), lengths, enums, IDs; reject unknown fields if strict.
- Treat message content like any untrusted input: **render as text** in the UI (`textContent`, React's default escaping); never `innerHTML` / `dangerouslySetInnerHTML` without sanitising (DOMPurify).
- Validate **destination variables** (`@DestinationVariable`) with regexes; they come from the client.
- Avoid deserialising polymorphic types from client JSON (Jackson default typing → RCE gadgets).
- Keep Spring updated. Known STOMP-related CVEs: **CVE-2018-1270** and **CVE-2018-1275** (SpEL injection via the simple broker `selector` header; 1275 covered an incomplete fix for 4.3.x). Upgrading Spring fixed them; disabling the selector header removes the surface entirely.

8.10 Logging and privacy

- **Never log CONNECT headers raw** (tokens, passwords). Redact `Authorization`, `passcode`, cookies.
- Turn off `debug` in stompjs in production (it can print frames).

- Don't log full message bodies at INFO (PII). Log ids, sizes, destinations.
- For audit trails (financial/healthcare), log **who sent what to which destination when**, with correlation IDs, to a secure store.
- Multi-tenant: include `tenantId` in every destination and **authorise it**; never let tenant come from the payload.

8.11 Security checklist (copy into your PR template)

- [ ] `wss://` everywhere; HSTS on the domain.
- [ ] Allowed origins = explicit allow-list (no `*`).
- [ ] Authentication at handshake or CONNECT; unauthenticated frames rejected.
- [ ] `@Order(HIGHEST_PRECEDENCE + 99)` on the CONNECT auth interceptor.
- [ ] Client SEND allowed only to `/app/**`; SUBSCRIBE only to allow-listed destinations; deny by default.
- [ ] Ownership checks for parameterised topics (`/topic/orders.{id}`); handles revocation.
- [ ] No wildcard subscriptions from clients.
- [ ] Identity from `Principal`, never payload.
- [ ] Token expiry enforced (per-message check + server-side close + max connection age).
- [ ] Size limits, rate limits, connection caps, `timeToFirstMessage`.
- [ ] Generic ERROR messages; no stack traces.
- [ ] Logs redacted; stompjs debug off.
- [ ] Broker not internet-exposed; dedicated creds; TLS to broker.
- [ ] Dependencies patched (Spring, Tomcat, Netty, stompjs).
- [ ] Pen-test WebSocket endpoints (Burp/ZAP support WebSocket message tampering).

Next: Chapter 9: Reliability, Performance & Tuning

CHAPTER 9

Chapter 9: Reliability, Performance & Tuning

Goal: make real-time messaging that doesn't lose important data, doesn't fall over under load, and degrades gracefully. This is what separates "it works on my laptop" from senior-level production work.

9.1 ELI5

- **Reliability** = letters don't get lost, don't arrive twice in a harmful way, and arrive in the right order when order matters.
- **Performance** = the post office handles lots of letters quickly without its sorting room overflowing.
- **Resilience** = when a truck breaks down, letters still get delivered later, and everybody knows what happened.

9.2 Delivery guarantees: what STOMP over WebSocket really gives you

Guarantee	Meaning	Where you get it
At-most-once	Delivered 0 or 1 times; may be lost	Default: simple broker, `ack:auto`, disconnected clients miss messages
At-least-once	Delivered ≥1 times; duplicates possible	Full broker + durable queue + `ack:client(-individual)` + redelivery; **plus** client-side dedupe
Exactly-once (effectively-once)	Effect applied once	**Not a transport property.** At-least-once + idempotent processing (dedupe by message id)

Truth bomb for interviews: With Spring's simple broker, if a client is disconnected for 3 seconds (tunnel, deploy, Wi-Fi switch), **every message published in those 3 seconds is lost for that client.** TCP's reliability only covers bytes on a *live* connection. Design for it:

9.2.1 The "WebSocket as notification, database as truth" pattern (most important pattern in this book)

1. All state lives in a database (source of truth).
2. Every change gets a **monotonic version/sequence number** or event id.
3. WebSocket pushes **events** (or just "something changed, version N").
4. Client tracks the **last version** it applied.
5. On (re)connect or **gap detected** (received N+2 after N), the client **fetches missed changes** via REST (`GET /lists/L1/events?after=N`) or a full snapshot.

```ts
let lastSeq = 0;
function onEvent(e: { seq: number; ... }) {
  if (e.seq <= lastSeq) return;                // duplicate or stale → ignore (idempotent)
  if (e.seq > lastSeq + 1) return resync();    // gap → fetch missing
  apply(e); lastSeq = e.seq;
}
client.onConnect = () => { subscribeAll(); resync(); };   // after reconnect always resync
```

9.2.2 Subscribe-then-snapshot (avoid the race)

If you **fetch the snapshot first and subscribe second**, changes between the two are lost. Correct order:

1. Subscribe to the live topic; **buffer** incoming events.
2. Fetch snapshot (with its version `V`).
3. Apply buffered events with `seq > V`, drop the rest.
4. Continue live.

9.2.3 Server-side: the transactional outbox

If your service writes to the DB and then publishes to the broker, a crash between the two loses the event (or the reverse publishes a lie). **Outbox pattern:** write the event into an `outbox` table in the **same DB transaction**, and a relay process (polling or CDC like Debezium) publishes it to the broker/WebSocket layer and marks it sent. Gives at-least-once publish; consumers dedupe by event id.

Simpler alternative for small apps: `@TransactionalEventListener(phase = AFTER_COMMIT)` (no lie on rollback, but an event can be lost if the process crashes right after commit; the REST resync covers that).

9.2.4 Client → server reliability

- Client sends a command with a `clientRequestId` (UUID).
- Server processes **idempotently** (unique constraint on request id, or a processed-ids cache with TTL).
- Server replies (to `/user/queue/acks` or via the resulting event containing `clientRequestId`).
- Client retries if no confirmation within a timeout (or after reconnect). Duplicates are harmless.
- STOMP `receipt` confirms the broker/server **received** the frame, not that your business logic succeeded.

9.2.5 Durable delivery to offline users

WebSocket is for **online** users. For offline users:

- Store notifications in DB (inbox table), show on next login.
- Send **push notifications** (FCM/APNs) or email for important ones.
- With RabbitMQ you *can* use durable per-user queues, but per-user durable queues for millions of users are an operational burden. The DB inbox is usually better.

9.3 Ordering

Layer	Ordering behaviour
TCP / one WebSocket connection	Ordered
Spring inbound channel (thread pool)	**Not ordered by default** across messages of one session → `setPreserveReceiveOrder(true)` (6.1+)
Spring outbound channel	**Not ordered by default** → `setPreservePublishOrder(true)` (5.1+)
Simple broker fan-out	Per subscriber, follows outbound ordering setting
RabbitMQ single queue, single consumer	Ordered (redeliveries can reorder)
Multiple publishers / instances	**No global order**. Use per-entity sequence numbers
Kafka backplane	Ordered per partition (partition by entity id)

Rules of thumb:

- Most apps need **per-entity** ordering (per chat room, per document), not global.
- Put a **sequence number per entity** in each event, generated where the state changes (DB row version).
- Clients drop stale versions (last-writer-wins by version).

9.4 Heart-beats and timeouts: tuning table

Setting	Recommended starting point	Why
STOMP heart-beat (both ways)	10 s – 25 s	Below every idle timeout in the path; fast dead-peer detection
Proxy/LB idle timeout	≥ 2–3 × heart-beat (e.g., 60–3600 s)	Never kill healthy connections
Client `connectionTimeout`	5–10 s	Don't hang forever on half-open connects
Client reconnect delay	1 s → backoff to 30 s, with jitter	Avoid thundering herds
Spring `timeToFirstMessage`	10–30 s	Kill sockets that never CONNECT
Spring `sendTimeLimit`	10–20 s	Kill stuck writes to dead clients
Spring `sendBufferSizeLimit`	512 KB – few MB	Bound memory for slow clients
Mobile	10–20 s heart-beats	Networks change often; NAT timeouts are short

Trade-off: shorter heart-beats → faster detection but more traffic/battery (each beat is 1 byte of payload but a packet and radio wake-up on mobile). 100k connections × 1 beat/10 s each way = 20k tiny packets/s. Fine for servers, measurable for phones.

9.5 Slow consumers and backpressure

9.5.1 What happens with a slow client in Spring

- Messages to a session go through `ConcurrentWebSocketSessionDecorator`. Only **one thread sends** to a session at a time; others **buffer** messages in a per-session queue.
- If buffered bytes exceed `sendBufferSizeLimit` or a send takes longer than `sendTimeLimit`, the session is closed (Spring's `CloseStatus.SESSION_NOT_RELIABLE`, 4500), and stats count it under "send limit".
- Without these limits, one slow client on a bad network could make the server buffer unbounded data → `OutOfMemoryError`.

9.5.2 Strategies

1. **Bound buffers** (above), so the slow client gets disconnected and resyncs on reconnect.
2. **Conflation/coalescing:** for state that changes frequently (prices, positions, cursors), send only the **latest** value per key every X ms. A slow client doesn't need 50 stale prices.
3. **Batching:** send arrays of events every 50–250 ms instead of one frame per event (fewer frames, less overhead).
4. **Deltas:** send changes, not full objects.
5. **Smaller audiences:** shard topics (per room, per symbol), subscribe only to what's visible on screen.
6. **Prioritise:** separate critical messages (orders) from high-volume ones (market data); maybe separate connections/endpoints.

Coalescing example (server side, per destination, latest-wins):

JAVA

```
@Component
@RequiredArgsConstructor
public class PriceConflator {
    private final SimpMessagingTemplate messaging;
    private final ConcurrentHashMap<String, Price> latest = new ConcurrentHashMap<>();

    public void onTick(Price p) { latest.put(p.symbol(), p); }         // called thousands of times/s

    @Scheduled(fixedRate = 200)                                        // 5 updates/s per symbol max
    public void flush() {
        for (String symbol : latest.keySet()) {
            Price p = latest.remove(symbol);
            if (p != null) messaging.convertAndSend("/topic/prices." + symbol, p);
        }
    }
}
```

9.6 Thread pools and blocking

- `clientInboundChannel` threads run your `@MessageMapping` methods. **Blocking** (slow DB, HTTP calls) in handlers occupies these threads; when all are busy, messages **queue** (default queue capacity is unbounded) → latency grows, memory grows.
- Default core size = 2 × CPUs. For I/O-bound handlers, either **increase the pool** (e.g., 50–200) or **offload** to another executor and reply asynchronously (return `CompletableFuture`).

- Set a **bounded queue** + rejection policy for back-pressure on inbound traffic only if you have a plan for rejected messages (clients must retry).
- `clientOutboundChannel` mostly writes to sockets. Thread count rarely needs to be high unless clients are slow (limits handle that).
- Java 21 **virtual threads**: since **Spring Framework 6.2**, `ChannelRegistration.executor(Executor)` lets you plug any `Executor` into a channel (it takes precedence over `taskExecutor()` settings). Example:

  ```java
  @Override
  public void configureClientInboundChannel(ChannelRegistration r) {
      r.executor(new VirtualThreadTaskExecutor("ws-in-"));   //
  org.springframework.core.task.VirtualThreadTaskExecutor
  }
  ```

  Load test before adopting: virtual threads help handlers that block on I/O, but they don't fix slow clients or fan-out costs. Keep `setPreservePublishOrder` / `setPreserveReceiveOrder` in mind, since ordering is still not guaranteed across threads.
- Watch `WebSocketMessageBrokerStats` (`queued tasks`) or custom metrics on executor queue sizes.

```java
@Override
public void configureClientInboundChannel(ChannelRegistration r) {
    r.taskExecutor().corePoolSize(32).maxPoolSize(64).keepAliveSeconds(60).queueCapacity(50_000);
}
```

Note: with `ThreadPoolTaskExecutor`, `maxPoolSize` only takes effect when the **queue is full**. With an unbounded queue, the pool never grows beyond `corePoolSize`. A very common misconfiguration.

9.7 Payload size and format

- JSON is fine for most apps. Optimise only with data:
  - Short field names for high-frequency messages; omit nulls (`spring.jackson.default-property-inclusion=non_null`).
  - Numbers instead of strings; epoch millis instead of ISO strings for hot paths.
- **Binary formats** (Protocol Buffers, MessagePack, CBOR, FlatBuffers) shrink payloads 30–80% and parse faster; send as binary WebSocket frames with STOMP `content-type: application/x-protobuf` and a custom `MessageConverter`.
- **Compression** (`permessage-deflate`): big wins for verbose JSON, costs CPU and memory per connection. Measure.
- Keep messages **small** (< a few KB). For big content, send an id/URL and fetch over HTTP (CDN-cacheable, resumable).

9.8 Observability: what to measure

9.8.1 Metrics (Micrometer → Prometheus/Grafana/Datadog)

Metric	Why
Active WebSocket sessions (per instance, per transport)	Capacity, imbalance
Connects/s, disconnects/s, by close code	Reconnect storms, auth failures, proxy timeouts (1006)
STOMP CONNECT failures (auth)	Token problems, attacks
Inbound messages/s, outbound messages/s	Throughput, fan-out
Message size histogram	Detect oversized payloads
Inbound/outbound executor active threads + queue size	Saturation
Sessions closed due to send limits	Slow clients / network problems
Handler latency (`@MessageMapping` timing)	Slow business logic
End-to-end latency (publish ts → client receive ts, sampled)	User-visible lag
Broker: connections, queue depth, unacked, memory alarms	Broker health
`BrokerAvailabilityEvent` state	Relay connectivity
Subscriptions count per destination (top N)	Hot topics

Session gauge example:

```java
@Component
public class WsMetrics {
    private final AtomicInteger sessions = new AtomicInteger();
    public WsMetrics(MeterRegistry registry, SimpUserRegistry users) {
        Gauge.builder("ws.sessions.active", sessions, AtomicInteger::get).register(registry);
        Gauge.builder("ws.users.connected", users, SimpUserRegistry::getUserCount).register(registry);
    }
    @EventListener void on(SessionConnectedEvent e) { sessions.incrementAndGet(); }
    @EventListener void on(SessionDisconnectEvent e) { /* guard against double events with a
Set<sessionId> */ sessions.decrementAndGet(); }
}
```

(Because `SessionDisconnectEvent` can fire twice, track session ids in a concurrent set and decrement only when removal succeeds.)

Executor metrics:

```java
@Bean
public MeterBinder wsExecutorMetrics(@Qualifier("clientInboundChannelExecutor") ThreadPoolTaskExecutor
in,
                                     @Qualifier("clientOutboundChannelExecutor") ThreadPoolTaskExecutor
out) {
    return registry -> {
        new ExecutorServiceMetrics(in.getThreadPoolExecutor(), "ws.inbound",
List.of()).bindTo(registry);
        new ExecutorServiceMetrics(out.getThreadPoolExecutor(), "ws.outbound",
List.of()).bindTo(registry);
```

```
    };
}
```

(Bean names `clientInboundChannelExecutor` / `clientOutboundChannelExecutor` / `brokerChannelExecutor` are Spring's defaults, and by default they are `ThreadPoolTaskExecutor`s. If you plugged in a custom executor, adapt the binding.)

9.8.2 Logging

- Log connect/disconnect with session id, user id, close code, duration, client version, IP (respect privacy rules).
- Use **MDC** with session id/user id in an `ExecutorChannelInterceptor` (`beforeHandle` → put, `afterMessageHandled` → clear) so handler logs are traceable.
- Spring debug logging for troubleshooting: `logging.level.org.springframework.web.socket=DEBUG`, `logging.level.org.springframework.messaging=DEBUG` (very verbose; never leave on in prod).

9.8.3 Tracing

- Propagate a **trace/correlation id** in STOMP headers (`traceparent` style) from client → server → broker → consumers. Create spans manually around handlers (Micrometer Observation API) since auto-instrumentation for STOMP is limited.

9.8.4 Alerts

- Session count drops > 30% in 1 min (mass disconnect).
- Connect rate spikes (reconnect storm).
- Close code 1006 rate increase (network/proxy issue).
- Inbound queue size > threshold for 5 min.
- Broker unavailable.
- Send-limit closes increasing.

9.9 Load testing

Tool	STOMP support
Gatling	WebSocket support; write STOMP frames as text (`"CONNECT\naccept-version:1.2\n\n\u0000"`), check responses
k6	`k6/ws` and the newer `k6/websockets` (browser-like API) modules; send raw STOMP frames
Artillery	WebSocket engine; raw frames
JMeter	WebSocket Samplers plugin; raw frames
Custom Java/Kotlin	`WebSocketStompClient`/Krossbow to simulate realistic clients

k6 sketch:

```JS
import ws from 'k6/ws';
import { check } from 'k6';
export const options = { vus: 1000, duration: '5m' };
const NUL = '\u0000';
export default function () {
  const res = ws.connect('wss://staging.example.com/ws', { headers: { 'Sec-WebSocket-Protocol':
'v12.stomp' } }, (socket) => {
    socket.on('open', () => {
      socket.send(`CONNECT\naccept-version:1.2\nhost:staging\nheart-
beat:10000,10000\nAuthorization:Bearer ${__ENV.TOKEN}\n\n${NUL}`);
    });
    socket.on('message', (m) => {
      if (m.startsWith('CONNECTED')) {
        socket.send(`SUBSCRIBE\nid:sub-0\ndestination:/topic/chat.public\n\n${NUL}`);
        socket.setInterval(() => socket.send(`SEND\ndestination:/app/chat.send\ncontent-
type:application/json\n\n{"text":"hi"}${NUL}`), 5000);
      }
    });
    socket.setInterval(() => socket.send('\n'), 10000);   // client heart-beat
    socket.setTimeout(() => socket.close(), 290000);
  });
  check(res, { 'status 101': (r) => r && r.status === 101 });
}
```

Load-test lessons:

- Load generators hit their own limits (ephemeral ports, fds, CPU). Use several machines/IPs.
- Test **realistic mixes**: mostly idle connections + bursts; reconnect storms (kill a pod mid-test); slow clients (throttle network).
- Measure **end-to-end latency percentiles** (p50/p95/p99), not just throughput.
- Soak tests (hours) find memory leaks (subscriptions not cleaned, presence maps growing).

9.10 Graceful degradation

- If WebSocket can't connect after N tries, fall back to **polling** for critical screens (or SockJS).
- Show **connection status** in the UI ("Live", "Reconnecting…", "Offline, showing data from 10:42").
- Queue user actions offline (outbox) and replay with idempotency keys.
- Feature flags to **turn off** real-time features (e.g., typing indicators) under load while keeping core ones.
- Server-side **load shedding**: reject new connections with close code **1013 (Try Again Later)** when overloaded; clients back off longer.

9.11 Reliability checklist

- [ ] Every event has an id/sequence; clients dedupe and detect gaps.
- [ ] Clients resync via REST after reconnect.
- [ ] Subscribe-then-snapshot ordering.
- [ ] Commands carry idempotency keys; server dedupes.
- [ ] Events published after commit (or via outbox).

- [ ] Ordering requirements identified; per-entity sequence or `preservePublishOrder`.
- [ ] Heart-beats < all idle timeouts; timeouts tuned.
- [ ] Send buffer/time limits configured; conflation for high-frequency data.
- [ ] Handlers non-blocking or pool sized; bounded queues considered.
- [ ] Metrics + alerts on sessions, close codes, queues, broker.
- [ ] Load, soak and chaos (pod kill, broker restart, network loss) tested.
- [ ] Offline delivery via DB inbox/push for important notifications.

Next: Chapter 10: Testing, Debugging & the Bug Encyclopedia

CHAPTER 10

Chapter 10: Testing, Debugging & the Bug Encyclopedia

Goal: test real-time features confidently, debug any problem methodically, and look up any symptom to find its cause and fix. Keep this chapter open during incidents.

10.1 ELI5

When the telephone doesn't work, you don't randomly shake it. You check, **in order:** Is it plugged in? Is there a dial tone? Did the other person pick up? Are you speaking the same language? Did they hear you? Debugging WebSocket/STOMP is the same: check **layer by layer, from bottom to top.**

10.2 Testing strategy (the pyramid)

Level	What	Tools
Unit	Controller methods, services, interceptors, converters	JUnit, Mockito
Slice/integration	Full Spring context + real WebSocket + STOMP client	`@SpringBootTest(webEnvironment = RANDOM_PORT)`, `WebSocketStompClient`, Awaitility
Broker integration	Relay with a real RabbitMQ	Testcontainers
Front-end unit	Hooks/components with a mocked STOMP client	Jest/Vitest, mock client
End-to-end	Two browsers see each other's updates	Playwright/Cypress (two contexts)
Contract	Message schemas stay compatible	JSON Schema / AsyncAPI validation in CI
Load/soak/chaos	Capacity, leaks, resilience	k6, Gatling, pod-kill scripts

10.2.1 Unit testing a controller (no WebSocket at all)

JAVA

```
class TodoWsControllerTest {
    TodoService todos = mock(TodoService.class);
    ListAccessService access = mock(ListAccessService.class);
    TodoWsController controller = new TodoWsController(todos, access);

    @Test
    void upsertChecksMembershipAndDelegates() {
        Principal anna = () -> "anna";
        TodoCommand cmd = new TodoCommand("L1", null, "Buy milk", false, null, "req-1");
        controller.upsert(cmd, anna);
```

```
        verify(access).requireMember("anna", "L1");
        verify(todos).upsert(cmd, "anna");
    }
}
```

10.2.2 Unit testing an interceptor (build STOMP messages by hand)

JAVA

```java
@Test
void rejectsSubscriptionToForeignList() {
    ListAccessService access = mock(ListAccessService.class);
    when(access.isMember("anna", "L2")).thenReturn(false);
    SubscriptionGuard guard = new SubscriptionGuard(access);

    StompHeaderAccessor acc = StompHeaderAccessor.create(StompCommand.SUBSCRIBE);
    acc.setDestination("/topic/lists.L2");
    acc.setSubscriptionId("sub-0");
    acc.setSessionId("s1");
    acc.setUser(() -> "anna");
    acc.setLeaveMutable(true);                                    // so getAccessor(...) works inside
the interceptor
    Message<byte[]> msg = MessageBuilder.createMessage(new byte[0], acc.getMessageHeaders());

    assertThrows(AccessDeniedException.class, () -> guard.preSend(msg, mock(MessageChannel.class)));
}
```

10.2.3 Full integration test with a real STOMP client

JAVA

```java
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class ChatIntegrationTest {

    @LocalServerPort int port;
    @Autowired TestTokens tokens;            // helper that mints JWTs signed with the test key

    WebSocketStompClient stompClient;

    @BeforeEach
    void setup() {
        stompClient = new WebSocketStompClient(new StandardWebSocketClient());
        stompClient.setMessageConverter(new MappingJackson2MessageConverter()); // Spring 7:
JacksonJsonMessageConverter
    }

    @AfterEach
    void tearDown() { stompClient.stop(); }

    private StompSession connect(String user) throws Exception {
        StompHeaders h = new StompHeaders();
        h.add("Authorization", "Bearer " + tokens.forUser(user));
        return stompClient.connectAsync("ws://localhost:" + port + "/ws", new WebSocketHttpHeaders(), h,
                new StompSessionHandlerAdapter() {}).get(5, TimeUnit.SECONDS);
    }

    @Test
    void messageIsBroadcastToRoom() throws Exception {
        BlockingQueue<ChatMessage> received = new LinkedBlockingQueue<>();
        StompSession bob = connect("bob");
        bob.subscribe("/topic/chat.public", new StompFrameHandler() {
            public Type getPayloadType(StompHeaders headers) { return ChatMessage.class; }
            public void handleFrame(StompHeaders headers, Object payload) { received.add((ChatMessage)
payload); }
        });
```

```
        // Make sure the SUBSCRIBE has been processed before sending (avoid race): use a receipt
        // or a small await. Receipts need setTaskScheduler on the client and broker support.
        Thread.sleep(200);   // simplest; prefer receipts or polling a /app/ping round-trip

        StompSession anna = connect("anna");
        anna.send("/app/chat.send", new ChatMessage(null, "hello"));

        ChatMessage msg = received.poll(5, TimeUnit.SECONDS);
        assertThat(msg).isNotNull();
        assertThat(msg.sender()).isEqualTo("anna");      // server set identity, not client
        assertThat(msg.text()).isEqualTo("hello");
    }

    @Test
    void connectWithoutTokenFails() {
        assertThatThrownBy(() -> stompClient.connectAsync("ws://localhost:" + port + "/ws",
                new StompSessionHandlerAdapter() {}).get(5, TimeUnit.SECONDS))
            .isInstanceOf(Exception.class);
    }
}
```

Tips:

- **Race between SUBSCRIBE and SEND** is the #1 cause of flaky WebSocket tests. Use receipts (`session.setAutoReceipt(true)` + `subscribe(...).addReceiptTask(latch::countDown)`, where the server supports receipts), or a "ping" round-trip through the same session, or Awaitility polling of `SimpUserRegistry` subscriptions.
- Use `BlockingQueue.poll(timeout)` or Awaitility, never fixed long sleeps.
- Deterministic alternative used by the Appendix B sample: in tests, run the inbound channel with **one thread** (`registration.taskExecutor().corePoolSize(1)` behind a test property), so a session's frames are handled in arrival order. A round-trip `@SubscribeMapping` request then proves earlier SUBSCRIBEs are registered.
- Test negative paths: forbidden subscriptions, invalid payloads → error queue, expired tokens.
- Test **user destinations** with two sessions for the same user (both receive) and a different user (doesn't).

10.2.4 Broker relay tests with Testcontainers

JAVA

```
@Testcontainers
@SpringBootTest(webEnvironment = RANDOM_PORT)
class RelayIT {
    @Container
    static RabbitMQContainer rabbit = new RabbitMQContainer("rabbitmq:3-management")
            .withPluginsEnabled("rabbitmq_stomp")      // API name may differ by Testcontainers version;
            .withExposedPorts(5672, 15672, 61613);     // alternatively execInContainer("rabbitmq-
plugins","enable","rabbitmq_stomp")

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry r) {
        r.add("broker.relay.host", rabbit::getHost);
        r.add("broker.relay.port", () -> rabbit.getMappedPort(61613));
    }
    // ... same client-based tests; plus: restart broker and assert BrokerAvailabilityEvent + recovery
}
```

10.2.5 Front-end tests

- Wrap the STOMP client behind your own small interface (`RealtimeClient { subscribe, publish, onState }`) so components can be tested with a fake that you control (`fake.emit('/topic/x', payload)`).
- For lower-level tests, libraries such as **mock-socket** can fake the browser `WebSocket`.
- **Playwright E2E:** open two browser contexts logged in as different users; act in one and `expect(page2.getByText('Buy milk')).toBeVisible()` in the other. Also test offline: `context.setOffline(true)` → assert "Reconnecting…" → `setOffline(false)` → assert resync.

10.2.6 Contract tests for messages

- Define payload schemas (JSON Schema or **AsyncAPI** document listing channels/destinations, messages, and payloads).
- In CI: validate server-produced sample messages against the schema; generate TypeScript types for the client from the same schema; fail on breaking changes (removed/renamed fields).

10.3 The debugging method: layer by layer

```
Layer 7c  App logic        Did my handler run? Was the right data sent? Did UI render it?
Layer 7b  STOMP routing    Destination/prefix correct? Subscribed? Authorised? Converted?
Layer 7a  STOMP session    CONNECT → CONNECTED? ERROR frame? Heart-beats negotiated?
Layer 6   WebSocket        101 received? Close code? Frames flowing?
Layer 5   HTTP/proxy       Upgrade headers forwarded? Timeouts? Status code?
Layer 4   TLS/TCP/DNS      Resolves? Port open? Cert valid? Firewall?
```

Step-by-step:

1. **Browser DevTools → Network → WS.** Is there a request to `/ws`? Status **101**? If not, look at the status code (Chapter 2 §2.3.5 table).
2. **Messages tab.** Do you see `CONNECT` going up and `CONNECTED` coming down? If you see `ERROR`, read its `message` header and body.
3. Check **heart-beat** header values in CONNECT/CONNECTED.
4. Check your **SUBSCRIBE** frames: exact destination string, `id`.
5. Send and verify the **SEND** frame destination (`/app/...`) and `content-type`.
6. Do **MESSAGE** frames arrive? With which `subscription` id? If MESSAGE frames arrive but the UI doesn't update → client code bug (wrong callback, parse error, state bug).
7. **Server logs** with `DEBUG` on `org.springframework.messaging` and `org.springframework.web.socket` (temporarily): you'll see each message, destination, handler resolution, and broker deliveries.
8. `WebSocketMessageBrokerStats` log line: session counts, abnormal closes, queue sizes.
9. **Close code** in `onWebSocketClose` (1000/1001/1006/1008/1009/1011/4xxx) narrows it down fast.
10. Reproduce with **raw tools** (`wscat` / `websocat` /Postman) to separate client-library bugs from server bugs.
11. **Behind proxies:** test directly against the app (port-forward to the pod) vs through the LB. If direct works and LB doesn't, it's proxy config.

12. **Packet level:** Wireshark/tcpdump on the server (`tcpdump -i any port 8080 -A`) for unencrypted internal hops.

10.4 The Bug Encyclopedia (symptom → cause → fix)

A. Connection & handshake problems

#	Symptom	Likely cause	Fix
A1	`WebSocket connection to 'wss://.../ws' failed` with **404**	Wrong path; app has `server.servlet.context-path` (e.g., `/api` → `/api/ws`); SockJS endpoint used with raw WS; gateway route missing	Use the correct full path; for SockJS endpoints use SockJS client or `/ws/websocket` ; add route
A2	Handshake returns **200**	Request hit an HTML page (SPA fallback / static route) or proxy didn't treat it as upgrade	Route `/ws` to the backend before SPA fallback; fix proxy Upgrade headers
A3	**403** on handshake	Origin not allowed (`setAllowedOrigins`); Spring Security HTTP rules deny `/ws` ; CSRF in HTTP layer (rare for GET)	Add origin to allow-list (exact scheme+host+port); `permitAll` or authenticate `/ws` deliberately
A4	**400** "Can "Upgrade" only to "WebSocket"" / "Connection header must include Upgrade"	Proxy stripped/rewrote `Upgrade` / `Connection`	Nginx: `proxy_http_version 1.1` + `Upgrade` + `Connection` headers
A5	Works locally, fails in prod behind Nginx/Ingress	Same as A4, or HTTP/2-only hop without extended CONNECT	Fix proxy config; ensure HTTP/1.1 upgrade to backend
A6	Browser: "Mixed Content: … insecure WebSocket endpoint"	`ws://` from an `https://` page	Use `wss://`
A7	SockJS: CORS error on `/ws/info`	SockJS uses XHR → CORS applies; origin not allowed	`setAllowedOriginPatterns(...)` on the endpoint
A8	Android emulator can't connect to `ws://localhost:8080`	`localhost` = the emulator	Use `10.0.2.2` (emulator) or LAN IP (device)
A9	Android: `CLEARTEXT communication not permitted`	Android 9+ blocks cleartext	Use `wss://` , or dev-only network security config
A10	iOS: connection fails to `ws://`	ATS blocks non-TLS	`wss://` or dev-only ATS exception
A11	Immediate close **1006** with no response, only in some environments	TLS certificate invalid/self-signed; corporate proxy; firewall	Fix certificate chain; test with `openssl s_client` ; try SockJS/443
A12	Spring log: "Handshake failed due to invalid Upgrade header: null"	Plain HTTP request reached WS endpoint (health check, crawler, misrouted)	Usually harmless; make health checks hit `/actuator/health`
A13	"Unexpected server response: 502/504"	LB can't reach backend / backend slow / backend not listening on that port	Check target health, ports, security groups
A14	New connections refused at ~8k per instance	Tomcat `server.tomcat.max-connections` (default 8192) reached	Raise `max-connections` (+ fds), or scale out

#	Symptom	Likely cause	Fix
A 15	Connection fails only on corporate networks	Proxy blocks/breaks WebSocket	Use `wss://` on 443; enable SockJS fallbacks

B. Connection drops and reconnects

#	Symptom	Likely cause	Fix
B 1	Drops every **~60 s** when idle	Nginx/ALB idle timeout (60 s default) with no heart-beats	Enable heart-beats (10–25 s) + raise proxy timeouts
B 2	Drops after exactly **30 s** on GCP	Backend service timeout = WS max duration	Raise backend service timeout (e.g., 3600 s+)
B 3	Drops around **100 s** idle behind Cloudflare	Cloudflare idle timeout	Heart-beats < 100 s
B 4	Frequent **1006** on mobile	Network changes, background suspension	Heart-beats, reconnect with backoff, resync, push notifications for background
B 5	Client: "did not receive server activity for the last N ms" then reconnect	Heart-beats were negotiated but the server's beats (or any data) don't reach the client in time: outbound thread pool saturated, a proxy buffering the stream, long GC pauses, or the browser throttling timers in a background tab	Check the outbound pool and GC logs, set `proxy_buffering off`, give heart-beats more tolerance, and resync when a tab becomes visible again
B 6	Server closes sessions with **4500** / stats show "send limit"	Slow client exceeded `sendBufferSizeLimit` / `sendTimeLimit`	Reduce message rate (conflate/batch), raise limits moderately, investigate client network
B 7	**1009** close	Message larger than container buffer or Spring limit	Raise `setMessageSizeLimit` and container buffers, or send smaller messages
B 8	Everyone disconnects at deploy and servers spike	Expected disconnect + synchronized reconnect (thundering herd)	Jittered backoff, rolling deploy, readiness/preStop, capacity headroom
B 9	Infinite reconnect loop, each attempt gets ERROR "Invalid token"	Token expired; client reuses old token	Refresh in `beforeConnect`; on auth ERROR force re-login; don't retry identical credentials forever
B 10	Connection closes ~60 s after connect without traffic, server log "no messages received"	`timeToFirstMessage` exceeded: client never sent CONNECT (e.g., RN NULL issue, client bug)	Fix client CONNECT; check RN flags
B 11	RabbitMQ log "missed heartbeats from client, timeout: Xs" and closes	Client heart-beats not reaching broker in time (blocked JS thread, mobile sleep)	Increase interval; avoid blocking main thread; reconnect
B 12	Connections slowly become unbalanced across pods	Long-lived connections don't rebalance	Least-conn LB; max connection age with jitter

C. STOMP session problems

#	Symptom	Likely cause	Fix
C1	WebSocket opens but `onConnect` never fires	CONNECT not sent/parsed: RN NULL chopping; wrong sub-protocol; server not STOMP endpoint (raw handler)	RN: `forceBinaryWSFrames` + `appendMissingNULLonIncoming`; check endpoint config
C2	Immediately ERROR + close after CONNECT	Exception in an inbound interceptor (auth failed, CSRF missing)	Read ERROR `message`; fix token/CSRF; customise error handler for clearer messages
C3	ERROR "Failed to send message to ExecutorSubscribableChannel[clientInboundChannel]; nested exception is ... AccessDeniedException"	Spring Security authorisation denied the frame	Fix message security rules or interceptor order (`@Order(HIGHEST_PRECEDENCE + 99)`)
C4	`MissingCsrfTokenException` / "Could not verify the provided CSRF token because no token was found" on CONNECT	`@EnableWebSocketSecurity` requires CSRF token in CONNECT	Send the CSRF header in `connectHeaders`, or configure without CSRF for header-token auth (Chapter 8)
C5	`Principal` is `null` in `@MessageMapping`	User never set: used `StompHeaderAccessor.wrap()` copy, or interceptor not on inbound channel, or handshake unauthenticated	Use `MessageHeaderAccessor.getAccessor(...)` + `setUser` on CONNECT
C6	CONNECTED received but heart-beat header is `0,0`	Simple broker without TaskScheduler; or client sent `0,0`	Configure `setTaskScheduler` + `setHeartbeatValue`
C7	`BeanCurrentlyInCreationException` / circular dependency when injecting TaskScheduler	Scheduler depends on message broker config	Dedicated scheduler bean or `@Lazy` injection
C8	Old client gets ERROR on ACK	Client sends 1.1-style ACK (`message-id` / `subscription`) to a 1.2 session	Use 1.2 `id` = MESSAGE `ack` header (update library)
C9	ERROR "Broker not available" / `MessageDeliveryException: Message broker not active`	Relay can't reach broker (down, wrong host/port, credentials, network policy, reactor-netty missing)	Check `BrokerAvailabilityEvent` logs, broker connectivity, credentials, add `io.projectreactor.netty:reactor-netty` + `io.netty:netty-all`
C10	A rejected CONNECT/SUBSCRIBE (bad token, forbidden destination) gets **no ERROR frame**; the client hangs until its connect timeout; server logs `Failed to send GenericMessage [...]` from `OrderedMessageChannelDecorator`	`StompEndpointRegistry#setPreserveReceiveOrder(true)` is on. The ordering decorator catches and only **logs** exceptions thrown by inbound interceptors, so `StompSubProtocolHandler` never sends ERROR or closes the session (verified in Spring Framework 7.0.9 and `main`)	Don't combine `setPreserveReceiveOrder(true)` with rejecting interceptors, or make the client time out the CONNECT (`connectionTimeout`) and treat it as an auth failure. Found while testing the Appendix B sample
C11	Client sees close code **1002** right after an ERROR frame	Expected: Spring closes the session with `CloseStatus.PROTOCOL_ERROR` (1002) after any STOMP ERROR (default error path and custom `StompSubProtocolErrorHandler` alike)	Read the ERROR frame's `message` header for the reason; don't treat 1002 as a network problem

D. Routing & delivery problems

#	Symptom	Likely cause	Fix
D1	`@MessageMapping` method never called	Client sends to `/chat.send` instead of `/app/chat.send`; prefix mismatch; class not a Spring `@Controller` bean; path separator mismatch (`.` vs `/`)	Align prefix; component scanning; consistent `PathMatcher`
D2	Return value of `@MessageMapping` not received by client	Client subscribed to a different destination than the default (`/topic` + input destination)	Use explicit `@SendTo`; subscribe to the right topic
D3	`SimpMessagingTemplate.convertAndSend("/app/x")` doesn't invoke controller	Template sends to **broker** channel; `/app` only routes **client** messages	Call the service directly; or send to broker destinations only
D4	Message sent to `/queue/x` silently disappears	Broker only configured for `/topic` (`enableSimpleBroker("/topic")`)	Add the prefix to the broker config
D5	Subscribed but receives nothing	Subscribed **before** connected (exception swallowed); typo in destination; leading/trailing spaces in header; destination not authorised (dropped)	Subscribe in `onConnect`; log frames; check security rules
D6	`convertAndSendToUser` messages never arrive	Name mismatch with `Principal#getName()` (email vs id vs `sub`); client subscribed to `/queue/x` instead of `/user/queue/x`; no Principal on session; user connected to **another instance**	Align names; subscribe with `/user` prefix; set Principal; enable user destination broadcast (relay)
D7	Users on server A don't get messages sent on server B	Simple broker per instance	Broker relay or Redis/Kafka backplane
D8	Duplicate messages in UI	Multiple subscriptions to same destination (React re-renders, StrictMode, subscribing in `onConnect` and in components, multiple Client instances)	Single client + ref-counted subscriptions; unsubscribe on cleanup
D9	Messages lost after reconnect	No re-subscribe; events during downtime not replayed	Subscribe in `onConnect`; resync via REST with last seq
D10	Messages arrive out of order	Thread pools; multiple producers	`setPreservePublishOrder(true)`, `setPreserveReceiveOrder(true)`; sequence numbers
D11	Client logs "Unhandled received MESSAGE"	MESSAGE for a subscription that was already unsubscribed (in-flight), or subscription id mismatch	Usually benign; ensure ids are unique; handle `onUnhandledMessage`
D12	`@SendToUser` reply goes to all tabs	`broadcast = true` default	`@SendToUser(broadcast = false)`
D13	`@SendToUser` for anonymous users	No Principal → Spring falls back to the **session** (single session delivery)	Works for the current session; for later pushes keep session id or assign anonymous Principal in handshake
D14	RabbitMQ wildcard subscription doesn't match	Using `/` separators in routing keys (`/topic/prices/AAPL`) with `*` / `#`	Use dots: `/topic/prices.AAPL`, `AntPathMatcher(".")`
D15	RabbitMQ ERROR "not_found"	`/amq/queue/x` where queue x doesn't exist	Declare queue first, or use `/queue/x`

#	Symptom	Likely cause	Fix
D16	RabbitMQ queue count grows forever	Per-session user queues (`/queue/x-user...`) not auto-deleted	Use `/exchange/amq.direct/...` user destinations, policies with `x-expires`
D17	Subscription works for user A but A can still see list after being removed	Authorisation only at SUBSCRIBE time	Revoke: close sessions / re-check on change / send through user destinations

E. Payload / conversion problems

#	Symptom	Likely cause	Fix
E1	Handler receives `null` fields / conversion error	Missing `content-type: application/json` ; record/class not deserialisable; unknown properties failing	Send content-type; check Jackson config; `@JsonIgnoreProperties(ignoreUnknown = true)`
E2	`MessageConversionException: Cannot convert from [[B] to [com.x.Dto]`	No suitable converter (Jackson missing, wrong content-type like `text/plain`)	Add Jackson; set content-type; register converter
E3	Dates arrive as arrays `[2024,1,1,...]` or fail	Jackson without JavaTimeModule / custom ObjectMapper not used for messaging	Use Boot's ObjectMapper (auto-configured), or register modules in your converter
E4	Client gets `"\"hello\""` (double-quoted)	Returning a `String` converted as JSON	Return an object, or configure `StringMessageConverter` precedence
E5	Non-ASCII text garbled / frames corrupted	`content-length` computed in characters; wrong charset	Let the library compute; UTF-8 everywhere
E6	`JSON.parse` error in client	Body isn't JSON (string/error text); binary body read as text	Check `content-type` ; use `binaryBody` for binary
E7	Large payload fails only for some users	Over message size limits (B7)	Limits or redesign
E8	`@Valid` not triggering	Missing validation starter / `@Valid`	Add starter and annotation
E9	`MethodArgumentNotValidException` handler not invoked	Imported MVC class instead of `org.springframework.messaging.handler.annotation.support.MethodArgumentNotValidException`	Fix import

F. Server performance & stability

#	Symptom	Likely cause	Fix
F1	Latency grows under load, CPU low	Blocking handlers saturating inbound pool; `maxPoolSize` ineffective with unbounded queue	Increase core size; offload blocking work; async returns
F2	`OutOfMemoryError`	Slow clients buffering; unbounded executor queues; leaked maps (presence, rate-limit buckets); huge subscriptions	Send limits; bounded queues; cleanup on disconnect; heap dump analysis

#	Symptom	Likely cause	Fix
F3	`IllegalStateException: The remote endpoint was in state [TEXT_PARTIAL_WRITING]`	Concurrent sends to a raw `WebSocketSession`	Wrap with `ConcurrentWebSocketSessionDecorator` or single-threaded sending
F4	GC pauses → heart-beat timeouts → mass reconnects	Large heap, allocation-heavy fan-out	GC tuning (G1/ZGC), reduce allocations, conflation, more instances
F5	Broker CPU/memory high, connections at limit	Relay creates 1 TCP connection per client	Size broker, or move to backplane pattern
F6	`SessionDisconnectEvent` handlers run twice	Documented behaviour	Make handlers idempotent
F7	Wrong "online" counts in cluster	Local in-memory presence	Redis-based presence with TTL, or `userRegistryBroadcast`
F8	Autoscaler doesn't add pods while sockets max out	HPA on CPU	Scale on connection count/memory custom metrics

G. Client-specific problems

#	Symptom	Likely cause	Fix
G1	`TypeError: There is no underlying STOMP connection`	`publish` / `subscribe` before connected or after disconnect	Check `client.connected`; outbox; subscribe in `onConnect`
G2	Two connections in dev (React)	StrictMode double-invokes effects	Singleton client; tolerate in dev
G3	Memory leak in SPA	Subscriptions/listeners not cleaned on unmount	Unsubscribe in effect cleanup
G4	Stale state in callbacks	Closure captured old state	Functional `setState(prev => ...)`, refs
G5	React Native: `ReferenceError: TextEncoder`	Missing polyfill on old RN/JSC	Import `text-encoding` polyfill
G6	React Native: frames garbled / never CONNECTED	NULL byte truncation	`forceBinaryWSFrames: true`, `appendMissingNULLonIncoming: true`
G7	Old tutorial code `Stomp.over(...)` doesn't reconnect	Legacy API with a socket instance	Migrate to `new Client({...})` + `activate()`
G8	Token visible in logs	`debug` enabled in production, or token in URL	Disable debug; CONNECT headers; redact
G9	Background tab stops receiving / timers throttled	Browser throttles timers in background tabs; heart-beats may be delayed	stompjs: `heartbeatStrategy: TickerStrategy.Worker` (heart-beats from a Web Worker), raise `heartbeatToleranceMultiplier`; resync on `visibilitychange`

#	Symptom	Likely cause	Fix
G 11	Reconnecting **stops forever** after the auth server or token endpoint was briefly down; Node logs an unhandled rejection (`fetch failed`)	stompjs 7.x awaits `beforeConnect` **without try/catch** and calls `_connect()` with `void` . A throwing `beforeConnect` rejects, nothing schedules the next attempt	Wrap the token fetch in `beforeConnect` with try/catch (keep the old headers and let the attempt fail normally); add `onChangeState` monitoring
G 12	After a server restart the client says "reconnecting" forever; debug shows `Connection not established in Nms, closing socket` and nothing after	The connection attempt got stuck in CONNECTING, and `close()` on it never produced a `close` event (observed with Node's built-in WebSocket), so no reconnect was scheduled	`discardWebsocketOnCommFailure: true` (stompjs drops the socket and synthesises the close → reconnect proceeds)
G 13	Client keeps showing stale data after the server lost its state (restart with in-memory store, DB restore)	Resync asked for "events after seq N" but the server's sequence restarted below N and replied "nothing new"	Server must answer "reload" (e.g., HTTP 410) when the client's seq is **ahead** of its own; client then loads a full snapshot
G 10	App reconnect hammering after server outage	Fixed short `reconnectDelay` across all clients	stompjs: `reconnectTimeMode: ReconnectionTimeMode.EXPONENTIAL` + `maxReconnectDelay` . It adds no jitter, so also randomise `reconnectDelay` per client (e.g., `1000 + Math.random()*4000`)

10.5 "What do I do when..." playbook

...users report "real-time stopped working" (incident)?

1. Check dashboards: session count, connect rate, close codes, broker availability.
2. Are connections being established (101s at LB)? If not → LB/proxy/certificate/DNS.
3. Are CONNECTs failing (ERROR)? → auth provider/JWKS endpoint down? clock skew? CSRF change?
4. Are messages produced? Check the producing service logs and the broker (queue depth, publish rates).
5. Are messages routed? Check broker bindings/subscriptions; relay availability events.
6. Mitigate first (roll back recent deploy, scale out, restart a stuck broker node), then root-cause.

...a deploy causes a spike of errors? Jittered reconnect present? Readiness gating? Maybe the new version changed message formats (backward compatibility!) or security rules (403/ERROR).

...memory grows steadily over days? Heap dump: look for `ConcurrentWebSocketSessionDecorator` buffers (slow clients), your own maps keyed by session id (missing cleanup on disconnect), `DefaultSubscriptionRegistry` sizes (subscriptions never unsubscribed), executor queues.

...one customer says they never get updates but others do? Their network/proxy (corporate) → check their close codes/transport; their user id mapping (D6); authorisation for their tenant; they're subscribed to the wrong destination after a client version mismatch.

...messages are slow (seconds of delay)? Inbound/outbound queue sizes, GC pauses, broker queue backlogs, proxy buffering (`proxy_buffering off`), client main-thread blocking (huge re-renders per message → batch UI updates).

...you must add a new real-time feature fast? Reuse the existing connection, define destination naming + authorisation rule + schema first, add to AsyncAPI doc, implement with resync story, load test fan-out.

10.6 Useful log/config snippets

PROPERTIES

```
# Temporarily, when debugging locally
logging.level.org.springframework.web.socket=DEBUG
logging.level.org.springframework.messaging=DEBUG
logging.level.org.springframework.security.messaging=DEBUG
logging.level.org.springframework.web.socket.config.WebSocketMessageBrokerStats=INFO
```

JAVA

```
// Log every inbound STOMP command (redacting secrets) for troubleshooting
public class StompAuditInterceptor implements ChannelInterceptor {
    private static final Logger log = LoggerFactory.getLogger(StompAuditInterceptor.class);
    @Override
    public Message<?> preSend(Message<?> m, MessageChannel c) {
        StompHeaderAccessor a = MessageHeaderAccessor.getAccessor(m, StompHeaderAccessor.class);
        if (a != null && a.getCommand() != null) {
            log.debug("in cmd={} session={} user={} dest={} subId={}",
                a.getCommand(), a.getSessionId(),
                a.getUser() != null ? a.getUser().getName() : null,
                a.getDestination(), a.getSubscriptionId());
        }
        return m;
    }
}
```

Next: Chapter 11: Architecture, Design Patterns & Organisational Best Practices

CHAPTER 11

Chapter 11: Architecture, Design Patterns & Organisational Best Practices

Goal: design real-time systems the way strong senior engineers and architects do, and run them in a big organisation with many teams, environments, audits and on-call rotations.

11.1 ELI5

Building one walkie-talkie is easy. Running a **city-wide radio network** means rules: who gets which channel, what words to use, what to do when a tower breaks, how to add new channels without confusing old radios, and who fixes things at 3 a.m. That's what this chapter is about.

11.2 Decision framework: should this feature be real-time at all?

Ask:

1. **Does the user need it within seconds?** If minutes are fine → polling/refresh/email.
2. **Direction?** Server→client only → consider **SSE**. Both ways, high frequency → **WebSocket**.
3. **Online only?** Offline users need **push notifications / inbox** anyway.
4. **Audience size per message?** 1 (private) vs 10 (room) vs 1M (broadcast) changes the design.
5. **Message rate?** 1/min vs 50/s per client: conflation, binary formats.
6. **Consistency needs?** Is missing an event acceptable? (UI hint → yes; balance update → no → resync/ack design.)
7. **Ops maturity?** Stateful infra, brokers, on-call. Or buy a managed service.

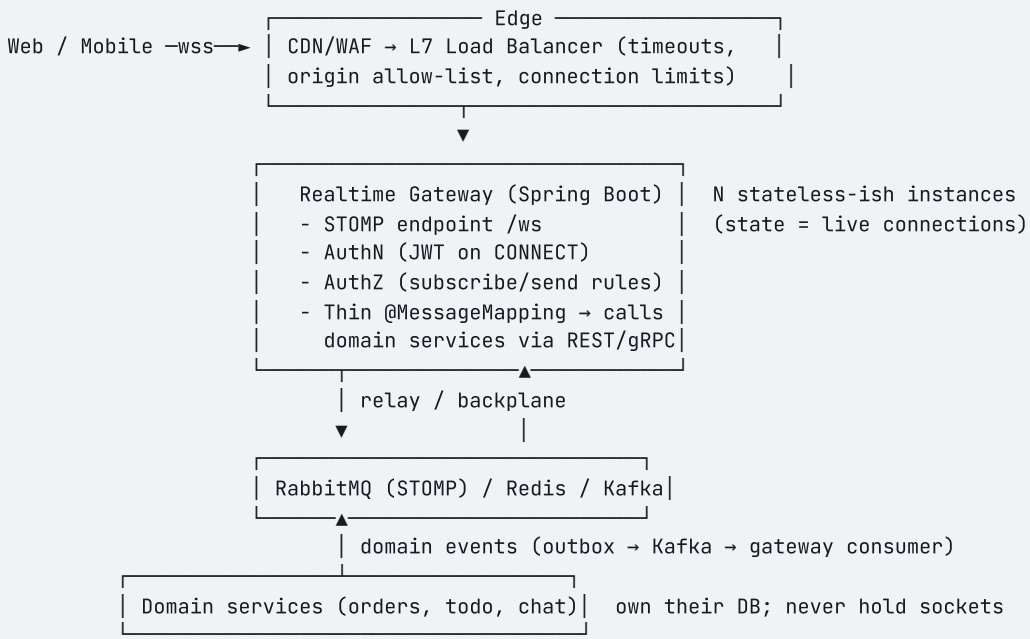
Why STOMP (vs raw WebSocket/custom JSON)? Standard framing, pub/sub semantics, Spring integration (`@MessageMapping`, user destinations, security), broker interoperability, mature clients. **Why not?** Very high-frequency binary data (games, audio), non-Spring stacks where Socket.IO/SignalR are idiomatic, or when you need features STOMP lacks (built-in replay → consider Kafka-backed gateways, Ably, etc.).

Architecture Decision Record (ADR) template

```
# ADR-012: Real-time updates for shared to-do lists
Status: Accepted (2026-10-01)
Context: Users collaborate on lists; changes must appear on other devices within 2s. Mobile + web.
Decision: STOMP over WebSocket via Spring Boot "realtime-gateway"; RabbitMQ relay in prod; JWT in
CONNECT;
          events carry per-list sequence numbers; clients resync via REST after reconnect.
Alternatives: SSE (no client→server channel needed? we need typing/presence → rejected), polling
(latency/cost),
```

```
          Firebase (data residency), Socket.IO (Java stack).
Consequences: Stateful service to operate; RabbitMQ capacity planning; need LB timeouts; on-call
runbook.
```

11.3 Reference architecture for a company



Principles:

- **Separate the real-time gateway** from domain services. Domain teams publish **events**; the gateway owns connections, auth, fan-out. Domain services stay stateless and independently deployable.
- **Events in, pushes out:** the gateway consumes domain events (Kafka/RabbitMQ) and maps them to STOMP destinations, so domain teams don't need to know about WebSockets.
- **Commands via REST** (or `/app` messages forwarded to services). Many companies send **commands over HTTP** (easy auth, idempotency, retries, observability) and use WebSocket **only for server→client pushes**. That's a valid, simpler design.
- **Gateway is horizontally scalable**; connection state is disposable (clients reconnect + resync).

11.4 Destination naming conventions (write them down!)

A consistent, documented scheme prevents bugs and security holes.

```
Client → server (commands)       /app/{domain}.{action}              /app/todo.upsert, /app/chat.send
Public broadcast                  /topic/{domain}.{scope}             /topic/system.announcements
Entity-scoped                     /topic/{domain}.{entityType}.{id}   /topic/todo.list.L1,
/topic/chat.room.42
Tenant-scoped                     /topic/t.{tenantId}.{domain}...     /topic/t.acme.orders.123
Private per user                  /user/queue/{purpose}               /user/queue/notifications,
/user/queue/errors
```

```
Request/reply initial data        /app/{domain}.{resource}            (@SubscribeMapping)
/app/todo.lists
```

Rules:

- **Dots** as separators (RabbitMQ-friendly), lowercase, no spaces.
- IDs must be **validated** (regex) and **authorised**.
- Never encode secrets or PII (emails) in destinations (they appear in logs and broker UIs).
- Keep a **registry** (AsyncAPI) of every destination: owner team, payload schema, auth rule, rate.

11.5 Message envelope and versioning

Use a consistent envelope for every event:

JSONC

```jsonc
// (comments added for explanation; real JSON has none)
{
  "id": "01J9Z6Q7T4M8K2H5V3B1C0D9EF",          // unique event id (ULID/UUID) for dedupe
  "type": "todo.item.updated",                  // event type
  "version": 2,                                 // schema version of `data`
  "seq": 1842,                                  // per-entity sequence for ordering/gap detection
  "occurredAt": "2026-10-15T10:21:33.120Z",
  "correlationId": "c-7f2a…",                   // trace across systems / ties to clientRequestId
  "actor": { "id": "u-123", "type": "user" },
  "data": { "listId": "L1", "itemId": "I9", "title": "Buy milk", "done": true }
}
```

Versioning rules (consumers in the field may be **old app versions for months**, especially mobile):

- **Additive changes only** within a version (add optional fields; never remove/rename/retype).
- Clients **ignore unknown fields** and unknown event `type`s.
- Breaking change → new `version` or new `type`, and the server sends **both** until old clients are gone (or negotiate by client version sent in CONNECT headers, e.g., `client-version: 5.2.0`).
- Track client versions in metrics to know when it's safe to drop old formats.

AsyncAPI (document your real-time API like OpenAPI documents REST)

YAML

```yaml
asyncapi: 3.0.0
info: { title: Realtime Gateway, version: 1.4.0 }
servers:
  production:
    host: api.example.com
    pathname: /ws
    protocol: wss          # STOMP over WebSocket; document the STOMP layer in description/bindings
channels:
  todoList:
    address: /topic/todo.list.{listId}
    parameters: { listId: { description: "List id, must be a member" } }
    messages:
      itemUpdated: { $ref: '#/components/messages/TodoItemUpdated' }
operations:
  receiveTodoUpdates:
    action: receive
    channel: { $ref: '#/channels/todoList' }
```

```yaml
components:
  messages:
    TodoItemUpdated:
      payload:
        type: object
        required: [id, type, seq, data]
        properties:
          id: { type: string }
          type: { const: todo.item.updated }
          seq: { type: integer }
          data: { $ref: '#/components/schemas/TodoItem' }
```

Generate docs and TypeScript/Java types from it; validate in CI.

11.6 Design patterns catalogue

Pattern	Problem	Solution
Notification + fetch	Big payloads, consistency	Push small "changed (id, version)" events; client fetches via REST
Snapshot + stream	Initial state + live updates without gaps	Subscribe first, fetch snapshot (or `@SubscribeMapping`), apply buffered events with seq > snapshot version
Resync on reconnect	Missed events while offline	Client stores last seq; REST `?after=seq` or full reload
Idempotent commands	Retries cause duplicates	`clientRequestId` + server dedupe
Optimistic UI	Latency	Apply locally, reconcile on server event, roll back on error
Conflation	High-frequency updates	Latest-value-per-key every X ms
Batching	Many tiny messages	Send arrays every X ms
Fan-out on write	Broadcast to many	Broker/backplane fan-out, topic per scope
Request/reply over STOMP	RPC-like call	`@SendToUser(broadcast=false)` reply or `reply-to` + `correlation-id` headers
Presence with TTL	Who's online across a cluster	Redis keys with TTL refreshed by heart-beats/events; publish presence changes
Outbox	Reliable event publishing	Event row in same transaction; relay publishes
Gateway pattern	Many teams, one connection	Central real-time gateway consuming domain events
Backplane	Scale simple broker	Redis/Kafka fan-out between nodes
Circuit breaker on push	Downstream overload	Shed non-critical events (typing indicators) first

---

11.7 Worked designs (interview-ready)

11.7.1 Chat application (Slack/WhatsApp-like, web + mobile)

- **Destinations:** `/topic/chat.room.{roomId}` (channels), `/user/queue/dm` (direct messages), `/user/queue/receipts` (delivered/read), `/topic/chat.room.{roomId}.typing` (ephemeral).
- **Send:** `/app/chat.send` with `clientMsgId`; server validates membership, persists (DB with per-room `seq`), then publishes after commit.
- **History:** REST paginated `GET /rooms/{id}/messages?before=seq`.
- **Reliability:** clients dedupe by `clientMsgId` / `id`, detect gaps with `seq`, resync on reconnect; offline → push notification.
- **Typing indicators:** ephemeral, rate-limited (max 1 per 3 s per user), never persisted, dropped first under load.
- **Read receipts:** batched ("read up to seq N"), not per message.
- **Presence:** Redis TTL keys, publish only changes; for huge orgs, presence only for contacts/visible users.
- **Scale:** gateway + RabbitMQ relay or Kafka partitioned by roomId → per-room ordering.
- **Security:** room membership checked on SUBSCRIBE and SEND; sender from Principal; XSS-safe rendering; attachments via pre-signed URLs, not over the socket.

11.7.2 Notification system

- `/user/queue/notifications` for online users; persist all in an **inbox table** (unread counts via REST).
- Domain services emit events → notification service decides channels (in-app, push, email) → gateway pushes in-app.
- On connect: client fetches unread count + latest notifications via REST, then listens.

11.7.3 Live dashboard / stock ticker

- `/topic/prices.{symbol}`; client subscribes only to visible symbols.
- **Conflation** (e.g., 4–10 updates/s per symbol), deltas, binary encoding for very high rates.
- Slow clients get dropped (send limits) and resync with snapshot.
- Separate critical (order fills: `/user/queue/fills`, at-least-once with resync) from best-effort market data.

11.7.4 Collaborative to-do list (this repo's domain)

- Events per list with `seq`; snapshot via `@SubscribeMapping("/todo.list.{id}")` or REST.
- Conflict handling: optimistic concurrency (`expectedVersion`) → conflict error to `/user/queue/errors` → client refetches item.
- Offline-first mobile: local DB (SQLite/WatermelonDB/Realm), outbox of commands with `clientRequestId`, replay on reconnect.
- For rich-text **collaborative editing** use OT/CRDT (e.g., Yjs/Automerge). Those usually have their own sync protocol over WebSocket; STOMP can carry their binary updates but is rarely the best fit.

11.7.5 Live location tracking (delivery app)

- Driver app publishes location every 3–5 s to `/app/driver.location` (HTTP is also fine).
- Server stores latest location (Redis GEO), publishes to `/topic/order.{orderId}.location` for the customer (authorised to that order only).
- Conflate; send only when moved > N meters.

11.7.6 Long-running job progress

- Client starts job via REST → gets `jobId`.
- Subscribes to `/user/queue/jobs` or `/topic/job.{jobId}` (authorised).
- Worker publishes progress via the event pipeline; final state persisted; client polls REST once on reconnect.

11.8 Organisational best practices

11.8.1 Ownership and governance

- One team owns the **real-time platform** (gateway, broker, client SDK, docs, runbooks). Domain teams own **their events**.
- **Destination registry** (AsyncAPI) reviewed like an API change; security review for new destinations (auth rule mandatory).
- **Shared client SDK** (TypeScript + Kotlin/Swift wrappers) encapsulating connection, reconnect, resync, auth, metrics. Teams shouldn't each write their own STOMP setup.

11.8.2 Environments and configuration

- Same topology in staging as prod (proxy, LB timeouts, broker relay). "Works in dev with simple broker, fails in prod with RabbitMQ" is a classic.
- Externalise: allowed origins, broker host/creds (secret manager), heart-beat values, limits, feature flags.
- Never commit broker passwords; rotate credentials.

11.8.3 Code organisation (Spring)

```
realtime-gateway/
  config/        WebSocketConfig, WebSocketSecurityConfig, BrokerRelayConfig, TransportLimits
  security/      JwtConnectInterceptor, SubscriptionGuard, RateLimitInterceptor, CustomStompErrorHandler
  controller/    *WsController (thin: validate, authorise, delegate)
  events/        Domain event consumers → SimpMessagingTemplate publishers
  presence/      PresenceService (Redis)
  metrics/       WsMetrics, executor binders
  docs/          asyncapi.yaml
```

- Controllers thin; business logic in services (testable without sockets).
- All destination strings in **one constants class** (no string typos scattered around).

11.8.4 Code review checklist for real-time changes

- [ ] New destination documented (AsyncAPI) with schema, owner, auth rule.
- [ ] SUBSCRIBE/SEND authorisation implemented and tested (including negative tests).
- [ ] Identity from Principal; tenant isolation.
- [ ] Payload backward compatible; unknown fields ignored by clients.
- [ ] Event has id + seq; client handles duplicates/gaps; resync path exists.
- [ ] Published after commit/outbox.
- [ ] Rate and size reasonable; conflation if high-frequency.
- [ ] No blocking calls in handlers without pool sizing.
- [ ] Metrics/logs added; no PII/tokens logged.
- [ ] Client unsubscribes on cleanup.

11.8.5 SLOs and on-call

Example SLOs:

- **Connection success rate** ≥ 99.5% (successful CONNECTED / attempts, excluding auth failures).
- **Push latency** p95 < 1 s from event commit to client receipt (sampled).
- **Unexpected disconnect rate** < X per connection-hour.

Runbook entries (each with symptoms, dashboards, mitigation, escalation):

1. Mass disconnect / reconnect storm.
2. Broker unavailable (relay).
3. Auth failures spike (JWKS/IdP problem, clock skew, key rotation).
4. Memory pressure on gateway pods.
5. Message lag / backlog in broker or Kafka consumer.
6. Single tenant/customer can't connect (proxy, origin, firewall).

11.8.6 Release practices

- Rolling deploys with **connection draining** and jittered client reconnect.
- **Canary** gateway instances: route a small % of new connections; compare error/close-code rates.
- **Feature flags** for new event types (server sends only when flag on; client ignores unknown types anyway).
- Mobile: keep old payload versions alive long enough; force-upgrade path for truly breaking changes.
- Chaos drills: kill a gateway pod, restart a broker node, inject latency. Verify recovery and resync.

11.8.7 Compliance and audit

- Data residency: brokers and gateways in the right region; per-region topics.
- Retention: real-time messages that matter must also be persisted (audit/compliance), not only pushed.
- Access logs: who connected when, from where (IP), which user, close reason.
- Pen tests include WebSocket (CSWSH, IDOR on subscriptions, injection, DoS).

11.9 Anti-patterns (and what to do instead)

Anti-pattern	Why it's bad	Do instead
One WebSocket per component/screen	Many sockets, duplicates, leaks	One connection, shared client, ref-counted subs
Using WebSocket as the source of truth	Messages lost on disconnect	DB is truth; push is notification; resync
Sending huge payloads (files, big lists) over the socket	Size limits, head-of-line blocking, memory	Send ids/URLs, fetch over HTTP/CDN
Trusting `from` / `userId` in payloads	Impersonation	Principal from session
`setAllowedOriginPatterns("*")` with cookies	CSWSH	Allow-list
No authorisation on SUBSCRIBE	Data leaks (IDOR)	Per-destination ownership checks
Simple broker in a multi-instance deployment	Messages reach only some users	Relay or backplane
No heart-beats	Silent dead connections, proxy timeouts	10–25 s heart-beats
Fixed-interval reconnect for everyone	Thundering herd	Backoff + jitter
Business logic inside interceptors	Untestable, slow	Interceptors for cross-cutting only
Blocking I/O in `@MessageMapping` with default pool	Latency collapse	Size pools / async
Publishing before DB commit	Clients see rolled-back data	After-commit listeners / outbox
No message schema/versioning	Breaking old mobile apps	Envelope + additive changes + AsyncAPI
Logging full frames in prod	Token/PII leaks, cost	Redacted, sampled logs
Per-message DB lookup for authorisation on every MESSAGE	Expensive	Authorise on SUBSCRIBE, revoke on change

---

11.10 Migration guides

11.10.1 From polling to WebSocket

1. Keep the polling endpoint (fallback + resync).
2. Add WebSocket push of "changed" events; reduce polling interval to a slow safety net (e.g., 5 min) or only on reconnect.
3. Measure load reduction and latency; remove polling for supported clients later.

11.10.2 From simple broker to broker relay

1. Introduce RabbitMQ in staging with the STOMP plugin; dedicated users/vhost.
2. Switch `enableSimpleBroker` → `enableStompBrokerRelay` with the same prefixes; switch to `.` separators if using wildcards (affects client destinations, so plan client rollout).
3. Fix user destinations to auto-delete (`/exchange/amq.direct/...`) or add queue expiry policies.

4. Enable `userDestinationBroadcast` and `userRegistryBroadcast`.
5. Load test (broker connections = clients!); add broker monitoring; then scale gateway instances.

11.10.3 Removing SockJS

1. Measure the share of `HttpStream` / `HttpPoll` sessions (`WebSocketMessageBrokerStats`).
2. Add a raw WebSocket endpoint alongside SockJS (`/ws-native`), migrate clients to `brokerURL`.
3. Remove SockJS when fallback usage is negligible.

11.10.4 From legacy `stomp.js` (`Stomp.over`) to `@stomp/stompjs`

`Stomp.over(socket)` → `new Client({ webSocketFactory | brokerURL })`; `connect(headers, cb)` → `connectHeaders` + `onConnect`; `send(dest, headers, body)` → `publish({ destination, headers, body })`; `disconnect` → `deactivate`; move subscriptions into `onConnect`.

11.10.5 From `AbstractSecurityWebSocketMessageBrokerConfigurer` to `@EnableWebSocketSecurity` (mandatory for Spring Security 7, where the old class is removed)

Replace `configureInbound(MessageSecurityMetadataSourceRegistry)` rules with an `AuthorizationManager<Message<?>>` bean built from `MessageMatcherDelegatingAuthorizationManager.Builder`; handle CSRF (on by default); keep your CONNECT auth interceptor ordered before security.

11.11 Chapter summary

- Decide deliberately whether a feature needs real-time and which technology.
- Central real-time gateway + domain events is the scalable organisational pattern.
- Document destinations, schemas and auth rules (AsyncAPI); version payloads additively.
- Patterns: notification+fetch, snapshot+stream, resync, idempotent commands, conflation, outbox, presence with TTL.
- Run it like a product: SDK, runbooks, SLOs, canaries, chaos drills, security reviews.

Next: Chapter 12: Interview Questions & Answers

Chapter 12: Interview Questions & Answers, Part 1

How to use this chapter: questions are grouped by topic and tagged by level: **[E]** entry/junior, **[M]** mid-level, **[S]** senior/lead/architect. Each answer is written the way you should *say* it in an interview: direct answer first, then the "why", then a detail that shows depth. These cover the questions commonly asked at product companies, service companies, fintech, and e-commerce for Java/Spring, full-stack, front-end, and system-design rounds.

Part 1: Fundamentals, WebSocket protocol, browser API, STOMP, Spring. Part 2: Security, scaling & brokers, reliability & performance, debugging scenarios, system design, coding tasks, behavioural, rapid-fire.

A. Fundamentals & comparisons

A1. [E] What is a WebSocket? A protocol (RFC 6455) that provides a **persistent, full-duplex** communication channel over a single TCP connection. It starts as an HTTP/1.1 request with `Upgrade: websocket`; the server answers `101 Switching Protocols`, and from then on both sides exchange lightweight **frames** at any time. It's used for real-time features like chat, live dashboards, notifications, collaboration and games.

A2. [E] Why not just use HTTP for real-time? HTTP is request/response and client-initiated: the server can't push data unprompted. Emulating push with polling wastes requests (most return nothing), adds latency equal to the interval, and repeats headers each time. WebSocket keeps one connection open with 2–14 bytes of framing overhead per message and lets the server push instantly.

A3. [E] Explain short polling, long polling, SSE, and WebSocket.

- *Short polling*: client requests every N seconds. Simple, wasteful, latency up to N.
- *Long polling*: server holds the request until data arrives or a timeout, then the client re-requests. Near real-time but one full HTTP exchange per message.
- SSE: one long HTTP response streaming `text/event-stream` events, **server → client only**, text only, auto-reconnect with `Last-Event-ID` built in.
- *WebSocket*: bidirectional, binary or text, lowest overhead, but you implement reconnect/heart-beats and run stateful infrastructure.

A4. [M] When would you choose SSE over WebSocket? When data flows only server → client (notifications, feeds, progress, LLM token streaming), when you want built-in reconnect/resume, simpler infrastructure (plain HTTP, works great with HTTP/2 multiplexing), and no need for binary. Client → server actions can go over normal REST calls.

A5. [M] When should you NOT use WebSocket? Infrequent updates (polling is simpler), one-way feeds (SSE), cacheable content (HTTP/CDN), background/offline delivery on mobile (push notifications), request/response APIs (REST/gRPC), very large file transfers (HTTP with resumable uploads), or teams without capacity to operate stateful connection infrastructure.

A6. [M] Is WebSocket built on HTTP? Only the **opening handshake** is HTTP (an HTTP/1.1 GET with Upgrade). After `101`, the connection speaks the WebSocket framing protocol directly over TCP (or TLS). It's "HTTP-compatible" so it can use ports 80/443 and pass through HTTP infrastructure. Over HTTP/2 (RFC 8441) and HTTP/3 (RFC 9220), it's bootstrapped with an extended CONNECT on a stream.

A7. [E] What ports and URL schemes does WebSocket use? `ws://` (default port 80) and `wss://` (TLS, default 443). Fragments (`#`) aren't allowed in WebSocket URIs.

A8. [M] What's the difference between WebSocket and a raw TCP socket? TCP is a byte stream with no message boundaries, no browser access, no HTTP-compatible handshake. WebSocket adds a handshake through HTTP infrastructure, message framing (text/binary), masking, control frames (ping/pong/close), origin-based security, and a browser API. Browsers can't open raw TCP sockets.

A9. [M] Is WebSocket stateful or stateless? Implications? Stateful: each server holds live connections. Implications: load balancers must support upgrades and long connections; scaling out needs a broker/backplane so messages reach users on other instances; deployments disconnect clients (need reconnect + resync); memory per connection; heart-beats needed to survive idle timeouts; metrics shift to concurrent connections and message rates.

A10. [E] What is full-duplex? Both parties can send at the same time, independently. HTTP/1.1 is effectively half-duplex per exchange (request, then response).

A11. [M] Compare WebSocket with WebRTC and WebTransport. WebRTC: peer-to-peer media and data channels over UDP (SCTP/DTLS), can be unreliable/unordered, needs signalling (often done via WebSocket) and STUN/TURN. WebTransport: client-server over HTTP/3/QUIC with multiple streams and unreliable datagrams, no head-of-line blocking; newer with growing support. WebSocket: single ordered reliable TCP stream, universally supported.

A12. [M] What is Socket.IO and can a WebSocket client talk to a Socket.IO server? Socket.IO is a library with its own protocol (Engine.IO underneath) that uses WebSocket or HTTP long-polling, adding rooms, acknowledgements, reconnection. Its packets have their own framing, so a **plain WebSocket client can't talk to a Socket.IO server** (and vice versa) without the Socket.IO client.

A13. [M] Was HTTP/2 Server Push a replacement for WebSockets? No. It pushed resources into the browser cache in anticipation of requests (e.g., CSS), not application messages to JavaScript, and Chrome removed it in 2022. Real-time alternatives on HTTP/2 are SSE or WebSocket over HTTP/2 (RFC 8441).

A14. [E] What is pub/sub? Topic vs queue? Publish/subscribe decouples senders from receivers through named destinations. A **topic** delivers each message to **all** current subscribers (fan-out); a **queue** delivers each message to **one** of the consumers (competing consumers, load balancing). In STOMP, destination semantics are broker-defined: `/topic` and `/queue` are conventions.

A15. [S] How does TCP's reliability relate to message reliability in WebSocket apps? TCP guarantees ordered delivery of bytes **only while the connection lives.** If the connection drops, bytes in flight and messages published while disconnected are lost; neither side automatically knows what the other

received. Application-level reliability needs ids/sequence numbers, acks, persistence and resync after reconnect.

B. WebSocket protocol (RFC 6455)

B1. [E] Describe the WebSocket handshake. Client sends `GET /path HTTP/1.1` with `Host`, `Upgrade: websocket`, `Connection: Upgrade`, `Sec-WebSocket-Key` (base64 of 16 random bytes), `Sec-WebSocket-Version: 13`, and optionally `Origin`, `Sec-WebSocket-Protocol`, `Sec-WebSocket-Extensions`. Server answers `HTTP/1.1 101 Switching Protocols` with `Upgrade`, `Connection` and `Sec-WebSocket-Accept`, plus the selected sub-protocol/extensions. Then frames flow.

B2. [M] How is `Sec-WebSocket-Accept` computed and why? `base64(SHA-1(Sec-WebSocket-Key + "258EAFA5-E914-47DA-95CA-C5AB0DC85B11"))`. It proves the server understood the WebSocket handshake (not some HTTP server or cache blindly replying), preventing cross-protocol confusion and stale cached responses. It is **not** authentication or encryption.

B3. [M] What status code indicates a successful handshake? What if the server returns 200 or 302? `101 Switching Protocols`. Anything else is a failed handshake. Browsers don't follow redirects for WebSocket; a 200 typically means the request hit a normal page (misrouting or a proxy not forwarding Upgrade).

B4. [M] Describe the structure of a WebSocket frame. Byte 0: FIN (1 bit), RSV1–3 (3 bits), opcode (4 bits). Byte 1: MASK bit + 7-bit payload length. If length is 126, the next 2 bytes hold the length; if 127, the next 8 bytes. Then a 4-byte masking key if MASK=1, then the payload. Overhead: 2–14 bytes.

B5. [E] List the opcodes. 0x0 continuation, 0x1 text, 0x2 binary, 0x8 close, 0x9 ping, 0xA pong; 0x3–0x7 and 0xB–0xF reserved.

B6. [M] What is masking and why do only clients mask? Clients XOR the payload with a random 4-byte key per frame. It prevents attacker-controlled JavaScript from producing predictable bytes on the wire that confused intermediaries could misinterpret as HTTP requests (cache poisoning, per the 2011 Huang et al. research). The server isn't attacker-scriptable in that model, so server frames aren't masked. A server must close the connection (1002) on an unmasked client frame. Masking is not encryption: the key is in the frame.

B7. [M] What is fragmentation? Can control frames be fragmented? A message can be split into frames: first frame has the opcode and FIN=0, following frames use opcode 0 (continuation), the last has FIN=1. It allows streaming messages of unknown size and keeps the connection responsive. Control frames can't be fragmented and must be ≤125 bytes, but they **can be interleaved** between fragments of a data message.

B8. [M] What's the difference between a frame and a message? A message is the application-level unit (one text or binary payload). A frame is the wire unit. One message = one or more frames. APIs like the browser's deliver whole messages.

B9. [M] What are ping and pong frames? Control frames for keep-alive/liveness. Receiving a ping requires replying with a pong carrying the same payload. An unsolicited pong can be a one-way heart-beat. Browsers auto-reply to pings, but JS can't send pings, which is why app-level heart-beats (like STOMP's) exist.

B10. [M] Explain the closing handshake. One side sends a Close frame (optional 2-byte status code + UTF-8 reason ≤123 bytes), stops sending data, the other replies with a Close frame, and the TCP connection closes (server closes TCP first, ideally). If TCP dies without close frames, the API reports 1006.

B11. [E/M] Name common close codes. 1000 normal, 1001 going away, 1002 protocol error, 1003 unsupported data, 1005 no status (reserved), 1006 abnormal (reserved, never sent), 1007 invalid payload (e.g., bad UTF-8), 1008 policy violation, 1009 message too big, 1010 mandatory extension, 1011 internal error, 1012 service restart, 1013 try again later, 1015 TLS failure (reserved). 3000–3999 for libraries/frameworks, 4000–4999 private use.

B12. [M] What does close code 1006 mean and how do you debug it? "Abnormal closure": the connection was lost without a close frame. The code is generated locally, never sent. Causes: network loss, server crash, proxy/LB idle timeout, TLS errors, the server killing the TCP connection. Debug with timing (exactly 60 s → proxy timeout), server logs, LB logs, heart-beat configuration, mobile network transitions.

B13. [M] What's a sub-protocol? Give STOMP's. An application-level protocol negotiated in the handshake via `Sec-WebSocket-Protocol`. The client offers a list; the server selects one. STOMP: `v10.stomp`, `v11.stomp`, `v12.stomp`.

B14. [M] What is `permessage-deflate`? Trade-offs? RFC 7692 extension compressing each message with DEFLATE (RSV1 marks compressed messages). It saves bandwidth for text/JSON but costs CPU and memory per connection (sliding windows with context takeover), and mixing secrets with attacker-controlled data in the same compression context can enable CRIME/BREACH-style leaks.

B15. [M] Text vs binary frames? Text frames must contain valid UTF-8 (otherwise close 1007); binary frames can be any bytes. Binary is used for protobuf/MessagePack, images, or when bodies may contain NULL bytes.

B16. [S] How does WebSocket work over HTTP/2? HTTP/2 has no Upgrade mechanism. RFC 8441 adds `SETTINGS_ENABLE_CONNECT_PROTOCOL`; the client sends an extended CONNECT (`:method CONNECT`, `:protocol websocket`) on a new stream; the server returns `:status 200`; there's no Sec-WebSocket-Key/Accept exchange. Many WebSockets can share one TCP connection. RFC 9220 does the same for HTTP/3.

B17. [M] What is the maximum message size? The protocol allows payloads up to 2^63−1 bytes per frame. In practice servers impose limits (Tomcat 8 KB message buffer by default, Spring STOMP 64 KB, API Gateway 128 KB), and exceeding them causes close 1009. Design messages to be small.

B18. [S] Why must proxies be configured specially for WebSocket? `Upgrade` and `Connection` are **hop-by-hop** headers that proxies normally don't forward, and proxies often default to HTTP/1.0 upstream or short read timeouts. You must forward the headers with HTTP/1.1 (Nginx: `proxy_http_version 1.1`, `proxy_set_header Upgrade $http_upgrade`, `Connection "upgrade"`) and raise idle timeouts.

B19. [M] Does the Same-Origin Policy / CORS apply to WebSocket? No. Any page can open a WebSocket to any server, and the browser includes the page's `Origin` header and cookies per SameSite rules. Servers must validate `Origin` to prevent Cross-Site WebSocket Hijacking.

B20. [S] Why does the RFC require the server to close the TCP connection first? So the server ends up in TCP TIME_WAIT instead of the client, letting the client reopen a connection immediately.

B21. [M] What does "half-open connection" mean and how do you detect it? One side believes the connection is open while the other side is gone (e.g., phone lost signal, NAT entry expired). TCP doesn't notice until it sends data and retransmissions fail (can take many minutes). Detect with ping/pong or application heart-beats and a timeout.

B22. [S] Could you implement a WebSocket server by hand? What's hard? Yes: parse the HTTP upgrade, compute Accept, parse frames. Hard parts: partial TCP reads/buffering, fragmentation, masking, UTF-8 validation across fragments, control frames during fragmented messages, close handshake, backpressure, extensions, limits. In production, use a mature library.

C. Browser API & clients

C1. [E] How do you open a WebSocket in the browser? `const ws = new WebSocket('wss://host/ws', ['v12.stomp']);` then handle `open`, `message`, `error`, `close`, and send with `ws.send(data)` after `open`.

C2. [E] What are the readyState values? 0 CONNECTING, 1 OPEN, 2 CLOSING, 3 CLOSED.

C3. [M] Can you set custom headers (Authorization) on a browser WebSocket? No. The constructor only takes URL and protocols. Workarounds: cookies (with Origin checks), token in a query parameter (logged, so use short-lived tickets), the sub-protocol hack, or authenticating in the first message (STOMP CONNECT headers, the usual choice).

C4. [M] What does `bufferedAmount` tell you? Bytes queued by `send()` not yet transmitted. Use it for client-side backpressure (pause sending when high). `send()` never blocks.

C5. [M] Does the browser WebSocket reconnect automatically? No (unlike `EventSource`). Implement reconnect with exponential backoff + jitter, re-subscribe, and resync state. Libraries like stompjs (`reconnectDelay`) do the reconnect part.

C6. [M] Why is `onerror` useless for diagnostics? Browsers intentionally hide error details to prevent network probing from web pages. Use DevTools and the `close` event's code/reason.

C7. [M] What can you pass to `close()`? Code 1000 or 3000–4999 (others throw `InvalidAccessError`) and a reason ≤123 UTF-8 bytes (else `SyntaxError`).

C8. [S] How would you share one WebSocket across many tabs? SharedWorker owning the socket with `MessagePort` communication, or leader election (Web Locks/BroadcastChannel) where one tab holds the connection and rebroadcasts. Trade-off: complexity vs connection count.

C9. [M] What's "thundering herd" in reconnects and how do you prevent it? When a server restarts, all clients reconnect at the same moment and overload the remaining capacity. Prevent with exponential backoff, random jitter, rolling deploys, and server-side load shedding (close code 1013).

C10. [M] How do you handle WebSocket in React correctly? One shared client (module singleton/context), subscriptions in `useEffect` with cleanup (unsubscribe), ref-counted per destination to avoid duplicates, a ref for the latest callback to avoid re-subscribing, and functional state updates to avoid stale closures. Be aware that StrictMode double-invokes effects in development.

C11. [M] What special issues exist with STOMP on React Native? RN's WebSocket historically truncated text frames at the NULL byte that terminates STOMP frames. Fix with stompjs options `forceBinaryWSFrames: true` and `appendMissingNULLonIncoming: true`. Older engines need a `TextEncoder`

polyfill. Android emulator must use `10.0.2.2` for host localhost; Android/iOS block cleartext `ws://` by default; background apps get suspended, so you need push notifications and resync on foreground.

C12. [M] What's SockJS and is it still needed? A WebSocket emulation library/protocol that falls back to HTTP streaming/polling when WebSocket is blocked (old proxies, IE). Spring supports it via `.withSockJS()`. Today it's rarely needed: WebSocket over `wss://443` works almost everywhere. Keep it only if metrics show users on fallback transports. Fallback transports require sticky sessions.

C13. [S] What happens to a WebSocket when a laptop sleeps or a phone changes networks? The TCP connection usually dies silently (half-open). Without heart-beats the client may think it's connected for minutes. With heart-beats, the library detects missing traffic, closes, and reconnects; the app must then re-subscribe and resync missed data.

D. STOMP protocol

D1. [E] What is STOMP? Simple (or Streaming) Text Oriented Messaging Protocol: a simple, text-based, frame-oriented messaging protocol (HTTP-like frames) that defines commands such as CONNECT, SEND, SUBSCRIBE, ACK and MESSAGE, so clients and brokers in any language can interoperate. It runs over TCP or WebSocket.

D2. [E] Why use STOMP over WebSocket instead of raw WebSocket? WebSocket only moves messages; it has no notion of destinations, subscriptions, headers, acknowledgements, receipts or errors. STOMP standardises those, gives you broker interoperability (RabbitMQ, ActiveMQ), many client libraries, and lets Spring route messages to `@MessageMapping` methods. Without it you'd invent an ad-hoc protocol.

D3. [E] Describe the STOMP frame format.

```
COMMAND\n
header1:value1\n
header2:value2\n
\n
body\0
```

Command line, header lines, a blank line, an optional body, and a NULL byte terminator. In 1.2 lines can end with CRLF.

D4. [E] List client and server frames. Client: CONNECT (or STOMP), SEND, SUBSCRIBE, UNSUBSCRIBE, ACK, NACK, BEGIN, COMMIT, ABORT, DISCONNECT. Server: CONNECTED, MESSAGE, RECEIPT, ERROR.

D5. [M] What headers does CONNECT require in STOMP 1.2? `accept-version` and `host`. Optional: `login`, `passcode`, `heart-beat`. Custom headers (like `Authorization`) are allowed and are how token auth is usually done.

D6. [M] How is the version negotiated? Client lists versions in `accept-version`; server picks the highest common one and returns it in CONNECTED's `version` header. If there's none, the server sends ERROR with a `version` header listing supported versions and closes. If `accept-version` is absent, 1.0 is assumed.

D7. [M] Explain STOMP heart-beating and its negotiation. CONNECT has `heart-beat:cx,cy` (client can send every cx ms; wants to receive every cy ms); CONNECTED has `sx,sy`. Client → server interval = 0 if cx=0 or sy=0, else max(cx, sy). Server → client = 0 if sx=0 or cy=0, else max(sx, cy). A heart-beat is an EOL sent when idle; any traffic counts. If nothing arrives within the interval (plus tolerance), the peer is considered dead and the connection is closed.

D8. [M] Example: client `heart-beat:5000,10000`, server `heart-beat:20000,0`. Result? Client → server: sy=0 → **no** client heart-beats. Server → client: sx=20000, cy=10000 → max = **20000 ms**.

D9. [M] What are the ack modes? `auto` (default; message considered delivered when sent), `client` (client must ACK; **cumulative**, so acking a message acks all earlier ones on that subscription), `client-individual` (must ACK each message individually; not cumulative). Unacked messages may be redelivered if the client disconnects.

D10. [M] How do ACK/NACK work in 1.2 vs 1.1? In 1.2, the MESSAGE carries an `ack` header, and ACK/NACK send `id:<that value>`. In 1.1, ACK/NACK used `subscription` and `message-id` headers. NACK (since 1.1) tells the broker the message wasn't consumed; the broker may redeliver, dead-letter or discard it.

D11. [M] What is a receipt? Any client frame except CONNECT can include `receipt:<id>`; the server replies with `RECEIPT receipt-id:<id>` after processing it. Since STOMP is stream-based, it also implies receipt of all earlier frames. Used for confirming sends, subscriptions, and graceful DISCONNECT. It doesn't confirm delivery to subscribers.

D12. [M] What happens when the server sends ERROR? It contains a `message` header and details in the body (and `receipt-id` if related to a frame with a receipt). The server **must close the connection** right after. So ERROR is fatal and shouldn't be used for ordinary business validation errors (send those as regular MESSAGEs to a user error queue).

D13. [M] What's the purpose of the `id` header in SUBSCRIBE? Required since 1.1. It uniquely identifies the subscription within the connection, enables multiple subscriptions to the same destination, appears in MESSAGE's `subscription` header so the client can dispatch, and is used by UNSUBSCRIBE.

D14. [M] What does `content-length` do? Why is it important? It gives the body size in **bytes (octets)**. With it, the body may contain NULL bytes (binary). Without it, the body ends at the first NULL. A classic bug: computing it from string length (characters) breaks non-ASCII text.

D15. [M] How are special characters in headers handled? STOMP 1.1+ escapes them inside header names/values: a newline is written as the two characters `\n`, a colon as `\c`, a backslash as `\\`, and (1.2) a carriage return as `\r`. Any other escape (like `\t`) is a fatal protocol error. CONNECT and CONNECTED frames are not escaped (backward compatibility with 1.0). Header values aren't trimmed, so `destination: /x` has a value with a leading space.

D16. [M] What if a header is repeated? Only the first occurrence is used.

D17. [M] Which frames can have a body? Only SEND, MESSAGE and ERROR.

D18. [M] What are STOMP transactions? `BEGIN transaction:tx1`, then SEND/ACK/NACK frames with `transaction:tx1` are buffered until `COMMIT` (applied atomically) or `ABORT` (discarded). Uncommitted transactions are aborted on DISCONNECT or connection failure. Broker support varies (Spring's simple broker doesn't support them).

D19. [M] How do you disconnect gracefully? Send `DISCONNECT` with a `receipt`, wait for the RECEIPT (guarantees earlier frames were received), then close the socket.

D20. [M] What's the STOMP command used for? An alias of CONNECT (1.1+), letting servers distinguish STOMP from HTTP on the same port. Most clients still send CONNECT for compatibility with 1.0 servers.

D21. [M] What's new in STOMP 1.1 and 1.2? 1.1: heart-beating, NACK, version negotiation, virtual hosting (`host`), header escaping/UTF-8, required SUBSCRIBE `id`, `client-individual` mode, STOMP frame. 1.2: CRLF line endings, simplified ACK/NACK via `ack` / `id`, `\r` escape, clarified repeated headers, connection lingering, clarified id scopes.

D22. [M] Does STOMP define what `/topic` and `/queue` mean? No. Destinations are opaque strings interpreted by the server/broker. `/topic` (fan-out) and `/queue` (point-to-point) are common conventions. RabbitMQ also has `/exchange/...`, `/amq/queue/...`, `/temp-queue/...`. Spring's simple broker treats all destinations as pub/sub.

D23. [S] Compare STOMP with MQTT and AMQP. STOMP: text, very simple, opaque destinations, ack modes, great for browsers/Spring. MQTT: compact binary, topic hierarchy with `+` / `#` wildcards, QoS 0/1/2, retained messages, last will; ideal for IoT/mobile. AMQP 0-9-1: binary, rich routing (exchanges, bindings), confirms; back-end messaging (RabbitMQ). AMQP 1.0: standardised enterprise messaging (Artemis, Azure Service Bus).

D24. [S] What are STOMP's limitations? Text overhead, ERROR is always fatal, no built-in request/reply correlation (conventions only), destination semantics vary by broker, no replay/history, spec unchanged since 2012, and no flow control beyond broker features like prefetch.

D25. [M] How does request/reply work over STOMP? By convention: the client sends with `reply-to` (e.g., a temp queue or user queue) and a `correlation-id`; the responder sends the reply there with the same correlation id. In Spring, `@SendToUser(broadcast = false)` or `@SubscribeMapping` covers most request/reply needs.

D26. [M] Can STOMP carry binary data? Yes: include `content-length` and an appropriate `content-type`, and preferably send as binary WebSocket frames. stompjs offers `binaryBody`.

E. Spring WebSocket / STOMP

E1. [E] How do you enable STOMP over WebSocket in Spring Boot? Add `spring-boot-starter-websocket`, create a `@Configuration` with `@EnableWebSocketMessageBroker` implementing `WebSocketMessageBrokerConfigurer`, register an endpoint (`registry.addEndpoint("/ws")`), set application prefixes (`/app`) and enable a broker (`enableSimpleBroker("/topic", "/queue")` or a relay).

E2. [E] What do `setApplicationDestinationPrefixes` and `enableSimpleBroker` do? Messages whose destination starts with the application prefix (e.g., `/app`) are routed to `@MessageMapping` methods. Messages and subscriptions with broker prefixes (e.g., `/topic`, `/queue`) go to the broker, which fans them out to subscribers.

E3. [M] Explain the three channels in Spring's STOMP support. `clientInboundChannel` carries messages from WebSocket clients; `clientOutboundChannel` carries messages to clients; `brokerChannel` carries messages from server-side application code to the broker. Inbound/outbound are backed by

thread pools (2 × CPUs core size by default). Interceptors can be attached to them.

E4. [M] Walk through what happens when a client SENDs to `/app/chat`. `SubProtocolWebSocketHandler` receives the WebSocket message → `StompSubProtocolHandler` decodes it into a `Message` → `clientInboundChannel` (interceptors run) → `SimpAnnotationMethodMessageHandler` matches `@MessageMapping("/chat")` → payload converted (Jackson) → method invoked → return value sent to `brokerChannel` (to `@SendTo` or default `/topic/chat`) → broker finds subscribers → `clientOutboundChannel` → encoded STOMP MESSAGE → WebSocket.

E5. [M] What's the default destination of a `@MessageMapping` return value? The input destination with the application prefix replaced by `/topic`, e.g., `/app/greeting` → `/topic/greeting`. Override with `@SendTo` or `@SendToUser`.

E6. [M] `@SendTo` vs `@SendToUser`? `@SendTo` sends to a broker destination (all subscribers). `@SendToUser` sends to the current user's user destination (`/user/queue/...`): all their sessions by default, or only the originating session with `broadcast = false`. Without an authenticated user, `@SendToUser` falls back to the current session.

E7. [M] What's `@SubscribeMapping` used for? Handling SUBSCRIBE frames to application destinations (e.g., `/app/rooms`). The return value is sent **directly back to that client** via the outbound channel, not through the broker, which makes it ideal for loading initial data when a UI component subscribes.

E8. [M] How do you send a message from a REST controller or scheduled job? Inject `SimpMessagingTemplate` (or `SimpMessageSendingOperations`) and call `convertAndSend("/topic/x", payload)` or `convertAndSendToUser(username, "/queue/x", payload)`.

E9. [M] Can `SimpMessagingTemplate.convertAndSend("/app/x")` trigger a `@MessageMapping`? No. The template sends to the `brokerChannel`; only client messages arriving on the inbound channel are routed to annotated handlers. Call the service directly instead.

E10. [M] Explain user destinations. Clients subscribe to `/user/queue/x`; Spring's `UserDestinationMessageHandler` translates it to a session-unique destination like `/queue/x-user{sessionId}`. Server code calls `convertAndSendToUser("anna", "/queue/x", payload)` → `/user/anna/queue/x` → resolved via `SimpUserRegistry` to all of anna's sessions. Requires a Principal whose `getName()` matches.

E11. [S] Why might `convertAndSendToUser` not deliver? Principal name mismatch (e.g., email vs id vs JWT `sub`), no Principal set on the session, client subscribed to `/queue/x` instead of `/user/queue/x`, user connected to another instance (need relay with user destination broadcast), wrong user prefix configuration, or the broker prefix (`/queue`) not enabled.

E12. [M] How do you get the current user in a `@MessageMapping` method? Add a `java.security.Principal` parameter (or `@AuthenticationPrincipal` with Spring Security's argument resolver), or read `SimpMessageHeaderAccessor#getUser()`.

E13. [M] How do you access the session id and session attributes? `SimpMessageHeaderAccessor#getSessionId()` / `@Header("simpSessionId")`, and `getSessionAttributes()` for the per-session map (populated by handshake interceptors like `HttpSessionHandshakeInterceptor` or your code).

E14. [M] `HandshakeInterceptor` vs `ChannelInterceptor`? `HandshakeInterceptor` runs once during the HTTP upgrade (can reject with an HTTP status, copy attributes, inspect HTTP headers/cookies). `ChannelInterceptor` runs for every STOMP message on a channel (`preSend`, `postSend`, `afterSendCompletion`) and is used for CONNECT authentication, authorisation, logging, rate limiting.

E15. [M] Why must you use `MessageHeaderAccessor.getAccessor(message, StompHeaderAccessor.class)` in an interceptor rather than `StompHeaderAccessor.wrap(message)`? `getAccessor` returns the message's own mutable accessor, so `setUser(...)` changes the actual message/session. `wrap` creates a copy, so changes are lost unless you build and return a new message.

E16. [M] How do you handle exceptions in message handlers? `@MessageExceptionHandler` methods in the controller or a `@ControllerAdvice` (supported since 5.3), typically combined with `@SendToUser("/queue/errors")` to report errors to the client. Unhandled exceptions are only logged, so the client receives nothing. Exceptions in inbound interceptors become STOMP ERROR frames (customisable via `StompSubProtocolErrorHandler`).

E17. [M] How do you validate STOMP payloads? Annotate the payload `@Valid` / `@Validated` with Bean Validation on the classpath (or provide a validator via `getValidator()`), and handle `org.springframework.messaging.handler.annotation.support.MethodArgumentNotValidException`.

E18. [M] How do you configure heart-beats with the simple broker? `registry.enableSimpleBroker("/topic","/queue").setTaskScheduler(scheduler).setHeartbeatValue(new long[]{10000,10000})`. Without a TaskScheduler, the simple broker doesn't do heart-beats (it answers `0,0`).

E19. [M] What transport limits does Spring have and what are the defaults? `WebSocketTransportRegistration`: `messageSizeLimit` 64 KB, `sendBufferSizeLimit` 512 KB, `sendTimeLimit` 10 s, `timeToFirstMessage` 60 s. Plus container limits (e.g., Tomcat 8 KB message buffers via `ServletServerContainerFactoryBean`).

E20. [S] What happens when a client is too slow to receive messages? Spring's `ConcurrentWebSocketSessionDecorator` buffers outbound messages per session; if the buffer exceeds `sendBufferSizeLimit` or a send takes longer than `sendTimeLimit`, the session is closed (status 4500 "session not reliable"). This protects server memory. The client reconnects and resyncs.

E21. [S] Is message order guaranteed in Spring STOMP? Not by default, because the inbound and outbound channels use thread pools. Enable `MessageBrokerRegistry#setPreservePublishOrder(true)` (5.1+) for outbound per-session order and `StompEndpointRegistry#setPreserveReceiveOrder(true)` (6.1+) for inbound per-session order; or use sequence numbers.

E22. [M] What session events does Spring publish? `SessionConnectEvent`, `SessionConnectedEvent`, `SessionSubscribeEvent`, `SessionUnsubscribeEvent`, `SessionDisconnectEvent` (may fire more than once per session, so handlers must be idempotent), and `BrokerAvailabilityEvent`.

E23. [M] How would you track online users? Listen to connected/disconnect events (count sessions per user, since a user can have many tabs), or query `SimpUserRegistry`. In a cluster use the multi-server user registry (relay `userRegistryBroadcast`) or Redis with TTLs. Publish presence changes to a topic.

E24. [M] Simple broker vs STOMP broker relay? Simple broker: in-memory, no setup, single instance only, subset of STOMP (no acks/transactions/limited receipts). Relay: forwards to RabbitMQ/ActiveMQ over TCP, supports clustering and full broker features, needs Reactor Netty, opens one TCP connection per client session plus a system connection.

E25. [M] What are `clientLogin` / `systemLogin` in the relay? `clientLogin/Passcode`: credentials the relay uses for each client session's TCP connection to the broker. `systemLogin/Passcode`: credentials for the shared system connection used to send messages from application code. End-user identity is not passed to the broker; Spring is the security boundary.

E26. [M] How do you use dot-separated destinations? `registry.setPathMatcher(new AntPathMatcher("."))` so `@MessageMapping("chat.room.{id}")` matches `/app/chat.room.42`. Useful with RabbitMQ, whose topic routing keys use dots.

E27. [M] How do you add SockJS fallback? `registry.addEndpoint("/ws").withSockJS()`. Clients use `new SockJS('https://host/ws')` (via stompjs `webSocketFactory`). Native WebSocket clients then use `/ws/websocket`. Fallback transports need sticky sessions in clusters.

E28. [M] How do you configure allowed origins? `addEndpoint("/ws").setAllowedOrigins("https://app.example.com")` or `setAllowedOriginPatterns("https://*.example.com")`. Default is same origin only. Avoid `*` in production, especially with cookie auth.

E29. [S] How do you size Spring's channel thread pools? Inbound threads run your handlers: for blocking I/O handlers increase core size or make handlers async; the queue is unbounded by default, so with `ThreadPoolTaskExecutor` `maxPoolSize` only matters if you bound the queue. Outbound rarely needs tuning except with many slow clients. Monitor active threads and queue sizes via `WebSocketMessageBrokerStats` or Micrometer.

E30. [M] What does `WebSocketMessageBrokerStats` show? Periodic (30 min default) INFO logs with current WebSocket sessions by transport, total sessions, abnormal closes (connect failures, send limit, transport errors), STOMP frames processed (CONNECT/CONNECTED/DISCONNECT), relay status, and inbound/outbound/SockJS thread pool stats.

E31. [M] How do you return a value asynchronously from a `@MessageMapping`? Return a `CompletableFuture` / `ListenableFuture` (or reactive types); Spring sends the result when it completes, freeing the inbound thread.

E32. [M] How do you customise JSON conversion? Spring Boot uses its auto-configured `ObjectMapper` for messaging. For custom converters, override `configureMessageConverters` (return true to keep defaults). Clients should send `content-type:application/json`.

E33. [S] How do you send to a specific session (not a user)? Create headers with `SimpMessageHeaderAccessor.create(SimpMessageType.MESSAGE)`, `setSessionId(id)`, `setLeaveMutable(true)`, and call `convertAndSendToUser(sessionId, "/queue/x", payload, accessor.getMessageHeaders())`. The client subscribes to `/user/queue/x`.

E34. [M] How do you write an integration test for a STOMP endpoint? `@SpringBootTest(webEnvironment = RANDOM_PORT)`, a `WebSocketStompClient` with a JSON converter, `connectAsync` to `ws://localhost:{port}/ws` with auth headers, subscribe with a `StompFrameHandler` that pushes to a `BlockingQueue`, send, and `poll` with a timeout. Avoid subscribe/send races with receipts or a round-trip.

E35. [M] Is Spring's STOMP support available in WebFlux? No built-in STOMP broker/annotation support in WebFlux. WebFlux has a reactive `WebSocketHandler` for raw messages; for higher-level reactive messaging consider RSocket.

E36. [S] Explain `SubProtocolWebSocketHandler`. The `WebSocketHandler` registered for STOMP endpoints. It delegates to sub-protocol handlers by negotiated `Sec-WebSocket-Protocol` (STOMP by default), manages session lifecycle, wraps sessions in `ConcurrentWebSocketSessionDecorator` with buffer/time limits, checks `timeToFirstMessage`, and closes sessions on shutdown.

E37. [M] What's `SimpMessageType`? The Spring-level message type in headers: CONNECT, CONNECT_ACK, MESSAGE, SUBSCRIBE, UNSUBSCRIBE, HEARTBEAT, DISCONNECT, DISCONNECT_ACK, OTHER. Security matchers (e.g., `simpTypeMatchers`) use it.

E38. [M] How do you access a custom STOMP header the client sent with SEND? `@Header("myHeader") String v` (Spring checks top-level then native headers) or `StompHeaderAccessor#getFirstNativeHeader("myHeader")`.

E39. [M] Can you use `@PreAuthorize` on `@MessageMapping` methods? Yes, with method security enabled; Spring Security's `SecurityContextChannelInterceptor` populates the `SecurityContext` from the message's user during handling.

E40. [S] What happens on application shutdown? The WebSocket infrastructure stops and `SubProtocolWebSocketHandler` closes open sessions (status 1001 Going Away). Clients reconnect to other instances. Combine with readiness probes, `preStop` delays and jittered reconnects for smooth rollouts.

E41. [M] How do you raise the max message size for big payloads? `registration.setMessageSizeLimit(...)` in `configureWebSocketTransport`, and container buffers via `ServletServerContainerFactoryBean#setMaxTextMessageBufferSize`. But prefer sending references and fetching large data over HTTP.

E42. [S] How does the selector header work in the simple broker and why is it sensitive? A SUBSCRIBE `selector` header holds a SpEL expression evaluated against message headers to filter deliveries. It caused CVE-2018-1270/1275 (SpEL injection → RCE in old versions, fixed by using `SimpleEvaluationContext`). Configure the header name with `setSelectorHeaderName`, or disable it (null) if unused.

E43. [M] What's `@DestinationVariable`? Extracts template variables from the message destination, e.g., `@MessageMapping("/rooms/{roomId}")` → `@DestinationVariable String roomId`. Validate it, since it's client input.

E44. [M] Where does the Principal of a WebSocket session come from? From the HTTP handshake request (`request.getPrincipal()` via Spring Security / `DefaultHandshakeHandler#determineUser`), or set on the STOMP CONNECT message by an interceptor (`accessor.setUser`). Spring then associates it with the session for subsequent messages.

E45. [M] What is `SimpUserRegistry`? A registry of connected users, their sessions and subscriptions (`getUser`, `getUsers`, `getUserCount`, `findSubscriptions`). Default local implementation; a multi-server view is available with the relay's user registry broadcast.

Continue to Part 2.

CHAPTER 12B

Chapter 12: Interview Questions & Answers, Part 2

Levels: **[E]** entry, **[M]** mid, **[S]** senior/lead. Part 1 covered fundamentals, the WebSocket protocol, the browser API, STOMP and Spring.

F. Security

F1. [E] How do you secure a WebSocket connection? Use `wss://` (TLS), validate the `Origin` header, authenticate (handshake session or token in STOMP CONNECT), authorise every SUBSCRIBE and SEND, validate input, limit sizes/rates/connections, and avoid leaking tokens in URLs/logs.

F2. [M] What is Cross-Site WebSocket Hijacking (CSWSH)? A malicious site opens a WebSocket to your server from the victim's browser; since CORS doesn't apply to WebSockets and the browser may attach cookies, a cookie-authenticated socket gets established on the attacker's behalf, letting them read/send messages as the victim. Prevent it with Origin allow-lists, token-based auth in the CONNECT frame (not cookies alone), SameSite cookies, and CSRF tokens on CONNECT (Spring Security's default).

F3. [M] How do you authenticate STOMP clients with JWT in Spring? Client sends `Authorization: Bearer <jwt>` as a CONNECT header. A `ChannelInterceptor` on `clientInboundChannel` checks `StompCommand.CONNECT`, decodes/validates the JWT (`JwtDecoder`), converts it to an `Authentication`, and calls `accessor.setUser(auth)` using the accessor from `MessageHeaderAccessor.getAccessor`. The config class gets `@Order(Ordered.HIGHEST_PRECEDENCE + 99)` so it runs before Spring Security's message authorisation. Spring keeps the user for the session.

F4. [M] Why not put the token in the WebSocket URL? URLs are logged by proxies, load balancers, servers and may stay in browser history; leaked tokens are replayable. If you must (e.g., gateway auth), use short-lived, single-use tickets.

F5. [M] How do you authorise subscriptions? With Spring Security message rules (`simpSubscribeDestMatchers(...)`) for coarse checks, plus a custom `ChannelInterceptor` on SUBSCRIBE that parses the destination (e.g., `/topic/orders.{id}`) and verifies ownership/membership. Deny by default, block wildcard subscriptions, and handle revocation (close sessions or stop delivery when membership changes).

F6. [M] How would you stop clients from publishing directly to `/topic/...`? Allow client SEND only to `/app/**` (Spring Security `simpDestMatchers("/app/**")...` + `simpTypeMatchers(MESSAGE, SUBSCRIBE).denyAll()` + `anyMessage().denyAll()`), so all broadcasts come from server code after validation.

F7. [M] How do you deal with token expiry for long-lived connections? Store expiry in session attributes at CONNECT; reject inbound messages after expiry; close sessions server-side when the token expires or is revoked (track `WebSocketSession`s via a decorator factory); enforce a maximum connection

age; client refreshes tokens in `beforeConnect` and reconnects.

F8. [M] What is CSRF protection in Spring WebSocket security? With `@EnableWebSocketSecurity`, Spring Security requires a valid CSRF token in the CONNECT frame headers by default, protecting cookie-authenticated sessions against CSWSH. Clients obtain the token from the server and add it to `connectHeaders`. For pure header-token auth you can configure the security interceptors manually without the CSRF interceptor.

F9. [M] Should you trust the `sender` field in a chat message payload? Never. Set identity on the server from the authenticated Principal; ignore or overwrite client-supplied identity fields.

F10. [S] What DoS vectors exist for WebSocket servers and how do you mitigate them? Connection floods (LB/IP limits, per-user session caps), idle unauthenticated sockets (`timeToFirstMessage`, auth deadline), huge frames (`messageSizeLimit`, container buffers), message floods (per-session rate limits), subscription floods (caps, no wildcards), slow readers (send buffer/time limits), expensive handlers (bounded pools, timeouts), reconnect storms (client jitter, 1013 close code).

F11. [M] How do you prevent XSS through chat messages? Treat message content as untrusted: render as text (React escapes by default), sanitise any HTML (DOMPurify), validate length/charset on the server, and use a Content Security Policy.

F12. [S] What known vulnerabilities affected Spring's STOMP support? CVE-2018-1270 (and CVE-2018-1275 for an incomplete 4.3.x fix): the simple broker evaluated the SUBSCRIBE `selector` header as SpEL with a full evaluation context, allowing remote code execution. Fixed by using `SimpleEvaluationContext` (5.0.5 / 4.3.16+). Lesson: patch dependencies and disable unused features.

F13. [S] Is checking the Origin header enough? No. It stops browser-based CSWSH but not non-browser clients (who can forge Origin). You still need real authentication and authorisation. Defence in depth: Origin + token auth + per-destination authorisation + rate limits.

F14. [M] Does the broker relay pass the end user's identity to RabbitMQ? No. The relay uses configured `clientLogin` / `systemLogin` credentials. The Spring application must enforce user-level security before forwarding frames. Never expose the broker's STOMP port publicly.

F15. [S] How do you isolate tenants in a multi-tenant real-time system? Derive tenant from the authenticated token (not payload), include tenant id in destination naming (`/topic/t.{tenant}.…`), authorise tenant on every SUBSCRIBE/SEND, separate vhosts/brokers for strict isolation, per-tenant rate limits and metrics.

F16. [M] What should never appear in logs for a STOMP system? CONNECT `Authorization` / `passcode` headers, cookies, full message bodies with PII. Disable verbose client debug logging in production.

G. Brokers, clustering & scaling

G1. [E] Why doesn't the simple broker work with multiple server instances? Each instance has its own in-memory subscription registry; a message published on instance A only reaches clients connected to A. Use an external broker relay or a pub/sub backplane.

G2. [M] How does the STOMP broker relay work? Spring opens a TCP connection to the broker for each client session (forwarding the client's frames for broker destinations and returning MESSAGE frames) and one shared "system" connection for messages sent by server code. It requires Reactor Netty

and publishes `BrokerAvailabilityEvent` on connectivity changes.

G3. [M] Which brokers support STOMP? RabbitMQ (STOMP plugin, port 61613; Web STOMP plugin, port 15674 `/ws`), ActiveMQ Classic, ActiveMQ Artemis, Apache Apollo (retired), plus others. Amazon MQ offers managed ActiveMQ/RabbitMQ.

G4. [M] Explain RabbitMQ STOMP destinations. `/exchange/<name>[/<pattern>]` (publish to/bind from an exchange), `/queue/<name>` (queue created if needed; competing consumers), `/amq/queue/<name>` (existing queue), `/topic/<routing-key>` (via `amq.topic`, wildcards `*` and `#` with dot-separated words), `/temp-queue/<name>` (temporary reply queues).

G5. [M] How do user destinations work across a cluster? Configure the relay with `userDestinationBroadcast` (unresolved user messages are broadcast so the instance holding the user's session delivers them) and `userRegistryBroadcast` (instances share their user registries → multi-server `SimpUserRegistry`).

G6. [S] The relay opens one TCP connection per client. What if you have 500k users? That's 500k broker connections: heavy for any broker. Alternatives: backplane pattern (simple broker per node + Redis/Kafka fan-out between nodes), shard users across broker clusters, or a dedicated real-time gateway that consumes from Kafka with a small number of connections and fans out locally.

G7. [M] Describe a Redis-based backplane. Each instance runs a simple broker for its own clients and subscribes to a Redis pub/sub channel. To broadcast, an instance publishes to Redis; every instance receives it and calls `SimpMessagingTemplate` locally. Redis pub/sub is at-most-once (missed during disconnects); use Redis Streams/Kafka for replay; shard channels for scale.

G8. [M] Do WebSockets need sticky sessions? Not for pure WebSocket, because a connection stays on the backend it was established with. Sticky sessions are needed for SockJS HTTP fallback transports (many HTTP requests per logical session) and helpful if you keep server-side state that survives reconnects.

G9. [M] How do you configure Nginx for WebSocket? `proxy_http_version 1.1; proxy_set_header Upgrade $http_upgrade; proxy_set_header Connection $connection_upgrade;` (with a `map` on `$http_upgrade`), long `proxy_read_timeout` / `proxy_send_timeout`, `proxy_buffering off`, forward Host/X-Forwarded headers.

G10. [M] Why do connections drop every 60 seconds behind a load balancer? The LB/proxy idle timeout (60 s default for Nginx and AWS ALB) closes connections without traffic. Fix with heart-beats shorter than the timeout and/or longer timeouts.

G11. [S] How do you autoscale WebSocket servers? CPU is a poor signal (idle connections cost memory). Scale on connections per pod, memory, or message throughput via custom metrics. Since existing connections don't rebalance to new pods, use least-connection balancing and optionally a max connection age with jitter to gradually rebalance.

G12. [S] How do you deploy WebSocket servers without disrupting users? Rolling deploys with readiness gating and `preStop` delays (stop new connections, then close existing ones with 1001), clients reconnect with jitter and resync, keep payloads backward compatible across versions, add capacity headroom, canary new versions and watch close codes/connection success rates.

G13. [M] What OS limits matter for many connections? File descriptors (`ulimit -n`), ephemeral port ranges for proxy→backend connections, `somaxconn`/SYN backlog for reconnect bursts, memory for socket buffers and TLS, container/pod limits, Tomcat `max-connections` (8192 default in Spring Boot).

G14. [S] Estimate capacity for a broadcast-heavy system. Deliveries/s = publishes/s × subscribers per message. E.g., 50 msgs/s to a 20k-subscriber topic = 1M deliveries/s → network bandwidth (1M × message size) and outbound processing dominate. Mitigate with conflation, batching, smaller payloads, sharding audiences, and more nodes.

G15. [M] How does RabbitMQ prefetch relate to STOMP? The SUBSCRIBE `prefetch-count` header limits unacknowledged messages delivered to a consumer when using `client`/`client-individual` ack, giving backpressure and fair dispatch among competing consumers.

G16. [S] How would you avoid per-session queue leaks with RabbitMQ user destinations? Use destinations RabbitMQ creates as auto-delete (e.g., `/exchange/amq.direct/<name>` for user destinations, per Spring's docs) or apply policies with `x-expires`; monitor queue counts.

G17. [M] Can browsers connect directly to RabbitMQ Web STOMP? Yes, but you lose Spring's application layer (`@MessageMapping`, user destinations, custom authorisation), expose the broker publicly, and must manage broker-level auth (e.g., OAuth2 plugin) and permissions per user. Most enterprises keep a gateway in front.

G18. [S] What changes when moving from a single node to multi-region? Users connect to the nearest region; cross-region messages need replication (federation/shovel in RabbitMQ, Kafka MirrorMaker, or a global pub/sub); latency and ordering across regions; data residency; presence aggregation; failover strategies (DNS/GSLB) with client reconnect.

H. Reliability & performance

H1. [M] What delivery guarantee does STOMP over WebSocket give? By default effectively at-most-once: messages are lost if the client is disconnected or the simple broker restarts. At-least-once requires a durable broker queue, client acks and redelivery, and idempotent consumers; "exactly-once" is achieved as at-least-once + deduplication.

H2. [S] How do you make sure clients don't miss updates during disconnects? Persist state/events in a database with per-entity sequence numbers; clients track the last seq; on reconnect or gap detection they fetch missed events (REST `?after=seq`) or a snapshot; WebSocket messages act as notifications. Subscribe before fetching the snapshot and merge by seq.

H3. [M] How do you handle duplicate messages? Give each event a unique id and/or per-entity sequence; consumers ignore ids already processed or seqs ≤ last applied. For commands, the client sends an idempotency key (`clientRequestId`) and the server dedupes.

H4. [S] How do you avoid publishing events for transactions that roll back? Publish after commit (`@TransactionalEventListener(phase = AFTER_COMMIT)`) or use the transactional outbox pattern with a relay/CDC for at-least-once publication.

H5. [M] What's backpressure in WebSocket systems? Mechanisms to cope when a receiver can't keep up. Browser `send()` doesn't block (watch `bufferedAmount`); servers buffer per session. Spring bounds buffers (`sendBufferSizeLimit`, `sendTimeLimit`) and closes slow sessions. Application strategies:

conflation, batching, deltas, dropping non-critical updates, broker prefetch.

H6. [M] What is conflation? Sending only the latest value per key at a fixed interval instead of every intermediate update (e.g., price ticks), reducing load and helping slow clients.

H7. [M] How do you choose heart-beat intervals? Shorter than the smallest idle timeout in the path (proxies, LBs, NAT), typically 10–25 s; balance faster failure detection against traffic/battery cost; mobile may need shorter intervals due to NAT timeouts.

H8. [S] Why might `maxPoolSize` have no effect on Spring's inbound channel executor? `ThreadPoolTaskExecutor` creates threads beyond `corePoolSize` only when the queue is full; the default queue is unbounded, so the pool never grows. Increase `corePoolSize` or bound the queue (with a rejection strategy).

H9. [M] What metrics would you monitor? Active sessions, connects/disconnects per second by close code, CONNECT failures, inbound/outbound message rates and sizes, executor active threads/queue sizes, send-limit closes, handler latency, end-to-end latency, broker health (connections, queue depth, unacked, memory), relay availability.

H10. [M] How would you load test a STOMP server? k6/Gatling/Artillery/JMeter sending raw STOMP frames over WebSocket, or Java clients with `WebSocketStompClient`. Simulate realistic idle/active mixes, subscriptions, fan-out, reconnect storms, slow clients; run soak tests for leaks; distribute load generators to avoid client-side port/fd limits; measure p95/p99 latency.

H11. [S] JSON vs binary payloads: when to switch? When message rate × size makes bandwidth/CPU significant (market data, telemetry, games). Protobuf/MessagePack reduce size and parse time; send as binary frames with an appropriate content-type and a custom MessageConverter. Otherwise JSON's debuggability wins.

H12. [M] Does enabling compression always help? No. It helps large text payloads and bandwidth-limited clients but costs CPU and per-connection memory (significant at 100k+ connections) and can introduce side-channel risk. Measure.

H13. [S] How do you ensure ordering for a chat room across a cluster? Assign a per-room sequence number at write time (DB sequence/row version, or a single partition per room in Kafka); deliver in that order (partition by room); clients reorder/dedupe by seq and resync on gaps. Enable Spring's publish ordering per session.

H14. [M] How do you build presence ("online" status) reliably? Track sessions per user (multiple tabs/devices); store in Redis with TTL refreshed periodically or on activity; mark offline when the last session disconnects or TTL expires (handles crashes where disconnect events never fire); publish only changes; limit presence fan-out to relevant users.

I. Debugging & scenario questions

I1. [M] Users report messages stop arriving after about a minute of inactivity. What's going on? Almost certainly an idle timeout (proxy/LB, e.g., 60 s) killing silent connections. Check close codes (1006 at ~60 s), heart-beat negotiation (maybe `0,0` because the simple broker has no TaskScheduler), and proxy timeouts. Fix: enable heart-beats (10–25 s), raise timeouts, and make sure clients reconnect and resync.

I2. [M] It works locally but in production the handshake fails with 400/200. Why? The reverse proxy/ingress isn't forwarding `Upgrade` / `Connection` or uses HTTP/1.0 upstream, or the SPA's fallback route returns `index.html` (200) for `/ws`. Fix proxy config and routing order.

I3. [M] After deploying a second instance, some users miss messages. Why? The simple broker is per-instance. Users connected to instance B don't get messages published on instance A. Use a broker relay or backplane; for user destinations, enable user destination broadcast.

I4. [M] `convertAndSendToUser` works for some users but not others. Check that `Principal#getName()` matches the name passed (e.g., some users' principal is email, others id), that those users have a Principal at all, that they subscribed to `/user/queue/...`, and whether they're connected to another instance.

I5. [M] The React app shows every message twice. Duplicate subscriptions: multiple client instances, subscribing on every render, subscribing both in `onConnect` and in components, StrictMode double effects without cleanup, or reconnect without unsubscribing old handlers. Use a singleton client with ref-counted subscriptions and cleanup.

I6. [M] Mobile (React Native) app connects but never gets CONNECTED. NULL-terminator truncation in RN's WebSocket. Set `forceBinaryWSFrames` and `appendMissingNULLonIncoming`. Also check `TextEncoder` availability, cleartext restrictions and host addresses.

I7. [S] Server memory keeps growing during peak hours and the app eventually crashes. Suspect slow clients buffering outbound messages (check send limits), unbounded inbound queue due to blocking handlers, maps keyed by session not cleaned on disconnect, or subscriptions never unsubscribed. Take heap dumps, check `WebSocketMessageBrokerStats` queue sizes, configure send limits, bound queues, conflate high-frequency topics.

I8. [M] Clients get an ERROR frame immediately after CONNECT. An inbound interceptor threw: invalid/expired token, missing CSRF token with `@EnableWebSocketSecurity`, interceptor ordering (Spring Security authorisation running before your auth interceptor), or unauthorised destination. Read the ERROR `message` header; check server logs.

I9. [S] After a deploy, the system collapses under reconnects. Thundering herd: all clients reconnect simultaneously with a fixed delay, overwhelming remaining instances, auth services (JWKS/IdP) and snapshot REST endpoints. Mitigate with exponential backoff + jitter, rolling deploys, capacity headroom, caching of snapshots, rate limiting reconnects, 1013 "try again later".

I10. [M] Messages arrive out of order in the UI. Thread-pool-based channels can reorder. Enable `setPreservePublishOrder(true)` / `setPreserveReceiveOrder(true)` or include sequence numbers and reorder/drop stale events client-side.

I11. [M] A big JSON message closes the connection with 1009. It exceeds the container buffer (Tomcat 8 KB default per message) or Spring's `messageSizeLimit`. Raise limits deliberately or redesign to send references.

I12. [S] Only one corporate customer can't connect. Their proxy/firewall/TLS inspection blocks or breaks WebSocket. Confirm via their close codes and network logs; ensure `wss://` on 443; offer SockJS fallback or polling; check their Origin (embedded portals) and IP allow-lists.

I13. [M] `@MessageMapping` method isn't invoked even though the client sends. Destination prefix mismatch (`/chat` vs `/app/chat`), path separator mismatch (dots vs slashes without `AntPathMatcher(".")`), controller not scanned, or security rules silently dropping the message.

I14. [M] User removed from a project still receives its updates. Authorisation happened only at SUBSCRIBE. Implement revocation: on membership change, close the user's sessions (forcing re-subscribe with checks) or route updates via per-user delivery that re-checks membership.

J. System design questions (with answer outlines)

J1. [S] Design a chat system like WhatsApp/Slack. Requirements: 1:1 and group chat, delivery/read receipts, presence, typing, history, offline delivery, multi-device.

- *Connection layer*: real-time gateway (STOMP over WebSocket), JWT on CONNECT, heart-beats, LB with long timeouts.
- *Write path*: client sends `/app/chat.send` with `clientMsgId` → message service validates membership → persists with per-conversation `seq` (DB/Cassandra) → event to Kafka (partition by conversationId).
- *Fan-out*: gateway consumers deliver to `/topic/conv.{id}` or per-user queues; offline recipients get push notifications.
- *Reliability*: dedupe by `clientMsgId` , ordered by `seq` , resync history via REST on reconnect, delivery receipts as batched "delivered/read up to seq".
- *Presence/typing*: Redis TTL, ephemeral, rate-limited.
- *Scale*: shard gateways, Kafka partitions, conflation of typing/presence, hot group handling (large channels → pull model).
- *Security*: membership checks on subscribe/send, E2E encryption discussion, XSS, rate limits.

J2. [S] Design a real-time notification service for 10M users. Producers emit domain events → notification service applies preferences → persists to an inbox (for unread counts/history) → pushes in-app via gateway user destinations if online (presence lookup) else mobile push/email. Gateways scale horizontally with a backplane (Kafka/Redis keyed by userId → gateway node mapping in Redis to avoid broadcasting to all nodes). Clients on connect fetch unread count via REST.

J3. [S] Design a live sports score / stock ticker for millions of viewers. Huge fan-out, read-only, latest-value semantics. Use conflation at the source (e.g., 1–4 updates/s), a hierarchical fan-out (origin → regional relays/edges → gateways), small binary/delta payloads, CDN-backed snapshots for initial state, SSE or WebSocket depending on the need for client→server; drop intermediate updates for slow clients; no per-user state.

J4. [S] Design collaborative document editing. Use OT or CRDTs (Yjs/Automerge) for merging; WebSocket for low-latency sync of operations; server persists operation log/snapshots; awareness (cursors) as ephemeral, conflated messages; per-document routing (all collaborators of a document to the same node or a document actor); offline edits merged via CRDT on reconnect. STOMP can transport updates but many CRDT libraries ship their own protocols.

J5. [S] Design live order tracking for a food delivery app. Driver location updates via app (HTTP or WS) every few seconds → location service (Redis GEO, latest-only) → publish to `/topic/order.{id}.location` (authorised to customer/restaurant). Order status changes from order service events.

Conflation, battery-friendly intervals, push notifications for status changes when app is backgrounded, resync on foreground.

J6. [S] Design an online auction/bidding system. Bids via REST/command with idempotency key; strong consistency for the highest bid (single writer per auction: DB row lock or partitioned actor); broadcast new highest bid with `seq` to `/topic/auction.{id}`; clients show server-confirmed state; end-of-auction timing on the server; anti-sniping extension rules; rate limits; audit log.

J7. [S] Your simple-broker Spring app must scale from 1 to 20 instances. Plan the migration. Choose relay (RabbitMQ) vs backplane (Redis/Kafka) based on connection counts and features; switch destinations/naming (dots); configure user destination & registry broadcast or redesign presence in Redis; LB/proxy timeouts; remove local in-memory state; load test; staged rollout with metrics.

K. Coding / whiteboard tasks

K1. [M] Write the Spring config for a STOMP endpoint with a simple broker and heart-beats. See Chapter 5 §5.8. Key lines: `@EnableWebSocketMessageBroker`, `addEndpoint("/ws").setAllowedOriginPatterns(...)`, `setApplicationDestinationPrefixes("/app")`, `enableSimpleBroker("/topic","/queue").setTaskScheduler(ts).setHeartbeatValue(new long[]{10000,10000})`.

K2. [M] Write a CONNECT-frame JWT interceptor. See Chapter 8 §8.5.2. Mention `getAccessor`, `StompCommand.CONNECT`, `getFirstNativeHeader("Authorization")`, `setUser`, exception → ERROR, `@Order(HIGHEST_PRECEDENCE + 99)`.

K3. [M] Write a React hook that subscribes to a destination and cleans up. See Chapter 6 §6.6. Mention singleton client, `useEffect` cleanup, ref for callback, re-subscription on reconnect.

K4. [M] Write the STOMP frames to connect, subscribe and send.

```
CONNECT
accept-version:1.2
host:example.com
heart-beat:10000,10000

^@
SUBSCRIBE
id:sub-0
destination:/topic/news

^@
SEND
destination:/app/news.post
content-type:application/json
content-length:15

{"title":"Hi!"}^@
```

(`content-length` = 15 bytes for `{"title":"Hi!"}`.)

K5. [S] Implement exponential backoff with jitter.

```ts
const delay = Math.random() * Math.min(maxDelay, base * 2 ** attempt);   // "full jitter"
```

Reset `attempt` on successful connect; stop on permanent auth errors.

K6. [S] Compute the Sec-WebSocket-Accept for a given key. `base64(sha1(key + "258EAFA5-E914-47DA-95CA-C5AB0DC85B11"))`, e.g., key `dGhlIHNhbXBsZSBub25jZQ==` → `s3pPLMBiTxaQ9kYGzzhZRbK+xOo=`.

K7. [S] Parse a WebSocket frame header.

```java
int b0 = buf[0] & 0xFF, b1 = buf[1] & 0xFF;
boolean fin = (b0 & 0x80) != 0; int opcode = b0 & 0x0F;
boolean masked = (b1 & 0x80) != 0; long len = b1 & 0x7F; int off = 2;
if (len == 126) { len = ((buf[2] & 0xFF) << 8) | (buf[3] & 0xFF); off = 4; }
else if (len == 127) { len = 0; for (int i = 2; i < 10; i++) len = (len << 8) | (buf[i] & 0xFF); off =
10; }
byte[] mask = masked ? Arrays.copyOfRange(buf, off, off + 4) : null; if (masked) off += 4;
// payload[i] ^= mask[i % 4]
```

K8. [S] Write a subscription authorisation interceptor for `/topic/orders.{id}`. See Chapter 8 §8.6.3: regex the destination, look up ownership, throw `AccessDeniedException` (or drop + error queue).

L. Behavioural / experience questions (prepare your stories)

L1. Tell me about a real-time feature you built. Structure: problem → why WebSocket/STOMP → architecture (gateway, broker, auth) → hardest problem (e.g., missed updates on reconnect → seq + resync) → results (latency, load reduction) → what you'd do differently.

L2. Describe a production incident with WebSockets. E.g., mass disconnects every 60 s after an infra change (new LB with default idle timeout) → detected via close-code dashboards → mitigated by raising timeouts → fixed permanently with heart-beats and alerting → postmortem: add LB config to IaC review checklist.

L3. How did you scale a real-time system? Simple broker → RabbitMQ relay or Redis backplane, capacity tests, metrics-driven autoscaling.

L4. How did you ensure security? CSWSH protection, JWT on CONNECT, destination authorisation with ownership, pen-test findings fixed.

L5. How do you onboard other teams to your real-time platform? SDKs, AsyncAPI docs, destination naming rules, review process, examples, runbooks.

M. Rapid-fire (one-liners)

Q	A
WebSocket RFC?	6455 (Dec 2011)
Handshake status code?	101
Magic GUID?	258EAFA5-E914-47DA-95CA-C5AB0DC85B11

Q	A
`Sec-WebSocket-Version` ?	13
Who masks frames?	Client → server only
Max control frame payload?	125 bytes
Close code for abnormal closure?	1006 (never sent on the wire)
Close code: message too big?	1009
Close code: policy violation?	1008
App-defined close codes?	4000–4999
Opcode for ping/pong/close?	0x9 / 0xA / 0x8
STOMP frame terminator?	NULL byte (`\0`)
STOMP default port over TCP?	61613 (conventional)
RabbitMQ Web STOMP port?	15674 (`/ws`)
STOMP sub-protocol names?	v10.stomp, v11.stomp, v12.stomp
Required CONNECT headers (1.2)?	accept-version, host
Required SUBSCRIBE headers?	destination, id
Required MESSAGE headers?	destination, message-id, subscription
Default ack mode?	auto
Cumulative ack mode?	client
ERROR frame consequence?	Server closes the connection
Heart-beat formula?	max(sender's cx, receiver's cy); 0 disables
Spring default app prefix?	None, you set it (commonly `/app`)
Spring default user prefix?	`/user`
Spring STOMP message size default?	64 KB
Spring send buffer limit default?	512 KB
Spring send time limit default?	10 s
Spring time to first message default?	60 s
Default return destination of @MessageMapping?	`/topic` + input destination
Annotation for initial data on subscribe?	`@SubscribeMapping`
Relay dependency?	Reactor Netty
SockJS heart-beat default (Spring)?	25 s
Do pure WebSockets need sticky sessions?	No (SockJS fallback does)
CORS applies to WebSocket?	No: check Origin
Browser can send ping?	No
Browser can set Authorization header?	No

N. Tricky "gotcha" questions

1. "**Is `Sec-WebSocket-Key` used for security?**" No; it only proves protocol understanding.
2. "**Does masking encrypt data?**" No; it prevents proxy cache poisoning. TLS encrypts.
3. "**Can the server send a 1006 close code?**" No; 1005, 1006 and 1015 are reserved for local reporting.
4. "**Does a STOMP RECEIPT mean subscribers got the message?**" No; only that the server processed the frame.
5. "**Will the simple broker redeliver unacked messages?**" No; it doesn't support acks.
6. "**Does `@SubscribeMapping`'s return value go through the broker?**" No; directly to the subscribing client (unless `@SendTo`).
7. "**Do STOMP heart-beats use WebSocket ping frames?**" No; they're EOL bytes inside the WebSocket data stream.
8. "**Can two subscriptions on one connection share an id?**" No; ids must be unique per connection.
9. "**Can `ws.close(1001)` be called from browser JS?**" No; only 1000 or 3000–4999.
10. "**If the client sends `destination: /topic/a` (with space), what's the destination?**" `" /topic/a"`: values aren't trimmed.
11. "**Does `SessionDisconnectEvent` fire exactly once?**" Not guaranteed; can fire multiple times.
12. "**Does HTTP/2 Server Push replace WebSockets?**" No (and it's deprecated in browsers).
13. "**Will TCP tell me immediately when the client disappears?**" No; half-open connections need heart-beats.
14. "**Does `maxPoolSize` grow Spring's inbound pool?**" Not with the default unbounded queue.
15. "**Is SockJS a WebSocket implementation?**" It's an emulation layer with its own protocol, using WebSocket when possible.
16. "**Does turning on `setPreserveReceiveOrder(true)` change error behaviour?**" Yes (Spring 7.0.x): exceptions from inbound interceptors are only logged, so no ERROR frame is sent and the connection stays open.
17. "**What close code does a Spring STOMP client see after an ERROR frame?**" 1002 (`PROTOCOL_ERROR`); Spring closes the session after sending ERROR.
18. "**What happens if stompjs's `beforeConnect` throws?**" In 7.x the rejection is unhandled and no further reconnect is scheduled, so always catch inside it.

O. More questions (added after a research pass on common interview topics)

O1. [E] People call WebSocket "TCP for the web" and STOMP "HTTP for the web". What does that mean? WebSocket gives browsers a raw, bidirectional message pipe (like TCP gives programs a byte pipe). STOMP adds HTTP-like structure on top (a command, headers and a body), so both sides agree on what a message means and where it goes.

O2. [E] Walk me through how `/app/**` messages reach a controller. The client SENDs to `/app/chat.send`. Spring sees the application prefix `/app`, strips it, and matches the remainder (`/chat.send`) against `@MessageMapping` patterns. The payload is converted by a `MessageConverter` and the method is invoked on a `clientInboundChannel` thread.

O3. [M] How does Spring find the "right connection" when you call `convertAndSendToUser`? It doesn't look up a socket directly. It sends to `/user/{name}/queue/x`; `UserDestinationMessageHandler` asks `SimpUserRegistry` for the sessions of `{name}` and rewrites the destination to each session's unique queue (`/queue/x-user{sessionId}`). The broker delivers to the subscription the client created by subscribing to `/user/queue/x`.

O4. [M] Which Spring component sends STOMP heart-beats on the server? The **broker**: `SimpleBrokerMessageHandler` (when a `TaskScheduler` is configured) or the external broker via the relay. There is **no** "HeartbeatInterceptor" class in Spring's STOMP support (see section P).

O5. [M] What changed for WebSocket/STOMP in Spring Framework 7 / Spring Security 7? `MappingJackson2MessageConverter` is deprecated for removal and replaced by the Jackson 3 based `JacksonJsonMessageConverter`. Spring Security 7 **removed** `AbstractSecurityWebSocketMessageBrokerConfigurer` and `MessageSecurityMetadataSourceRegistry`, so you must use `@EnableWebSocketSecurity` with an `AuthorizationManager<Message<?>>`. (Spring 6.2 had already added `ChannelRegistration.executor(Executor)` for custom executors such as virtual threads.)

O6. [M] How do you run Spring's message channels on virtual threads? Since 6.2: `registration.executor(new VirtualThreadTaskExecutor("ws-in-"))` in `configureClientInboundChannel` (and similarly for outbound). Load test it, and remember ordering still needs the preserve-order flags.

O7. [M] What happens when the simple broker's subscriber is too slow? Messages queue in `ConcurrentWebSocketSessionDecorator`. Spring uses its `TERMINATE` overflow strategy and closes the session with `CloseStatus.SESSION_NOT_RELIABLE` (4500) when the send time or buffer limit is exceeded. The decorator also has a `DROP` strategy if you use it directly.

O8. [M] Does Spring Security protect outbound messages? No. It secures only the `clientInboundChannel`; you protect data by authorising SUBSCRIBE frames. That's a performance decision, since one inbound message can fan out to thousands of outbound ones.

O9. [M] Why should clients be denied SUBSCRIBE to `/queue/**` directly? Because user destinations resolve to `/queue/...-user{sessionId}` names. A client allowed to subscribe to `/queue/**` (or guess those names) could read other users' private messages. Allow `/user/queue/**` and deny the raw broker prefix.

O10. [M] How do typing indicators usually work in a chat design? They are ephemeral events sent over the existing socket, rate-limited, never persisted. Server-side they're often a short-TTL key (e.g., 5 s in Redis) so no explicit "stopped typing" event is needed. They're the first thing to shed under load.

O11. [M] How is presence implemented at scale? A per-user key with a short TTL refreshed by heart-beats/activity (offline when it expires, which handles crashes that never fire disconnect events), published as change events only to interested users. A user→gateway-node map (e.g., in Redis) lets you route private messages to the right node instead of broadcasting to all nodes.

O12. [S] How many WebSocket connections can one server handle? There's no single number: it depends on memory per connection (kernel buffers, container buffers such as Tomcat's 8 KB per-session message buffers, framework objects, your state), message rate and fan-out. Public write-ups quote tens to hundreds of thousands per node. In an interview, state assumptions (e.g., ~20–50 KB per connection → ~20k–50k per GB of heap) and say you'd **measure** with a load test. Also mention OS limits (file descriptors), Tomcat's `max-connections` (Spring Boot default **8192**) and ephemeral ports.

O13. [S] With 10 million users, how do you route a message to the one gateway node holding the recipient's socket? Keep a connection registry (userId → nodeId, with TTL) in Redis or a similar store. Producers publish to that node's channel/partition, not to all nodes. Alternatives: consistent hashing of users to gateway clusters, or Kafka partitions keyed by userId with each gateway consuming its assigned partitions.

O14. [M] What does `wasClean` in the browser close event mean? `true` if the closing handshake completed (close frames exchanged); `false` if the connection dropped (typically code 1006).

O15. [M] Your WebSocket works through Nginx but drops at exactly 60 s, even with Nginx timeouts raised. What else? Another hop: an AWS ALB in front (default idle timeout 60 s, configurable 1–4000 s), a corporate proxy, or the backend container's idle timeout. Heart-beats shorter than the smallest idle timeout fix all of them at once.

O16. [M] What are AWS API Gateway WebSocket limits worth knowing? 2-hour max connection duration, 10-minute idle timeout, 128 KB message payload, 32 KB frame size (non-adjustable). An oversized message closes the connection with 1009. It doesn't speak STOMP natively.

O17. [M] Why might a GCP HTTPS load balancer drop WebSockets after 30 seconds? Its backend service timeout (default 30 s) acts as the maximum WebSocket connection lifetime. Raise it (e.g., 3600 s+).

O18. [M] Your company still runs the community ingress-nginx controller for WebSockets. Any concern? Yes: it was retired in March 2026 (no more releases or security patches). Plan migration to a Gateway API implementation or another maintained controller, and re-verify WebSocket timeouts there.

O19. [M] What's the difference between `@SendToUser` with and without an authenticated user? With a user: sent to that user's sessions (all, or only the current one with `broadcast = false`). Without a user: Spring falls back to the **current session id** and forces `broadcast = false`. If neither exists, it throws `MissingSessionUserException`.

O20. [M] Can a STOMP client receive messages for destinations it never subscribed to? No. MESSAGE frames are only sent for active subscriptions (each carries the `subscription` id). A frame that arrives after a client-side unsubscribe reaches stompjs's `onUnhandledMessage`.

O21. [M] What's the role of `ExecutorChannelInterceptor`? Its `beforeHandle` / `afterMessageHandled` run on the thread that actually handles the message (the executor thread). That's the right place to set and clear `ThreadLocal`s such as the security context, MDC logging keys or tenant context. `preSend` runs on the sending thread.

O22. [S] How do you version a real-time API used by mobile apps that update slowly? Use an envelope with `type` and `version`; make additive changes only; clients ignore unknown fields/types; the client sends its version in CONNECT headers so the server can adapt or reject; track client versions in metrics; keep old formats until usage is negligible. Document everything in AsyncAPI.

O23. [M] How would you test that a subscription is active before sending in an integration test? Use STOMP receipts (`setAutoReceipt(true)` + `addReceiptTask`) where supported, a round-trip ping through the same session, or Awaitility polling `SimpUserRegistry#findSubscriptions`. Avoid fixed sleeps.

O24. [M] Why is RabbitMQ's `/queue/<name>` risky for Spring user destinations? RabbitMQ creates `/queue/` queues as **durable, non-exclusive, non-auto-deleted** by default, so per-session user queues (`/queue/x-user{sessionId}`) pile up after sessions end. Use `/exchange/amq.direct/...` (auto-delete) or expiry policies.

O25. [S] How do you do a zero-downtime broker (RabbitMQ) upgrade under the relay? Cluster RabbitMQ with quorum queues for durable data; upgrade node by node; front the cluster with a TCP LB or rotate addresses in a custom `TcpClient` ; Spring's relay reconnects its system session every 5 s and publishes `BrokerAvailabilityEvent` ; clients get disconnected sessions and reconnect with jitter; verify user-destination queues and bindings come back.

O26. [M] What does `heartbeatToleranceMultiplier` do in stompjs? The client treats the server as dead if no data arrives within `multiplier × incoming heart-beat interval` (default 2). Raise it for flaky mobile networks or throttled background tabs.

O27. [M] How do you avoid a reconnect storm with stompjs? Use `reconnectTimeMode: ReconnectionTimeMode.EXPONENTIAL` with `maxReconnectDelay` , and add your own randomness (stompjs doesn't add jitter), e.g., a random initial `reconnectDelay` per client or adjusting it in `onWebSocketClose` .

O28. [S] Where do you terminate TLS for WebSockets and why? Usually at the load balancer (offloads CPU/memory, central cert management); re-encrypt to the backend in zero-trust networks. Consider per-connection TLS memory cost if terminating on the app nodes.

O29. [M] What's the difference between `SessionConnectEvent` and `SessionConnectedEvent` ? `SessionConnectEvent` fires when the client's CONNECT arrives (has the native CONNECT headers). `SessionConnectedEvent` fires after the broker has answered with CONNECTED; the original CONNECT is in its `simpConnectMessage` header.

O30. [M] Is `@ServerEndpoint` how Spring Boot does STOMP? No. `@ServerEndpoint` and `@ClientEndpoint` are **Jakarta WebSocket (JSR 356)** annotations for raw WebSocket endpoints. Spring STOMP uses `@EnableWebSocketMessageBroker` , `@MessageMapping` and friends (see Appendix A).

P. Common wrong answers you will find online (don't repeat them)

Wrong claim	Correct answer
"Spring Boot sends STOMP heart-beats through a `HeartbeatInterceptor` ."	No such Spring STOMP class. Heart-beats are negotiated in CONNECT/CONNECTED and sent by the broker (simple broker with a `TaskScheduler` , or the external broker via the relay) and by clients.
"Spring Boot creates STOMP endpoints with `@ServerEndpoint` / `@ClientEndpoint` ."	Those are Jakarta WebSocket (JSR 356) annotations for raw WebSocket. STOMP uses `@EnableWebSocketMessageBroker` + `@MessageMapping` .
" `Sec-WebSocket-Key` secures the connection."	It only proves the server speaks WebSocket. Security = TLS + Origin + auth.
"Masking encrypts client data."	It prevents proxy cache poisoning; the key is sent in clear.
"CORS protects WebSocket endpoints."	CORS doesn't apply; check `Origin` (CSWSH).
"The simple broker supports acks/redelivery."	It doesn't support acks, receipts (except DISCONNECT) or transactions.

Wrong claim	Correct answer
"WebSocket guarantees delivery because it's on TCP."	Only while the connection lives; disconnects lose messages. You need ids, resync and persistence.
"Use HTTP/2 Server Push for real-time."	It was for cache preloading and has been removed from Chrome.
"1006 is sent by the server when something fails."	1006 is never sent; it's reported locally when the connection dropped without a close frame.
"Sticky sessions are always required for WebSockets."	Only for SockJS HTTP fallbacks (or app-specific reconnect state).
"Set `maxPoolSize` to scale Spring's inbound channel."	With the default unbounded queue the pool never grows past `corePoolSize`.
"`AbstractSecurityWebSocketMessageBrokerConfigurer` is the way to secure messages."	Deprecated in 5.8/6 and removed in Spring Security 7.

Back to the README · Next: Chapter 13: Cheat Sheet & Glossary

CHAPTER 13

Chapter 13: Cheat Sheet & Glossary

13.1 One-page cheat sheet

WebSocket handshake

```
GET /ws HTTP/1.1                          HTTP/1.1 101 Switching Protocols
Host: h                                   Upgrade: websocket
Upgrade: websocket                        Connection: Upgrade
Connection: Upgrade                       Sec-WebSocket-Accept: base64(sha1(key+GUID))
Sec-WebSocket-Key: <16 random bytes b64> Sec-WebSocket-Protocol: v12.stomp
Sec-WebSocket-Version: 13
Origin: https://app
Sec-WebSocket-Protocol: v12.stomp, v11.stomp
GUID = 258EAFA5-E914-47DA-95CA-C5AB0DC85B11
```

Frame

```
FIN|RSV1-3|opcode(4) | MASK|len(7) | [len16|len64] | [mask key 4B] | payload
opcodes: 0 cont, 1 text, 2 binary, 8 close, 9 ping, A pong
client→server masked; control frames ≤125 B, unfragmented
```

Close codes

`1000` normal · `1001` going away · `1002` protocol · `1003` unsupported · `1005*` no status · `1006*` abnormal · `1007` bad data · `1008` policy · `1009` too big · `1010` extension · `1011` internal · `1012` restart · `1013` try later · `1015*` TLS · `3000–3999` libs · `4000–4999` app. (`*` never sent on the wire)

STOMP frames

```
COMMAND\nheader:value\n\nbody\0
Client: CONNECT/STOMP SEND SUBSCRIBE UNSUBSCRIBE ACK NACK BEGIN COMMIT ABORT DISCONNECT
Server: CONNECTED MESSAGE RECEIPT ERROR
```

Frame	Required headers
CONNECT	accept-version, host (opt: login, passcode, heart-beat)
CONNECTED	version (opt: session, server, heart-beat)
SEND	destination
SUBSCRIBE	destination, id (opt: ack=auto/client/client-individual)
UNSUBSCRIBE	id
ACK / NACK	id (= MESSAGE's ack header)

Frame	Required headers
BEGIN/COMMIT/ABORT	transaction
MESSAGE	destination, message-id, subscription
RECEIPT	receipt-id
ERROR	(message) → connection closes

Heart-beat: client→server = (cx==0‖sy==0) ? 0 : max(cx,sy); server→client = (sx==0‖cy==0) ? 0 : max(sx,cy). **Escapes (1.2):** `\r` `\n` `\c` `\\` ; none in CONNECT/CONNECTED. First repeated header wins. Bodies only on SEND/MESSAGE/ERROR.

Spring essentials

JAVA

```
@Configuration @EnableWebSocketMessageBroker
class Cfg implements WebSocketMessageBrokerConfigurer {
  public void registerStompEndpoints(StompEndpointRegistry r) {
    r.addEndpoint("/ws").setAllowedOriginPatterns("https://*.example.com"); // .withSockJS()
  }
  public void configureMessageBroker(MessageBrokerRegistry r) {
    r.setApplicationDestinationPrefixes("/app");
    r.enableSimpleBroker("/topic","/queue").setTaskScheduler(ts).setHeartbeatValue(new long[]
{10000,10000});
    // r.enableStompBrokerRelay("/topic","/queue").setRelayHost(..).setRelayPort(61613)...
    r.setUserDestinationPrefix("/user");
    r.setPreservePublishOrder(true);
  }
  public void configureClientInboundChannel(ChannelRegistration reg) {
reg.interceptors(jwtConnectInterceptor); }
  public void configureWebSocketTransport(WebSocketTransportRegistration t) {

t.setMessageSizeLimit(64*1024).setSendBufferSizeLimit(512*1024).setSendTimeLimit(10_000).setTimeToFirstM
essage(30_000);
  }
}
```

Annotation / API	Purpose
`@MessageMapping("/x")`	Handle client SEND to `/app/x`
`@SubscribeMapping("/x")`	Reply once to SUBSCRIBE `/app/x`
`@SendTo("/topic/y")`	Broadcast return value
`@SendToUser("/queue/z")`	Return value to current user (`broadcast=false` → one session)
`@MessageExceptionHandler`	Handle exceptions (also in `@ControllerAdvice`)
`@DestinationVariable` , `@Header` , `@Headers` , `@Payload` , `Principal`	Method args
`SimpMessagingTemplate.convertAndSend` / `convertAndSendToUser`	Push from anywhere
`SimpUserRegistry`	Who's connected
`Session*Event` , `BrokerAvailabilityEvent`	Lifecycle

Annotation / API	Purpose
`StompSubProtocolErrorHandler`	Customise ERROR frames
`WebSocketMessageBrokerStats`	Built-in stats log

stompjs essentials

```ts
const c = new Client({ brokerURL: 'wss://h/ws', reconnectDelay: 5000, heartbeatIncoming: 10000,
heartbeatOutgoing: 10000,
  beforeConnect: async () => { c.connectHeaders = { Authorization: `Bearer ${await token()}` }; },
  onConnect: () => { c.subscribe('/topic/x', m => use(JSON.parse(m.body))); },
  onStompError: f => console.error(f.headers.message) });
c.activate();
c.publish({ destination: '/app/y', body: JSON.stringify(d), headers: { 'content-type':
'application/json' } });
await c.deactivate();
// React Native: forceBinaryWSFrames: true, appendMissingNULLonIncoming: true
```

Nginx

```nginx
map $http_upgrade $connection_upgrade { default upgrade; '' close; }
location /ws { proxy_pass http://backend; proxy_http_version 1.1;
  proxy_set_header Upgrade $http_upgrade; proxy_set_header Connection $connection_upgrade;
  proxy_read_timeout 3600s; proxy_send_timeout 3600s; proxy_buffering off; }
```

Top 10 rules

1. `wss://` + Origin allow-list + token in CONNECT.
2. Authorise every SUBSCRIBE and SEND; deny by default; identity from Principal.
3. Heart-beats 10–25 s, shorter than every idle timeout.
4. Reconnect with backoff + jitter; re-subscribe in `onConnect`.
5. DB is truth; events carry ids/seq; resync after reconnect.
6. Publish after commit (or outbox); idempotent commands.
7. Simple broker = single instance; cluster → relay or backplane.
8. Bound buffers (send limits), size pools, conflate high-frequency data.
9. Small messages; big data over HTTP.
10. Measure: sessions, close codes, queues, latency; load + chaos test.

13.2 Glossary

Term	Definition
ABORT	STOMP frame cancelling a transaction
ACK / NACK	STOMP frames acknowledging (or rejecting) a message for `client` / `client-individual` subscriptions
accept-version	CONNECT header listing STOMP versions the client supports

Term	Definition
AMQP	Advanced Message Queuing Protocol; binary messaging protocol (0-9-1 RabbitMQ, 1.0 OASIS standard)
AsyncAPI	Specification for documenting message-driven APIs (channels, messages, payloads)
At-least-once / at-most-once / exactly-once	Delivery guarantees: ≥1, ≤1, exactly 1 (effectively via dedupe)
Backplane	Pub/sub layer (Redis, Kafka) connecting multiple server instances so messages reach all their clients
Backpressure	Mechanism to slow producers when consumers can't keep up
bfcache	Browser back/forward cache; pages with open WebSockets may be evicted or have connections closed
Binary frame	WebSocket opcode 0x2, arbitrary bytes
Broker	Server that routes/stores messages between producers and consumers
Broker relay	Spring's `StompBrokerRelayMessageHandler` forwarding STOMP to an external broker
brokerChannel	Spring channel for messages from application code to the broker
bufferedAmount	Browser WebSocket property: bytes queued but not yet sent
ChannelInterceptor	Spring hook on message channels (`preSend` , `postSend` , `afterSendCompletion`)
clientInboundChannel / clientOutboundChannel	Spring channels for messages from / to WebSocket clients
Close frame / handshake	Opcode 0x8 frame with status code + reason; both sides exchange before TCP close
Comet	Umbrella term for long polling/HTTP streaming push techniques
Conflation	Sending only the latest value per key at intervals
CONNECT / CONNECTED	STOMP frames to open a session / server confirmation
content-length	STOMP header: body size in bytes; allows NULLs in body
Context takeover	permessage-deflate option to reuse the compression dictionary across messages
Continuation frame	Opcode 0x0, subsequent fragment of a message
Control frame	Close, ping, pong (≤125 bytes, not fragmented)
CSWSH	Cross-Site WebSocket Hijacking: attacker page opening an authenticated socket via victim's cookies
Destination	STOMP address string (e.g., `/topic/x`), broker-defined semantics
DISCONNECT	STOMP frame to close a session gracefully (with receipt)
Durable subscription/queue	Survives disconnects/restarts and keeps messages
EOL	End of line: LF, or CRLF in STOMP 1.2
ERROR	STOMP server frame reporting an error; connection then closes
Exchange	RabbitMQ routing entity (direct, topic, fanout, headers)

Term	Definition
Extended CONNECT	HTTP/2 (RFC 8441) / HTTP/3 (RFC 9220) mechanism to bootstrap WebSockets
Fan-out	Delivering one message to many subscribers
FIN bit	Marks the final frame of a message
Fragmentation	Splitting a message into multiple frames
Full-duplex	Both sides can send simultaneously
Half-open connection	One side thinks the connection is alive while the other is gone
HandshakeInterceptor	Spring hook around the HTTP upgrade
Heart-beat	Periodic EOL (STOMP) or ping/pong (WebSocket) to prove liveness
host header	STOMP CONNECT header naming a virtual host
Idempotency key	Client-generated id letting the server ignore duplicate commands
Jitter	Randomness added to retry delays to spread load
Long polling	Server holds a request open until data is available
Masking	XOR of client payloads with a random 4-byte key (anti cache-poisoning)
MESSAGE	STOMP server frame delivering a message to a subscription
message-id	Unique id of a MESSAGE frame
MQTT	Lightweight pub/sub protocol for IoT, QoS 0/1/2
Native headers	Spring's term for the raw STOMP headers inside a `Message`
Opcode	4-bit WebSocket frame type
Origin	Header carrying the page's origin; checked to prevent CSWSH
Outbox pattern	Writing events to a DB table in the same transaction for reliable publishing
permessage-deflate	WebSocket compression extension (RFC 7692)
Ping / Pong	WebSocket control frames for liveness
Prefetch	Max unacked messages a broker sends to a consumer
Presence	Tracking which users are online
Principal	Authenticated user identity attached to a WebSocket session
Pub/Sub	Publish/subscribe messaging pattern
Queue	Point-to-point destination; each message to one consumer
readyState	Browser WebSocket state: 0 CONNECTING, 1 OPEN, 2 CLOSING, 3 CLOSED
RECEIPT / receipt-id	STOMP server confirmation of a frame that carried a `receipt` header
Resync	Fetching missed data after reconnect or gap detection
RFC 6455	The WebSocket protocol specification
RSV bits	Reserved frame bits used by extensions (RSV1 = compressed in permessage-deflate)

Term	Definition
Sec-WebSocket-Accept	Server handshake header proving WebSocket support
Sec-WebSocket-Key	Client handshake nonce
Sec-WebSocket-Protocol	Sub-protocol negotiation header
SEND	STOMP frame to send a message to a destination
Sequence number (seq)	Monotonic per-entity counter for ordering/gap detection
SimpMessagingTemplate	Spring API to send messages to the broker
SimpUserRegistry	Spring registry of connected users/sessions/subscriptions
Simple broker	Spring's in-memory STOMP broker
SockJS	WebSocket emulation protocol/library with HTTP fallbacks
SSE	Server-Sent Events: one-way server push over HTTP
Sticky session	LB routing a client's requests to the same backend
STOMP	Simple/Streaming Text Oriented Messaging Protocol
Sub-protocol	Application protocol negotiated over WebSocket (e.g., `v12.stomp`)
SUBSCRIBE / UNSUBSCRIBE	STOMP frames to start/stop receiving messages from a destination
subscription header	MESSAGE header carrying the matching SUBSCRIBE id
Text frame	Opcode 0x1, UTF-8 payload
Thundering herd	Many clients reconnecting/retrying simultaneously
TLS	Transport Layer Security; makes `wss://`
Topic	Pub/sub destination; each message to all subscribers
Transaction (STOMP)	BEGIN/COMMIT/ABORT grouping SEND/ACK frames atomically
User destination	Spring `/user/...` destinations resolved to per-session destinations
Virtual host	Broker namespace (RabbitMQ vhost), selected by STOMP `host` header
WebSocketStompClient	Spring's Java STOMP client
WebTransport	HTTP/3-based API with streams and datagrams
wss://	WebSocket over TLS

13.3 Study plan

Day	Read	Practice
1	Ch 1–2	Decode frames by hand; compute an Accept key
2	Ch 3–4	Write raw STOMP frames with wscat/Postman; build the TinyStomp client
3	Ch 5	Build a Spring chat with `/app`, `/topic`, `/user/queue`
4	Ch 6	React (or React Native) client with reconnect + resync

Day	Read	Practice
5	Ch 7	Switch to RabbitMQ relay with Docker; run 2 instances behind Nginx
6	Ch 8	Add JWT CONNECT auth + subscription authorisation + tests
7	Ch 9–10	Add metrics, load test with k6, break things on purpose and fix them using the Bug Encyclopedia
8	Ch 11	Write an ADR + AsyncAPI doc for your app
9–10	Ch 12–13	Answer all questions aloud; do 2 system designs on a whiteboard

Back to the README.

APPENDIX A

Appendix A: Beyond Spring STOMP (Other Stacks, Protocols and Integrations)

Goal: fill the gaps around the main book: raw WebSocket APIs in Java (Jakarta, Spring, WebFlux), RSocket, Spring Session, a Kafka-backed real-time gateway, WebSocket over HTTP/2 in practice, Node.js, Socket.IO, SignalR, Go, GraphQL subscriptions, CRDT sync (Yjs) and MQTT over WebSocket. You don't need to master all of these, but interviewers love "how does X compare to STOMP?" and real companies mix stacks.

A.1 ELI5

STOMP is one language you can speak on the WebSocket telephone line. Other teams use other languages on the same line (Socket.IO, SignalR, GraphQL subscriptions, MQTT, Yjs), or talk on the line with no shared language at all (raw WebSocket). Knowing the neighbours helps you choose well and explain your choice.

A.2 Jakarta WebSocket (JSR 356) `@ServerEndpoint`

The **standard Java API** for raw WebSocket, implemented by Tomcat, Jetty, Undertow and others. No STOMP, no broker: you get sessions and messages.

JAVA

```java
@ServerEndpoint(value = "/raw/echo",
                decoders = ChatDecoder.class, encoders = ChatEncoder.class)
public class EchoEndpoint {

    private static final Set<Session> SESSIONS = ConcurrentHashMap.newKeySet();

    @OnOpen
    public void onOpen(Session session, EndpointConfig config) {
        session.setMaxIdleTimeout(0);                          // rely on app heart-beats instead
        session.setMaxTextMessageBufferSize(64 * 1024);
        SESSIONS.add(session);
    }

    @OnMessage
    public void onMessage(Session session, ChatMessage msg) {   // decoded by ChatDecoder
        for (Session s : SESSIONS) {
            if (s.isOpen()) s.getAsyncRemote().sendObject(msg); // async send; see thread-safety note
        }
    }

    @OnClose
    public void onClose(Session session, CloseReason reason) { SESSIONS.remove(session); }

    @OnError
```

```
    public void onError(Session session, Throwable t) { SESSIONS.remove(session); }
}
```

In **Spring Boot**, register annotated endpoints with a `ServerEndpointExporter` bean (embedded container):

JAVA

```java
@Bean
public ServerEndpointExporter serverEndpointExporter() { return new ServerEndpointExporter(); }
```

Key facts:

- `@ServerEndpoint` classes are instantiated **per connection** by the container by default, so they aren't Spring singletons. Injecting Spring beans needs `SpringConfigurator` or a custom `ServerEndpointConfig.Configurator`.
- `Session.getBasicRemote()` is **blocking**; `getAsyncRemote()` is non-blocking. Sending concurrently to the same session from multiple threads can throw `IllegalStateException` (Tomcat: `TEXT_PARTIAL_WRITING`). Serialise sends per session.
- `Encoder` / `Decoder` classes convert between objects and text/binary.
- `@ClientEndpoint` + `ContainerProvider.getWebSocketContainer().connectToServer(...)` for Java clients.
- **Interview point:** `@ServerEndpoint` is *not* how Spring STOMP works. Spring STOMP uses `@EnableWebSocketMessageBroker` + `@MessageMapping`.

A.3 Spring's raw `WebSocketHandler` API (servlet stack)

When you want Spring integration (beans, security, interceptors) without STOMP:

JAVA

```java
@Configuration
@EnableWebSocket
public class RawConfig implements WebSocketConfigurer {
    private final GameSocketHandler handler;
    public RawConfig(GameSocketHandler handler) { this.handler = handler; }

    @Override
    public void registerWebSocketHandlers(WebSocketHandlerRegistry registry) {
        registry.addHandler(handler, "/game")
                .addInterceptors(new HttpSessionHandshakeInterceptor())
                .setAllowedOriginPatterns("https://*.example.com");
    }
}

@Component
public class GameSocketHandler extends TextWebSocketHandler {
    private final Map<String, WebSocketSession> sessions = new ConcurrentHashMap<>();

    @Override
    public void afterConnectionEstablished(WebSocketSession raw) {
        // Make sends thread-safe + bounded: 5 s send limit, 256 KB buffer, drop or terminate on
overflow
        WebSocketSession s = new ConcurrentWebSocketSessionDecorator(raw, 5_000, 256 * 1024,
                ConcurrentWebSocketSessionDecorator.OverflowStrategy.DROP);
        sessions.put(raw.getId(), s);
    }
```

```
    @Override
    protected void handleTextMessage(WebSocketSession session, TextMessage message) throws Exception {
        TextMessage out = new TextMessage("echo:" + message.getPayload());
        for (WebSocketSession s : sessions.values()) if (s.isOpen()) s.sendMessage(out);
    }

    @Override
    public void afterConnectionClosed(WebSocketSession session, CloseStatus status) {
        sessions.remove(session.getId());
    }

    @Override
    public boolean supportsPartialMessages() { return false; }   // true → receive fragments (handle
buffering yourself)
}
```

Notes:

- `BinaryWebSocketHandler` for binary protocols (protobuf, game state).
- `PerConnectionWebSocketHandler` creates a handler instance per connection (when you want per-connection state in fields).
- `OverflowStrategy.DROP` drops the **oldest** buffered messages instead of closing the session when the *buffer* limit is exceeded; good for "latest state" streams. Exceeding the *send time* limit still terminates the session.
- Heart-beats: send `PingMessage`s from a scheduler and track `PongMessage`s in `handlePongMessage`.
- You get no destinations, subscriptions or user routing, so you build them. This is the main reason teams pick STOMP.

A.4 Spring WebFlux reactive WebSocket

WebFlux (Netty or servlet containers in reactive mode) has its own `org.springframework.web.reactive.socket.WebSocketHandler`. **No STOMP support** here.

JAVA

```
@Configuration
public class ReactiveWsConfig {
    @Bean
    public HandlerMapping wsMapping(PriceHandler handler) {
        return new SimpleUrlHandlerMapping(Map.of("/prices", handler), -1);   // order before annotated
controllers
    }
    // Needed only if you are NOT using the WebFlux config (@EnableWebFlux / Boot auto-config):
    // @Bean WebSocketHandlerAdapter handlerAdapter() { return new WebSocketHandlerAdapter(); }
}

@Component
public class PriceHandler implements WebSocketHandler {
    private final Sinks.Many<String> prices = Sinks.many().multicast().directBestEffort(); // drops for
slow subscribers

    public void publish(String json) { prices.tryEmitNext(json); }

    @Override
    public Mono<Void> handle(WebSocketSession session) {
        Mono<Void> input = session.receive()                          // Flux<WebSocketMessage>
                .map(WebSocketMessage::getPayloadAsText)
```

```
                .doOnNext(cmd -> { /* handle client commands */ })
                .then();
        Mono<Void> output = session.send(prices.asFlux()
                .onBackpressureLatest()                                  // conflation: keep only latest
when slow
                .map(session::textMessage));
        return Mono.zip(input, output).then();                          // completes when either side
ends
    }
}
```

- `session.receive()` returns a `Flux<WebSocketMessage>`; `session.send(Publisher)` returns `Mono<Void>`. The handler's returned `Mono<Void>` signals when the session is done.
- Reactive streams give **real backpressure** semantics (`onBackpressureLatest`, `onBackpressureBuffer(n)`, `limitRate`).
- Payload buffers (`DataBuffer`) are released by the framework when you map them in the pipeline; if you hold raw messages beyond the pipeline, call `retain()`/release correctly.
- Client: `ReactorNettyWebSocketClient().execute(uri, session -> ...)`.
- When to choose: fully reactive stack, high connection counts on Netty, streaming pipelines. If you want STOMP semantics, stay on Spring MVC or use RSocket.

A.5 RSocket (application protocol with reactive semantics)

RSocket is a binary application protocol with four interaction models: **request-response**, **fire-and-forget**, **request-stream**, **request-channel** (bidirectional streams), plus **flow control** (`request(n)`), **resumption** and **metadata** routing. It can run over TCP or **WebSocket**.

Spring Boot:

PROPERTIES

```
spring.rsocket.server.mapping-path=/rsocket
spring.rsocket.server.transport=websocket
```

JAVA

```
@Controller
public class PriceRSocketController {
    @MessageMapping("prices.{symbol}")                  // routing metadata, same annotation name as
STOMP!
    public Flux<Price> stream(@DestinationVariable String symbol) { return priceService.stream(symbol);
}

    @MessageMapping("orders.place")
    public Mono<OrderAck> place(OrderRequest req) { return orderService.place(req); }
}
```

Browser client: `rsocket-js`. **Comparison with STOMP:** RSocket has built-in request/response, backpressure and resumption, but it's binary, less widely adopted, has fewer brokers/tools, and its browser libraries are less mature. STOMP is simpler and ubiquitous with brokers. Use RSocket for reactive service-to-service or advanced streaming needs.

A.6 Spring Session integration (keep HTTP sessions alive)

Problem: with **cookie/HTTP-session authentication**, a user may be active only over WebSocket for an hour. The HTTP session isn't touched, so it **expires**, and the next REST call fails, or worse, the WebSocket keeps working for a logged-out user.

Spring Session provides `AbstractSessionWebSocketMessageBrokerConfigurer`, which:

- keeps the HTTP `Session` alive on incoming WebSocket messages, and
- closes WebSocket sessions when the HTTP session is terminated (logout/expiry).

JAVA

```
@Configuration
@EnableScheduling
@EnableWebSocketMessageBroker
public class WebSocketConfig<S extends Session> extends
AbstractSessionWebSocketMessageBrokerConfigurer<S> {

    @Override
    protected void configureStompEndpoints(StompEndpointRegistry registry) {   // note: not
registerStompEndpoints
        registry.addEndpoint("/ws").setAllowedOriginPatterns("https://app.example.com");
    }

    @Override
    public void configureMessageBroker(MessageBrokerRegistry registry) {
        registry.enableSimpleBroker("/queue/", "/topic/");
        registry.setApplicationDestinationPrefixes("/app");
    }
}
```

Combine with a Spring Session store (`spring-session-data-redis` / JDBC) for clustering. Not needed if you use stateless JWTs in CONNECT.

A.7 A Kafka-backed real-time gateway (complete pattern)

Large companies often keep domain services ignorant of WebSockets: they publish **events to Kafka**, and a **real-time gateway** pushes them to connected clients.

```
 order-service ──► Kafka topic "user-notifications" (key = userId) ──► realtime-gateway pods ──► STOMP
clients
 todo-service  ──► Kafka topic "list-events" (key = listId)        ──►        "
```

A.7.1 Broadcast-style consumption (simple, every pod gets everything)

Each gateway pod uses a **unique consumer group** so it receives **all** events, then delivers to its local sessions.

JAVA

```
@Component
@RequiredArgsConstructor
public class KafkaToStompBridge {
    private final SimpMessagingTemplate messaging;
    private final ObjectMapper json;

    @KafkaListener(topics = "list-events",
```

```
                    groupId = "realtime-gateway-#{T(java.util.UUID).randomUUID().toString()}") // unique
group per pod
    public void onListEvent(ConsumerRecord<String, String> rec) throws IOException {
        JsonNode event = json.readTree(rec.value());
        String listId = rec.key();
        messaging.convertAndSend("/topic/todo.list." + listId, event);     // simple broker: only local
subscribers get it
    }

    @KafkaListener(topics = "user-notifications",
                    groupId = "realtime-gateway-#{T(java.util.UUID).randomUUID().toString()}")
    public void onNotification(ConsumerRecord<String, String> rec) throws IOException {
        messaging.convertAndSendToUser(rec.key(), "/queue/notifications", json.readTree(rec.value()));
        // no-op on pods where this user has no session
    }
}
```

- Set `auto.offset.reset=latest` for broadcast groups (a new pod doesn't need history; clients resync via REST).
- Random group ids leave stale groups behind in Kafka. Use a stable per-pod id (e.g., the StatefulSet pod name) instead of a random UUID in production.
- Cost: every pod processes every event. Fine up to a few thousand events/s.

A.7.2 Targeted routing (scales further)

1. On connect, the gateway records `userId → podId` in Redis (TTL refreshed by heart-beats/events, removed on disconnect).
2. A **router** service consumes `user-notifications`, looks up the pod, and forwards to that pod's own channel (a Redis channel `pod:{podId}` or a per-pod Kafka partition).
3. If the user is offline → push notification / inbox only. This avoids every pod processing every message, but adds a lookup and a moving part.

A.7.3 Ordering and reliability

- Key Kafka records by entity id (`listId`, `conversationId`) → per-entity ordering within a partition.
- Include `seq` in the event so clients can detect gaps; resync via REST on reconnect.
- The gateway is **not** a durable store: if a pod restarts, its clients reconnect and resync.

A.8 WebSocket over HTTP/2 and HTTP/3 in practice

- **Browsers** can bootstrap WebSockets over HTTP/2 (RFC 8441) when the server advertises `SETTINGS_ENABLE_CONNECT_PROTOCOL`. If it doesn't, they open a separate HTTP/1.1 connection with the classic Upgrade. **Your code doesn't change**; `new WebSocket(url)` is the same.
- **Servers/proxies** vary: some implement RFC 8441 (e.g., Jetty and Envoy support it), many deployments don't, and often the edge speaks HTTP/2 to browsers while forwarding WebSockets to the app over HTTP/1.1 Upgrade. Check each hop's documentation before relying on it.
- **Benefits:** many WebSockets share one TCP+TLS connection (fewer handshakes, fewer sockets on mobile). **Drawbacks:** TCP head-of-line blocking affects all streams on that connection; per-stream flow control interacts with WebSocket backpressure; less mature tooling.

- **HTTP/3 (RFC 9220)** removes TCP head-of-line blocking via QUIC; support is still emerging. **WebTransport** is the HTTP/3-native alternative with unreliable datagrams.
- **Practical advice:** don't architect around HTTP/2 WebSockets; treat them as a transparent optimisation where available.

A.9 Node.js

A.9.1 Raw WebSocket server

See Chapter 3 §3.9 (`ws` library with ping/pong liveness). For STOMP semantics in Node, teams usually either:

- put **RabbitMQ Web STOMP** (or ActiveMQ) behind their auth layer, or
- use Socket.IO / their own JSON protocol on `ws`.

A.9.2 STOMP client in Node

JS

```
// Node 22+ has a global WebSocket. On older Node, polyfill it with the 'ws' package:
// import { WebSocket } from 'ws'; Object.assign(globalThis, { WebSocket });
import { Client } from '@stomp/stompjs';

const client = new Client({
  brokerURL: 'ws://localhost:8080/ws',
  connectHeaders: { Authorization: `Bearer ${process.env.TOKEN}` },
  onConnect: () => {
    client.subscribe('/topic/todo.list.L1', (m) => console.log('event', m.body));
    client.publish({ destination: '/app/todo.upsert', body: JSON.stringify({ listId: 'L1', title: 'from
node' }),
                     headers: { 'content-type': 'application/json' } });
  },
});
client.activate();
```

For STOMP over **TCP** (e.g., to RabbitMQ 61613) Node has libraries such as `stompit`; check maintenance status before adopting.

A.10 Socket.IO in depth (and how it differs from STOMP)

Architecture: Socket.IO (server: Node; clients: JS, Swift, Java, Python, etc.) sits on **Engine.IO**, which handles transports (HTTP long-polling first, then upgrade to WebSocket by default; or WebSocket-only if configured), heart-beats and reconnection.

Concept	Socket.IO	STOMP equivalent
Event	`socket.emit('chat', data)` / `socket.on('chat', fn)`	SEND to destination / MESSAGE on subscription
Rooms	`socket.join('room-1')`, `io.to('room-1').emit(...)` (server-side membership)	Client SUBSCRIBE to `/topic/room-1`
Namespaces	`io.of('/admin')` (multiplex logical channels)	Destination prefixes

Concept	Socket.IO	STOMP equivalent
Acknowledgements	`socket.emit('save', data, (ack) => ...)` callback (request/response built in)	Receipts / reply queues
Broadcast	`socket.broadcast.emit`	Topic publish
Middleware	`io.use((socket, next) => ...)` (auth)	Channel interceptors
Scaling	**Adapters** (Redis, Redis Streams, Postgres, cluster adapter)	Broker relay / backplane
Sticky sessions	**Required** when HTTP long-polling is enabled (multiple requests per session)	Only for SockJS fallbacks
Recovery	Connection state recovery (v4.6+) can restore rooms and missed packets for brief disconnects	Your resync design

Key differences to say in interviews:

- Socket.IO's room membership is **server-controlled** (`join`), so authorisation lives in your join logic. STOMP subscriptions are **client-requested**, so you must authorise SUBSCRIBE.
- Socket.IO has its own wire protocol, so you can't connect with a plain WebSocket or STOMP client.
- Socket.IO is idiomatic in Node ecosystems; STOMP is idiomatic with Spring/RabbitMQ.

A.11 ASP.NET Core SignalR

- **Hubs**: server classes whose methods clients invoke (`connection.invoke("SendMessage", user, text)`), and the server calls client methods (`Clients.Group("room").SendAsync("ReceiveMessage", ...)`).
- **Transports** negotiated: WebSockets → Server-Sent Events → Long Polling. A `negotiate` HTTP request precedes the connection unless you configure WebSockets-only with `skipNegotiation` .
- **Protocols:** JSON or MessagePack hub protocols.
- **Groups** (server-managed membership), **users** (by `IUserIdProvider`), **connections**.
- **Scale-out:** Redis backplane or **Azure SignalR Service** (managed connections). Sticky sessions are required for multi-server setups unless all clients use WebSockets with negotiation skipped, or you use Azure SignalR Service.
- **Comparison:** SignalR ≈ Socket.IO for .NET: RPC-style hub calls, server-managed groups. STOMP ≈ messaging with destinations. Java/Spring shops pick STOMP; .NET shops pick SignalR.

A.12 Go

- Libraries: `github.com/gorilla/websocket` (classic, widely used; the project was archived in 2022 and later revived under new maintainers) and `github.com/coder/websocket` (formerly `nhooyr.io/websocket` , context-aware, minimal).
- **Concurrency rule (gorilla):** at most **one concurrent writer** and **one concurrent reader** per connection. Typical pattern: one goroutine `readPump` , one goroutine `writePump` fed by a buffered channel; the write pump also sends pings on a ticker; the read pump sets a read deadline extended on each pong.

- STOMP in Go: `github.com/go-stomp/stomp` (client and a simple server, over TCP; wrap a WebSocket as `net.Conn` if needed).

```go
// writePump sketch (gorilla)
ticker := time.NewTicker(pingPeriod)
for {
    select {
    case msg, ok := <-c.send:
        c.conn.SetWriteDeadline(time.Now().Add(writeWait))
        if !ok { c.conn.WriteMessage(websocket.CloseMessage, []byte{}); return }
        if err := c.conn.WriteMessage(websocket.TextMessage, msg); err != nil { return }
    case <-ticker.C:
        c.conn.SetWriteDeadline(time.Now().Add(writeWait))
        if err := c.conn.WriteMessage(websocket.PingMessage, nil); err != nil { return }
    }
}
```

A.13 GraphQL subscriptions over WebSocket

- Modern protocol: `graphql-transport-ws` (implemented by the `graphql-ws` library). Message types: `connection_init` (with auth payload), `connection_ack`, `subscribe`, `next`, `error`, `complete`, `ping`, `pong`.
- Legacy protocol: `subscriptions-transport-ws` (Apollo, deprecated), which confusingly uses the sub-protocol name `graphql-ws`. Client and server must use the **same** protocol; this mismatch is a classic bug.
- Auth goes in `connection_init`'s payload (the same idea as STOMP CONNECT headers).
- **Spring for GraphQL** supports subscriptions over WebSocket (`spring.graphql.websocket.path=/graphql`) with `@SubscriptionMapping` returning a `Flux`.
- Comparison: GraphQL subscriptions give typed, client-selected fields per event; STOMP gives free-form messages. Scaling needs the same backplane thinking.

A.14 CRDT-based sync (collaborative editing): Yjs

- **Yjs** is a CRDT library; documents converge regardless of update order. **y-websocket** syncs them over WebSocket with a small binary protocol: **sync step 1** (state vector), **sync step 2** (missing updates), then incremental **updates**; plus **awareness** messages (cursors, presence).
- Servers: `y-websocket` server, **Hocuspocus** (auth, persistence hooks, Redis scaling), `y-redis`.
- Persistence: store merged document state (e.g., in a DB/S3) and/or update logs.
- Why not STOMP? You *can* carry Yjs updates as binary STOMP bodies on a `/topic/doc.{id}`, but you lose the ready-made sync handshake, awareness and persistence integrations. Most teams use the Yjs ecosystem directly and keep STOMP for other real-time features.
- **OT vs CRDT:** OT (Google Docs style) transforms operations via a central server; CRDTs merge without central coordination (better offline, more metadata).

A.15 MQTT over WebSocket

- MQTT brokers (Mosquitto, EMQX, HiveMQ, RabbitMQ's Web MQTT plugin) expose WebSocket listeners; browser clients like **mqtt.js** connect with sub-protocol `mqtt`.
- Topic wildcards `+` (one level) and `#` (multi-level), separator `/`; QoS 0/1/2; retained messages; last will; session persistence.
- Choose MQTT when IoT devices and browsers share the same pub/sub fabric. Choose STOMP when you're in the Spring/RabbitMQ web-app world.

A.16 The big comparison table

	STOMP/WS	Raw WS	Socket.IO	SignalR	GraphQL subs	RSocket	MQTT/WS	Yjs
Style	Messaging (destinations)	Bytes/ messages	Events + rooms	RPC hubs + groups	Typed queries	Reactive streams	IoT pub/sub	CRDT sync
Ecosystem	Spring, RabbitMQ, ActiveMQ	Any	Node	.NET	Any GraphQL	Spring, Java	IoT brokers	Collab editors
Who controls membership	Client subscribes (authorise!)	You	Server (`join`)	Server (groups)	Client query (authorise!)	Client request	Client subscribe (ACLs)	Doc-level auth
Request/response	Convention	You	Built-in acks	Built-in invoke	Queries/mutations	Built-in	Convention	n/a
Backpressure	Limits/prefetch	You	Limited	Limited	Limited	**Built-in**	QoS/inflight	n/a
Fallback transports	SockJS	No	Long-polling	SSE, long-polling	No	No	No	No
Browser client maturity	High	Native	High	High	High	Medium	High	High

---

A.17 Summary

- Raw WebSocket APIs (Jakarta, Spring `WebSocketHandler`, WebFlux) give full control with no messaging semantics.
- RSocket adds reactive streams, backpressure and resumption; Spring Session keeps cookie sessions aligned with sockets.
- A Kafka-backed gateway decouples domain teams from WebSockets.

- Socket.IO and SignalR are framework-specific alternatives with server-managed rooms/groups; GraphQL subscriptions, MQTT and Yjs serve specialised needs.
- The same production rules apply everywhere: auth on connect, authorise membership, heart-beats, backoff with jitter, resync, backplane for scale.

Back to the README · See also Appendix B: Runnable sample project and Appendix C: Verification notes.

APPENDIX B

Appendix B: Runnable Sample: Real-Time Shared To-Do Lists (Spring Boot + STOMP + TypeScript/React Native)

A small but **production-shaped** project that puts the book's patterns into working, tested code. Companion to The WebSocket + STOMP Book.

```
examples/stomp-todo/
├── server/   Spring Boot 4.1 (Spring Framework 7, Spring Security 7, Java 21): STOMP over WebSocket
├── client/   TypeScript client for web / Node / React Native + React hook + runnable demos
├── docker-compose.yml + rabbitmq/   RabbitMQ with the STOMP plugin (for relay mode)
└── README.md (this file)
```

What it demonstrates (and where the book explains it)

Feature	Where in the code	Book chapter
STOMP endpoint, `/app` + `/topic` + `/queue` + `/user` prefixes	`server/.../ws/WebSocketConfig.java`	5
Simple broker **with heart-beats** (TaskScheduler), SpEL selector disabled	`WebSocketConfig#configureMessageBroker`	4, 5, 8
Broker relay switch (`app.broker.mode=relay`) incl. multi-instance user destinations	same	7
JWT on the CONNECT frame, token expiry enforced on later frames	`ws/JwtConnectInterceptor.java`	8
Message authorisation, **deny by default**, without the CSRF-on-CONNECT of `@EnableWebSocketSecurity`	`WebSocketConfig#messageAuthorizationManager`	8
Per-list membership checks on SUBSCRIBE (IDOR protection)	`ws/SubscriptionGuard.java`	8
Per-session **rate limiting**	`ws/RateLimitInterceptor.java`	8
Clean, non-leaky **STOMP ERROR frames**	`ws/CustomStompErrorHandler.java`	5, 8
`@SubscribeMapping` snapshot, `@MessageMapping` commands, `@SendToUser(broadcast=false)` acks	`ws/TodoWsController.java`	5
Business errors as MESSAGEs on `/user/queue/errors` (connection survives)	`ws/WsExceptionHandlers.java`	4, 5
Per-list sequence numbers, **idempotency keys**, **optimistic versions**, bounded event log	`todo/TodoService.java`	9, 11
REST **snapshot + resync** endpoints (`410 Gone` → reload)	`web/ApiController.java`	9
Presence (multi-tab aware, idempotent disconnect handling)	`ws/PresenceTracker.java`	5

Feature	Where in the code	Book chapter
Transport limits (size, send buffer/time, time-to-first-message)	`WebSocketConfig#configureWebSocketTransport`	5, 9
Integration tests with a real `WebSocketStompClient` (11 tests)	`server/src/test/...`	10
Client: one connection, token refresh, heart-beats, **exponential backoff + jitter**, re-subscribe on reconnect	`client/src/TodoRealtimeClient.ts`	3, 6
Client: **subscribe-then-snapshot**, duplicate dropping, **gap detection + resync**, command outbox + acks	same	9
React / React Native hook	`client/src/useTodoList.ts`	6

Prerequisites

- Java 21+, Maven 3.9+ (server)
- Node.js 22+ (client demos use the built-in `WebSocket` and `fetch`)
- Docker (only for relay mode)

Run it

BASH

```bash
# 1) server
cd examples/stomp-todo/server
mvn spring-boot:run            # http://localhost:8080, STOMP endpoint ws://localhost:8080/ws

# 2) client demo (another terminal)
cd examples/stomp-todo/client
npm install
npm run demo
```

Demo users: **anna** and **bob** share list **L1**; **carol** owns **L2** only. Tokens come from a **dev-only** endpoint (`POST /api/dev-token {"user":"anna"}`), which you disable with `app.dev-tokens=false`.

Output from a real run of `npm run demo`:

```
bob status: live
bob sees L1 @seq 0: (empty)

--- anna adds two items
bob sees L1 @seq 1: [ ] Buy milk (v1)
bob sees L1 @seq 2: [ ] Buy milk (v1) | [ ] Walk the dog (v1)

--- anna completes "Buy milk"
bob sees L1 @seq 3: [x] Buy milk (v2) | [ ] Walk the dog (v1)

--- anna retries the same command twice (same clientRequestId)
bob sees L1 @seq 4: [x] Buy milk (v2) | [ ] Walk the dog (v1) | [ ] Only once please (v1)
first ack duplicate=false, second ack duplicate=true

--- anna edits with a stale version → CONFLICT (connection stays open)
anna got: CONFLICT
```

```
--- carol (not a member) tries to read L1 over REST
carol GET /api/lists/L1 → HTTP 403
```

Resilience demo (reconnect, re-subscribe, resync)

BASH

```bash
npm run resilience      # then restart the server when it says so
```

In a real run, after the restart bob went `reconnecting` → (backoff 2.2 s, then 4.4 s) → `live`, re-subscribed, noticed the server's sequence went **backwards** (in-memory state lost), reloaded the snapshot, and then received the new item live.

Tests

BASH

```bash
cd server && mvn test     # 11 end-to-end tests: auth, forged tokens, snapshot, broadcast order,
idempotency,
                          # conflicts, validation, forbidden subscriptions, spoofed topic publishes,
REST resync
cd ../client && npm run typecheck
```

Use it from this React Native app

TSX

```tsx
// App.tsx (sketch)
import { TodoRealtimeClient } from './examples/stomp-todo/client/src/TodoRealtimeClient';
import { useTodoList } from './examples/stomp-todo/client/src/useTodoList';

const client = new TodoRealtimeClient({
  baseUrl: 'http://10.0.2.2:8080',      // Android emulator → your machine. iOS simulator:
http://localhost:8080
  getToken: () => auth.getFreshToken(),  // your auth
  reactNative: true,                     // stompjs NULL-byte workarounds (forceBinaryWSFrames,
appendMissingNULLonIncoming)
});
client.start();                          // after login; client.stop() on logout

function ListScreen() {
  const { state, status, add, toggle } = useTodoList(client, 'L1');
  // render state.items, show `status` ("reconnecting", ...) in the UI
}
```

Add `@stomp/stompjs` to the app's dependencies. Android 9+ and iOS block cleartext `ws://` by default; use `wss://` or a dev-only exception. Reconnect on `AppState` `active`; use push notifications for background delivery (Chapter 6).

Relay mode (RabbitMQ)

BASH

```bash
cd examples/stomp-todo
docker compose up -d                        # RabbitMQ + STOMP plugin on 61613, UI on 15672
cd server
mvn spring-boot:run -Dspring-boot.run.arguments="--app.broker.mode=relay --app.broker.relay.client-
login=stomp --app.broker.relay.client-passcode=stomp-dev-password --app.broker.relay.system-login=stomp
--app.broker.relay.system-passcode=stomp-dev-password"
```

Honesty note: the relay configuration compiles and follows Spring's documentation, but it was **not executed** while writing this sample (no Docker daemon was available). Simple-broker mode was fully tested. Also note: with RabbitMQ, Spring's `/user/queue/...` destinations become per-session queues that RabbitMQ makes **durable and not auto-deleted** by default. For production, switch to auto-delete destinations (e.g., `/user/exchange/amq.direct/acks`) or add an expiry policy (Chapter 7 §7.5.3).

Things deliberately left simple (do these in production)

- **Storage is in memory.** Restarting loses data, and multiple server instances don't share it. Use a database; publish events after commit or through an outbox (Chapter 9).
- **HS256 dev tokens.** Validate tokens from your identity provider (issuer URI / JWKS) instead, and remove the dev-token endpoint.
- **Presence is per instance.** Use Redis with TTLs in a cluster.
- **Allowed origins** are localhost only (`app.allowed-origin-patterns`).
- Add metrics (session gauges, executor queues, close codes), alerts and load tests (Chapter 9).

Two production gotchas this sample uncovered (now in the book)

1. `setPreserveReceiveOrder(true)` **swallows inbound interceptor exceptions** (Spring Framework 7.0.x, checked in `OrderedMessageChannelDecorator`): a rejected CONNECT or SUBSCRIBE is only logged. The client gets **no ERROR frame and the connection stays open**, so the client just hangs. This sample therefore doesn't enable it. See Chapter 10, bug C10.
2. **stompjs** `beforeConnect` **must never throw**: stompjs 7.x awaits it without a try/catch, so a failing token fetch (e.g., auth server down) becomes an unhandled rejection and **silently stops the reconnect loop**. Also, with Node's WebSocket a connection attempt can get stuck; `discardWebsocketOnCommFailure: true` lets the reconnect continue. See Chapter 10, bugs G11 and G12.

APPENDIX C

Appendix C: Verification Notes and Sources

This appendix records **how the facts in this book were checked**, so you know what to trust and what to re-check for your own versions. Verification pass done in October 2026.

How it was checked

The rendered documentation sites (docs.spring.io, stomp.github.io, rabbitmq.com) weren't reachable from the environment used to write this book, so the facts were checked against **primary sources one level deeper**:

- the **source code** of Spring Framework (`main` branch, version 7.1.0-SNAPSHOT at the time), Spring Security, Spring Boot and Apache Tomcat,
- the **AsciiDoc sources** of the Spring Framework and Spring Security reference documentation (the same text that's published on docs.spring.io),
- the **RabbitMQ documentation sources** (rabbitmq-website repository),
- the published `@stomp/stompjs` **7.5.0** npm package (its compiled source),
- web searches for cloud provider limits and industry announcements.

Facts confirmed against source code / docs sources

Fact (chapter)	Confirmed in
Simple broker heart-beats need a `TaskScheduler`; default becomes `10000,10000` once set; `@Lazy` hint to avoid cycles (Ch 5)	`SimpleBrokerMessageHandler` javadoc + reference doc "handle-simple-broker"
Simple broker read check = `max(client cx, server sy) × 3`, write = `max(client cy, server sx)` (Ch 4)	`SimpleBrokerMessageHandler.SessionInfo`, `HEARTBEAT_MULTIPLIER = 3`
Simple broker supports "only a subset of STOMP commands (no acks, receipts, …)" (Ch 4/7)	Reference doc "handle-broker-relay"
Spring sends a RECEIPT for a DISCONNECT that carries a `receipt` header (Ch 4)	`StompSubProtocolHandler#getDisconnectReceipt`
Relay defaults: logins `guest`, system heart-beats 10000/10000, reconnect every 5 s, "Message broker not active" error, read check ×3 (Ch 7)	`StompBrokerRelayMessageHandler`
Relay needs `reactor-netty` + `netty-all` (Ch 5/7)	Reference doc "handle-broker-relay"
Transport defaults: 64 KB message, 512 KB send buffer, 10 s send time, 60 s time to first message (Ch 5)	`WebSocketTransportRegistration` javadoc
Slow session closed with `CloseStatus.SESSION_NOT_RELIABLE` = **4500**; overflow strategies `TERMINATE` (default) / `DROP` (Ch 2/9)	`CloseStatus`, `ConcurrentWebSocketSessionDecorator`

Fact (chapter)	Confirmed in
Sessions closed with `GOING_AWAY` (1001) on shutdown (Ch 7)	`SubProtocolWebSocketHandler#stop`
Channel thread pools default to 2 × processors; `maxPoolSize` ineffective with unbounded queue (Ch 9)	Reference doc "configuration-performance"
`ChannelRegistration.executor(Executor)` since **6.2** (virtual threads) (Ch 9)	`ChannelRegistration`
`setPreservePublishOrder` / `setPreserveReceiveOrder` semantics (Ch 5/9)	Reference doc "ordered-messages"
`@Order(Ordered.HIGHEST_PRECEDENCE + 99)` for the CONNECT auth interceptor (Ch 8)	Reference doc "authentication-token-based"
`SessionDisconnectEvent` may be published more than once; be idempotent (Ch 5)	Reference doc "application-context-events"
RabbitMQ auto-delete user queues via `/exchange/amq.direct/...`; `userDestinationBroadcast` (Ch 7)	Reference doc "user-destination"
`@SendToUser` without a user falls back to the session id with `broadcast=false`; else `MissingSessionUserException` (Ch 10/12)	`SendToMethodReturnValueHandler`
`MappingJackson2MessageConverter` deprecated for removal in 7.0; `JacksonJsonMessageConverter` since 7.0 (Ch 5/6)	Source of both classes
`WebSocketStompClient`: heart-beat `0,0` until `setTaskScheduler`, then `10000,10000`; `connectAsync`; receipt timeout 15 s; consider disabling heart-beats when simulating many clients (Ch 6/9)	`WebSocketStompClient` + reference doc "client"
`@EnableWebSocketSecurity` requires CSRF on CONNECT; CSRF "not configurable" with the annotation; manual configuration alternative (Ch 8)	Spring Security reference doc "websocket"
`AbstractSecurityWebSocketMessageBrokerConfigurer` / `MessageSecurityMetadataSourceRegistry` **removed in Spring Security 7** (Ch 8/11)	Spring Security reference doc "websocket"
Spring Security secures only the inbound channel (Ch 8)	Spring Security reference doc
Allowed-origins default = same origin since Spring 4.1.5 (Ch 5/8)	Spring Security reference doc
Tomcat WebSocket text/binary buffers default **8 KB** (Ch 2/5)	Tomcat `Constants.DEFAULT_BUFFER_SIZE`, `WsWebSocketContainer`
Spring Boot `server.tomcat.max-connections` default **8192**, accept count 100 (Ch 10)	Spring Boot `TomcatServerProperties`
stompjs 7.5.0 defaults: `reconnectDelay` 5000, LINEAR/EXPONENTIAL modes (no jitter), `maxReconnectDelay` 15 min, heart-beats 10000/10000, `heartbeatToleranceMultiplier` 2, `heartbeatStrategy` Interval/Worker, `connectionTimeout` 0, `maxWebSocketChunkSize` 8 KB, RN workarounds, `TypeError: There is no underlying STOMP connection` (Ch 6)	`@stomp/stompjs` 7.5.0 package source
RabbitMQ: STOMP port 61613; Web STOMP 15674 `/ws`; `/queue/` queues durable, non-exclusive, non-auto-delete by default; `/topic` subscribe queues auto-delete non-durable; durable subscriptions via `durable:true` + `auto-delete:false` (Ch 7)	RabbitMQ docs sources (stomp.md, web-stomp.md)

Fact (chapter)	Confirmed in
WebSocket handshake Accept example `s3pPLMBiTxaQ9kYGzzhZRbK+xOo=` and masking example bytes (Ch 2)	Recomputed with OpenSSL/Python

Behaviour verified by running the Appendix B sample (Spring Boot 4.1.1 / Spring Framework 7.0.9 / Spring Security 7, stompjs 7.5.0, Node 22)

Behaviour	How it was observed
JWT CONNECT auth, deny-by-default message rules, SUBSCRIBE membership checks, spoofed `/topic` SEND rejected	11 passing integration tests with `WebSocketStompClient`
ERROR frame is followed by a server-side close (code 1002)	`StompSubProtocolHandler#sendToClient` / `sendErrorMessage` + tests observing disconnects
`setPreserveReceiveOrder(true)` → interceptor exceptions only logged, no ERROR, connection stays open	Failing tests, then `OrderedMessageChannelDecorator` source (7.0.9 and `main`)
stompjs `beforeConnect` rejection stops reconnecting and is unhandled	Server-restart test + stompjs `_connect()` source
Connection attempt stuck in CONNECTING after timeout is recovered by `discardWebsocketOnCommFailure`	Server-restart test with debug logs
Exponential reconnect backoff doubles (2.2 s → 4.4 s), re-subscription, snapshot reload on sequence regression	Server-restart test
Idempotent retries, ordered per-list `seq`, CONFLICT as MESSAGE with connection kept open, REST 403 for non-members	`npm run demo` against the live server

Facts confirmed by web search (secondary sources; re-check provider docs for your account/region)

Fact	Notes
ingress-nginx retired March 2026 (announced Nov 2025; reaffirmed Jan 2026)	Kubernetes blog posts
AWS ALB idle timeout default 60 s, range 1–4000 s	AWS docs (translated pages), re:Post
AWS API Gateway WebSocket: 2 h max duration, 10 min idle, 128 KB message, 32 KB frame; 1009 on oversize	AWS quota table
GCP HTTPS LB backend service timeout (default 30 s) acts as WebSocket max lifetime	Community + docs quotes
Cloudflare closes WebSockets idle for ~100 s (Enterprise can adjust)	Community/support reports

Not independently re-verified in this pass

- The exact wording of the STOMP 1.2 specification (the stomp.github.io source wasn't reachable). Key protocol rules (heart-beat formula, ack modes, header escaping and its CONNECT/CONNECTED exemption) are consistent with the Spring and stompjs implementations above and with spec quotations found in search results.
- RFC 6455 text (well established; examples recomputed).

- Browser-specific behaviour (bfcache, SharedWorker support, background throttling) changes often. Test your target browsers.

Sources

- Spring Framework source and reference docs: https://github.com/spring-projects/spring-framework (published at https://docs.spring.io/spring-framework/reference/web/websocket.html)
- Spring Security WebSocket docs: https://github.com/spring-projects/spring-security (published at https://docs.spring.io/spring-security/reference/servlet/integrations/websocket.html)
- Spring Boot source: https://github.com/spring-projects/spring-boot
- Apache Tomcat source: https://github.com/apache/tomcat
- `@stomp/stompjs`: https://www.npmjs.com/package/@stomp/stompjs and https://stomp-js.github.io/
- RabbitMQ STOMP docs: https://www.rabbitmq.com/docs/stomp and https://www.rabbitmq.com/docs/web-stomp
- STOMP 1.2 specification: https://stomp.github.io/stomp-specification-1.2.html
- RFC 6455: https://www.rfc-editor.org/rfc/rfc6455
- Kubernetes ingress-nginx retirement: https://kubernetes.io/blog/2025/11/11/ingress-nginx-retirement/ and https://kubernetes.io/blog/2026/01/29/ingress-nginx-statement/
- AWS API Gateway WebSocket quotas: https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-execution-service-websocket-limits-table.html
- AWS ALB attributes: https://docs.aws.amazon.com/elasticloadbalancing/latest/application/edit-load-balancer-attributes.html

Back to the README.